FINASTRA

Lending

TRADE INNOVATION 2.11

SDK - Application Extension Guide

Version 1.0

Finastra Technical Publications
May 2025

INNOVATING FINANCE TOGETHER

Version History

Date	Version	Change Reference
29/05/2025	1.0	Initial Version

CONTENTS

VERSION HISTORY ..... **IV**
PREFACE ..... **VIII**
Screen Examples ix
Document Conventions ix
INTRODUCTION **1**
Customisation Functionality 1
Elements of a Customised System 2
AT RUN-TIME **3**
Master Records and Events 3
Linked Events 4
Workflow 4
Global Processing 4
Postings in a Customised System 5
Posting Principles 5
Generation of Postings 5
Charge-related Postings 5
Postings Defined for Each Event 6
Financing and Charge Accruals 6
Netting of Postings 7
Netting of Funds Movements 8
Bulking 8
Adding Data to a Posting 8
Viewing Data Added to Postings 10
Event Fields Available 10
Released Postings 10
Posting Identifier 10
CREATING A CUSTOMISATION PROJECT **13**
Setting up Eclipse 13
Registering a Software Bundle 13
Creating a New Project 14
USING THE CUSTOMISATION EDITOR **16**
Overview Tab 18
Variants 19
User defined codes 20
Service definitions 20
Extensions Tab 22
Adding a New Field 23
Adding a Table of Fields 27
Field Groups Tab 30

Layout Tab 35
Associations Tab 40
Context Layout design Tab 44
Open API Tab 46
Static Tables Tab 49
Snapshots Tab 51
Customisation.xml Tab 53
Opening an Existing Cutomisation Definition 53
Version 1.6.2 53
ENHANCING CUSTOMISATION 57
Overview 57
Resolver 59
Extension Handler 59
Initialisation 59
Validation 60
Calculated Event Field Retrieval 60
Saving 61
Posting Related 62
Access Methods 64
Driver Wrapper 64
Accessing Static Tables Directly 67
Extension Pane 68
Managing Control State 68
Action Methods 69
Extension View Web Pane 69
Action Methods 70
Embedded Views and UI Services 70
URLEmbeddedView 70
ContentEmbeddedView 71
UIService 71
Displaying Results 72
Security 72
Initialisation Class 73
External System Interaction 73
Calling an External System 73
Message Processing 74
Handling Exceptions 76
BUILDING YOUR CUSTOMISED COMPONENTS 77
Generating the Code 77
Deploying the Customisation 77
Database Scripts 77
Worked Example 78

Enhancing a Table Upgrade 79
END OF DAY USER CONTROL ACTIONS 80
Implementing a User Control Action 80
Configuring the User Control Action 81
TRANSLATING AMOUNTS 82
Overview 82
Implementing the Plug-in 83
Configuring the Plug-in 83
Overriding Default Currencies Example 84
DEVELOPING A FAX INTERFACE 85
Overview 85
Before you Begin 85
Developing the FAX Plug-in 86
Configuring the Plug-in 87
CONTROLLING DOCUMENT WINDOWS 88
USING AN EXTERNAL DMS 89
How Trade Innovation Uses a DMS 89
The Java Interface 90
ContentRepositoryItem 90
Meta-Data 91
ContentRepository2 91
Registering the Implementation 93
Existing Implementations 94
Pre-2.6 94
Post 2.6 95
RoutingContentRepository 95
ARCHIVE DELETE 96
Implementation 96
APPENDIX A CONVERTING AN EXISTING CUSTOMISATION PROJECT 97
Customisation Definition 97
Hand Written Code 97

Preface

This guide describes how to add extra data fields to Trade Innovation by using customisation facilities.

There are also a few chapters covering some smaller extension points where Java programming skills alone are necessary.

This is not an end-user task and so this document is intended for technical consultants and bank staff. It assumes that you are familiar with the basic principles of trade finance and with Trade Innovation itself. Experience of programming is a requirement for understanding the more technical parts of the document.

The User Guide contains the following chapters and appendices:

Chapter	Description
Introduction	Introduces the customisation functionality and the elements and technology involved. It also introduces some terminology used throughout this guide.
At Run-time	Summarises the runtime features of Trade Innovation where extensions can be made.
Creating a Customisation Project	Explains how to create a customisation project.
Using the Customisation Editor	Explains how to use the customisation editor.
Enhancing Customisation	Explains how to enhance the default customisation with hand-written code.
Building Your Customised Components	Explains the steps needed to build and deploy customised code.
End of Day User Control Actions	Explains how to define an end of day user control action.
Translating Amounts	Explains how to create a plug-in to translate an amount into words for a particular language.
Developing a FAX Interface	Explains the approach on how to create a plug-in to interface to a FAX-sending system.
Controlling Document Windows	Explains the approach on how to override managing document windows.
Using an External DMS	Explains the approach on how to implement an external DMS interface.
Archive Delete	Explains the approach on how to delete transactions that are eligible for deletion.
Appendix A	Provides instructions on upgrading a customisation project from an earlier version of Trade Innovation.

Screen Examples

This guide includes examples of screens. The screens shown in your system may not look the same as the examples in this guide. However, the actual content and user operation are identical.

Document Conventions

Note: This style is used to indicate text that contains information of more than usual significance, such as warnings or hints in using the software.

Italicized font is used to distinguish terms, which have a specific meaning in the context of this document when they are introduced.

A `monospace` font is used to reference items such as file and folders names, artefacts, and code snippets.

Introduction

This chapter provides an overview of customisation functionality and the architecture and technology involved. It also introduces some terminology used throughout this guide.

Customisation Functionality

The customisation facilities allow you to define additional data fields, referred to in this guide as *extra fields*, as extensions to the following tables in the Trade Innovation zone database:

- Master
- Event
- EventStep
- Customer
- TICustomerDetails
- Postings

This is done by creating a customisation project in the Eclipse IDE tool using supplied plug-ins. One of these plug-ins is the Customisation Editor that allows you to define new fields for each of the extension points listed. You can also add new static data tables to the Trade Innovation zone database.

Note: You cannot use customisation functionality to extend part-payment, foreign exchange or departmental limit details.

So that users will be able to access the extra fields that you have defined for the *Master*, *Event*, *EventStep*, *Customer* and *TICustomerDetails* tables, the user interface has place-holders that will include appropriate customised layouts. Each function has an Extra Data section at the bottom of the page. If you have licensed the Advanced SDK, it is also possible to place customised fields in the sections on the page where the context is more appropriate. In this case it is also possible to add new sections relative to the existing sections.

In addition with the Advanced SDK it is possible to define linked or embedded views. Linked views are accessed via a menu option, whereas embedded views appear in their own sections of the page. These can be thought of as simplified portlets. The Trade Innovation server either calculates and serves the content, or a URL is derived for the browser to retrieve the information from an external system.

For extra fields held in new static tables that you might define, the Trade Innovation software in the Static Data application provides a standard user interface. It is not possible to alter these layouts.

Extra fields have associated event field codes, and so can be included in documents and used in postings sent to back office systems integrated with Trade Innovation. It can also be used in rules, allowing the standard transaction processing functionality delivered with the system to be modified through customisation (see the *System Tailoring User Guide – Trade Innovation*).

A customisation project is an extension of the standard eclipse java project, and once the definition of the extensions is complete, java code is generated to support the customisation. The basic code produced is sufficient to enable simple customisations (for the input and maintenance of extra fields) without any additional development being required. More complex customisations, involving either the extension of postings or the retrieval of Trade Innovation's own transaction data to generate or validate extra fields, can be implemented by extending the generated code.

Note: *EventStep* data is for viewing only, and the associated field codes are not available for rules and documents.

Elements of a Customised System

A customised system has the following components in addition to those delivered with Trade Innovation:

- Static definitions of the extension details
- Database tables to hold any extra static data introduced by your customisation
- Extra fields added to a specific set of existing tables
- JSPs that define the customised layouts
- Control data to drive the customised Static Data editor facility
- Message schema extensions to allow extra data to be supplied on appropriate service request/response messages
- Configuration to optionally add customisation fields to Open API messages
- Configuration to link to the generated/extended java code

At Run-time

Once the customisation project has been generated and deployed to a Trade Innovation application server, the extensions will be called as required as the application runs.

When the application server starts, a check is made for the presence of the customisation static definitions. If found then the zone is treated as customised.

During transaction processing, calls into the extensions are automatically made in the following circumstances:

- When defining or accessing Customer and/or Trade Innovation Customer details
- When accessing Master or Event data
- If you click the OK or Pend buttons on a page that contains Extra Data, thus saving data that may include data in extra fields
- If a posting is generated and extra fields have been defined for the Posting table
- If you click the Extra Data button when the system option *CustomisedDataInSeparatePane* is used in masters and events
- If the workflow orchestration includes an external verification step, the details returned from the external system can update the customised fields, and those details are shown when enquiring on the result of that step

Master Records and Events

Each business transaction that Trade Innovation processes, such as a letter of credit, is based around a *master record*. The master record is used to store the current status of a trade finance transaction and includes, for example:

- The current amount of the transaction
- Current liability balance
- Date of expiry
- Whether the record is active or inactive - for example, whether it has expired
- Other data of a transaction-specific nature

All trade finance transactions move through a defined life-cycle. A typical life-cycle would involve, for example:

- Creation or issuance
- Amendments
- Payment
- Expiry
- Book off

Each stage in the life-cycle is handled as an event within the life of the transaction. The first event creates the master record for the transaction. Each modification to the record, such as an amendment or a payment, is achieved by adding an additional event, and expiry and book-off are typically the last events that affect the content of the master record.

In a customised system, for copying of data between event and master tables to be performed, you must specify the relationship between any extra fields you add at event level, and the equivalent extra fields you add at master level. You can do this using the Customisation Editor, which will then generate the code necessary to perform the required copying.

Linked Events

All the data fields in an event can be categorised as being event-level or master-level.

Event-level data consists of data fields which are used within the particular event only.

Master-level data consists of data fields of a transactional nature, which are copied back to the master record at appropriate points during the data processing.

Where events are linked, it is important that the customised data is defined against both events to ensure it is available to be carried forward. Examples of linked events include:

- A Claim Received event, followed by Outstanding Claim event
- An Amendment event, followed by Beneficiary Response to Amendment event

Workflow

When an event is being entered, the workflow that is followed may include using an external verification step. When a step of this type starts, the configuration is used to derive a step-level message document to be created and sent. The event step is then waiting for an external response.

When the response is received, it may indicate that:

- the external verification process was successful, and the step can complete automatically
- the external verification process requires user intervention to decide whether to proceed or not
- the external verification process requires user intervention to reject back to an earlier step in the workflow to address an issue that needs to be resolved

As part of the response, extra data can be returned that will populate customised data for the *EventStep* extension point. When viewing the results of the step, the customised data can provide the context needed for what happened in the external system.

The message that is returned is `TI.TFEXRRSP`. To provide errors or warnings to Trade Innovation from the external system, the message returned will use a `DeferredResponse` format rather than a `ServiceRequest`. For more information refer to the *SDK Systems Interfacing Guide - Trade Innovation*.

Global Processing

When defining the customisation required for a particular product/event combination, by default that is the one that will be used for all main banking entities in a zone. It is also possible to define a grouping identifier – called a *variant* – that can be used as an extra element of the cross reference.

The group of customisations based on the combination of product/event/variant for a particular variant value can then be used by one or more main banking entities. This is enabled by setting the branch trade finance option *ExtensionVariantName* to the variant value for each applicable main banking entity. If there isn't a customisation defined for the product/event/variant combination, then the one for the product/event is used instead. If that is not defined, then no customisation will be applied to that event.

Note that this mechanism only applies to the master and event extension points. Customisation for postings and customer details are the same for all main banking entities.

Postings in a Customised System

This section discusses key points relating to extending postings generated by the system.

Note: You must have a full understanding of the different types of postings that are generated by Trade Innovation, and decide how each posting type is to be affected before attempting to add extra fields. For details of the posting types generated see the *Business Reference Guide - Trade Innovation*.

Posting Principles

The postings generated by Trade Innovation are in two sets:

- User-defined postings, which are set up using the system tailoring application. These are generated and released as part of event processing
- Internally generated postings, arising from charge and financing details entered for a transaction. Postings of this type are generated as events are released, and subsequently during end-of-day accruals processing

These areas are covered in more detail in the following sections.

Generation of Postings

The postings generated by an event are essentially of two types:

- Postings without any associated settlement details
- Funds movements postings, where the posting has attached settlement instructions

Where a posting is to a charge income account, SP account or an account type which is not valid for funds movement the system generates a debit or credit posting to an account for the appropriate amount and value date.

Where a posting is a charge settlement posting or is defined as being an Event Party Settlement posting it will additionally have attached settlement instructions derived from the standing settlement instructions and master settlement instructions held for the party. These postings are used to:

- Project future funds movements. Where the value date is in the future the posting details are passed to the back office on a projection basis immediately the event is released. This allows the details of future funds movements to be included in nostro positions
- Post the projection on due date. When the value date is reached the system passes the posting to the back office to be posted
- Generate pay and receive advices to be sent to the account/nostro being debited

When considering this in the context of adding extra data to postings, it may be that a posting needs to be extended to include additional information to allow for the funds movement being passed to the back office as both a projection and then subsequently as a posting.

Charge-related Postings

Postings are generated as a result of taking charges, either standard or periodic. At the simplest level, a charge results in a debit to the charge payer and a credit to a profit and loss account. Depending upon the charge type and system options defined, Trade Innovation may also:

- Post to tax accounts
- Generate postings to income receivable accounts for periodic charges
- Convert charge amounts to a booking currency through exchange accounts
- Split fees between different branch profit and loss accounts

- Bill charges

Consequently, you should consider each different type of charge posting and determine what extra data it may require. It may be that the same extra data is required on all postings generated by a charge; alternatively, only some of the postings generated may require extra data. For example, the charge code may be added to all postings, but the extra fields you have added to the Customer table (for example) may only be required on the charge settlement postings.

Postings Defined for Each Event

Trade Innovation generates postings according to the posting definitions set up against the product and event. (Postings generated by the Manual Bookkeeping event are not set up using posting definitions, but entered directly as part of each event).

Postings are generally specified at event level, and use details from the event to generate the appropriate accounting. For example, postings can be set up for:

- The Import Letter of Credit Issue event, to record LC issuance liability and contra liability
- The Finance Import LC event, to record the deal and to post interest received in advance
- The Repay Fin event, to record the repayment of principal and interest

The posting definitions reference event fields, which provide information held for the event or master. These event fields (including customised event field) are also available to allow additional data to be added to postings by extending the generated customised code.

Postings can also be generated for each of the following attachments entered within the event:

Attachment	Description
Foreign exchange deals	Exchange deals entered as part of settlement
Collateral	Details of collateral recorded against the master
Part payments	Individual payments made under a Claim Received, Documents Presented or collection order Payment event
Disbursement	Payments under assignments, transfers and back-to-back credits
Assignments	Details of assignments
Advances	Finance deals with interest at maturity (for systems where the bills financing module is not installed)
Discounts	Finance deals with interest at inception (for systems where the bills financing module is not installed)
Repayments	The repayment of advance deals, for example trust receipt loans, import financing or outward collection financing (for systems where the bills financing module is not installed)

Trade Innovation's posting definitions reference event fields specific to these attachments, which provide information held for the attachment. These attachment-level event fields are also available (in addition to the master or event fields) to allow additional data to be added to postings by extending the generated customised code.

Financing and Charge Accruals

Accrual postings are generated as a result of the daily periodic charge accrual and bills financing accrual processes. Trade Innovation will for example:

- Transfer income to profit and loss on a daily basis for periodic charges
- Amortise interest received in advance for a finance deal

- Accrue interest or past due interest for a finance deal

As these postings are not associated with a specific event there is no specific event-level information. However, the event field mechanism can be used to obtain information about the master, finance interest or charge interest accrual details, which can then be added to the accrual postings by extending the generated customised code.

Netting of Postings

Once the individual postings (also known as raw postings) are generated for an event, the system attempts to net postings to the same account together to produce netted postings. For example, Trade Innovation will net all charge debits to the same account, resulting in a single debit rather than a series of individual debits.

Postings will be netted together providing that the following are the same:

- Account (customer account/system parameter/charge account)
- Currency
- Value date
- Customer ID
- Reference
- User codes
- Party
- Related party
- Team

It is the netted posting that is passed to the back office upon release of the event, not the individual raw postings.

The raw postings will continue to be netted together under customisation where the extra data is the same on each posting. Trade Innovation compares the data in the standard posting fields and in the extra data on each posting during the netting process. If for example an extra field is added at master level which is then applied to all postings, netting will continue to work as at present. Alternatively, the generated code can be overridden to achieve the required netting.

Typically, where information is added to the raw posting from the attachment this will result in the raw postings not being netted, resulting in a netted posting for the raw posting amount.

For example, without customisation the system would typically net together a charge settlement posting, a discount interest posting and finance deal posting. If the same master-level extra field was added to all postings, netting would continue to net the raw postings together.

Type	DR/CR	Posting Details	Extra posting field Col 1
Charge	Dr	Common details	Master code
Discount	Dr	Common details	Master code
Finance	Cr	Common details	Master code

However, if customisation were to add the charge code to the charge posting, the discount interest rate to the interest posting, and the term to the deal posting, this would result in three individual postings being passed to the back office.

Type	DR/CR	Posting Details	Extra posting fields Col 1	Col 2	Col 3	Col 4
Charge	Dr	Common details	Charge code			
Discount	Dr	Common details		Rate		
Finance	Cr	Common details			Start	Maturity

Netting of Funds Movements

Raw postings are also netted into netted funds movements on a similar basis, except that the settlement details are included in the netting criteria. These details are displayed in the Settlements window. This netting cannot be changed.

Netted funds movements are used by the system to generate pay and receive advices via SWIFT or Mail.

Bulking

The general ledger service used by the associated main banking entity determines whether accruals are passed separately or bulked to the same accounts.

If there is no bulking the system passes individual accrual postings.

If your system is set to bulk, Trade Innovation bulks together accrual postings with the same characteristics. In the bulking, accrual postings are bulked together where the following are the same:

- Branch
- SP or SK
- Currency
- Transaction code
- Related customer
- Related customer - customer type
- Related customer - analysis code
- Related customer - parent country
- Related customer - team

The accrual postings will continue to be bulked together under customisation where the extra data is the same on each posting. The system compares the data in the standard posting fields and in extra fields on each posting during the bulking process. The generated code can therefore be overridden to achieve the required bulking. For example, the code could be extended to:

- Never bulk accrual postings
- Bulk according to different criteria (in which case the required data on the resulting bulked posting would need to be set up according the relevant business rules)

Adding Data to a Posting

Each time a raw posting or an accrual posting is generated, the system allows data to be added according to the type of posting, as defined above by extending the default extension handler.

There are two methods which can be used to add extra data to a posting:

- Add extra data to posting. This will typically be used where some general identifier is being added to a raw posting
- Add extra data to settlement posting. This adds data to netted postings and will typically be used where attributes of the account (such as account type) need to be cross-referenced to determine what data is to be added

Note: This is particularly important for settlement postings where the original raw posting party account may have been overridden via settlement maintenance.

Examples of the resulting postings are as follows:

DR/CR	Type	Account
Dr	Netted	Settlement A/C 2
Cr	Netted	SK P&L
Cr	Raw	SK P&L
Dr	Raw	Settlement A/C 1

(But when you are in `addExtraFields()` method you would see A/C 1, and if you were in `addExtraSettlementFields()` method you would see A/C 2 against individual postings).

The code section applicable to posting type can be enhanced to add extra event fields. At the point the posting is generated, the posting extension handler class is called, allowing any extra data required to be added, with the system also providing the necessary context of master, event, posting details (including account) and attachment. The event field mechanism can be used to obtain details from the related items and/or condition the setting of the required extra fields. When a posting is being processed, the following information can be obtained:

- The posting itself, for example:
  - Whether debit or credit
  - Back office account number, external account number and IBAN number
  - Amount
  - Currency
  - Transaction code
  - SP System parameter the account represents
  - SK Charge code the account represents
  - Category code that the SP/SK parameters represent
- The account to which the posting is to be made:
  - Number
  - Short name
  - Contingent flag
  - Internal flag
  - Valid for funds movement
- The party for whom the posting was made, for example:
  - Analysis code
  - Risk country
  - Plus extra fields from the Customer table
  - Plus extra fields from the TICustomerDetails table
- The master or event, for example:

  - All standard extra fields such as issue date and expiry date
  - Plus extra fields on the Master and/or Event tables
- The attachment itself:
  - FX deal - rate, base currency equivalent
  - Collateral code - code, description
  - Part payment - type, term, number of days

Viewing Data Added to Postings

Extra data added to postings can be seen in when viewing postings in the Postings window.

Uncheck the Netted check box and click **Refresh**. Then highlight a posting from those listed and click **View**.

Event Fields Available

The System Tailoring application's *General Business Functions|Browse and Output Field Codes* menu option provides a means to view the event fields that are available for the following:

- Product/events
- Events/business areas
- Part codes
- Postings

This allows the field codes used for both standard event fields and extra event fields to be identified.

See the *System Tailoring User Guide - Trade Innovation* for full instructions on using this menu option.

Released Postings

When an event is released, netted postings are copied to a set of staging tables for transfer to the relevant back office system. As part of this process, all the extra fields added to a netted posting regardless of type are copied to an extension of the posting staging table.

Posting Identifier

Each posting generated by the system has a unique identifier. This posting code can be used in java to condition postings, or simply to confirm or verify that the correct posting has been identified.

The list of codes is as follows:

Posting	Identifier
SP101 posting for conversion of tax amount in charge currency	C001
SP101 posting for tax amount in charge booking currency	C002
SP145 tax accumulation account for tax amount in charge booking currency	C003
SP145 tax accumulation account for tax amount in charge currency	C004
SP101 for reimbursement charge due from seller/other - original charge amount	C005
SP101 for reimbursement charge due from seller/other - charge amount due to buyer	C006
Settlement of reimbursement of charges due to buyer from another party - charge amount - due to buyer	C007
Settlement of tax charge amount due from buyer	C008

Posting	Identifier
Settlement of tax charge amount due from seller	C009
Settlement of charge amount due from buyer	C010
Settlement of charge amount due from seller	C011
Settlement of charge amount due from other party (from an intermediary bank)	C012
Settlement of charge amount due from other party	C013
Reverse posting from SP for billed income when billed charge amount is taken	C014
Posting taken charge amount to SK P&L when billed charge amount is taken	C015
Reverse posting from customer billing account when billed charge amount is taken	C016
Settlement of charge amount due from billed customer	C017
Reverse posting from SP for billed income when billed charge amount is waived	C018
Reverse posting from SK P&L when billed charge amount is waived	C019
Reverse posting from customer billing account when billed charge amount is waived	C020
Settlement of charge amount due to another bank (OBC - Other Bank Charge)	C021
SP101 posting for charge amount in charge currency	C022
SP101 posting for charge amount in charge booking currency	C023
SK P&L for charge amount in charge booking currency	C024
SK P&L for charge amount in charge currency	C025
Split fee amount taken from the SK P&L account of the branch taking the main charge	C026
Split fee amount for the SK P&L account of the branch receiving the split fee	C027
Settlement of charge amount due from the charge payer	C028
SP101 posting for charge amount in charge payer currency	C029
SP101 posting for charge amount in charge receive currency	C030
Settlement of charge amount due to the charge receiver	C031
SP101 posting for periodic charge amount in charge currency	C032
SP101 posting for periodic charge amount in charge booking currency	C033
Posting of in advance periodic charge amount to SP620/SP621/SP622 in charge booking currency	C034
Posting of in advance periodic charge amount to SP620/SP621/SP622 in charge currency	C035
Settlement of in advance periodic charge amount due from the customer	C036
Reversal of periodic charge accrued amount to SP620/SP621/SP622 when in arrears charge is not taken	C037
Reversal of periodic charge accrued amount to SK P&L when in arrears charge is not taken	C038
Settlement of in arrears periodic charge amount due from the customer	C039
Posting of in arrears periodic charge amount to SP620/SP621/SP622	C040
Periodic charge accrual posting to SP620/SP621/SP622	C041
SK P&L for periodic charge amount in charge currency for behalf of branch	C042
Split periodic charge amount from the SK P&L account of the branch taking the main charge	C043

Posting	Identifier
Split periodic charge amount for the SK P&L account of the branch receiving the split fee	C044
Reverse posting from SP for defer charge when charge amount is taken	C045
Reverse posting from SK P&L for defer charge when charge amount is taken	C046
Reverse posting from customer defer charge account when charge amount is taken	C047
Reverse posting from SP for paid charge when charge amount is refunded	C048
Reverse posting from SK P&L for paid charge when charge amount is refunded	C049
Reverse posting from customer paid charge account when charge amount is refunded	C050
Settlement of tax charge amount due from other party (from an intermediary bank)	C051
Settlement of tax charge amount due from other party	C052
Balancing of Other Bank Charge due to value date difference	C053
Reverse future premium	C054
Reverse future premium contra	C055
Generate future premium	C056
Generate future premium contra	C057
Reverse pay arrears future premium	C058
Generate pay arrears future premium adjustment	C059
Finance P&L accrual for behalf of branch (In Advance/In Arrears/Past Due)	F001
Finance contra accrual for behalf of branch (In Advance/In Arrears/Past Due)	F002
Finance P&L accrual for input branch (In Advance/In Arrears/Past Due)	F003
Finance contra accrual for input branch (In Advance/In Arrears/Past Due)	F004
Finance participation P&L accrual for behalf of branch (In Advance/In Arrears/Past Due)	F005
Finance participation contra accrual for behalf of branch (In Advance/In Arrears/Past Due)	F006
Finance participation P&L accrual for input branch (In Advance/In Arrears/Past Due)	F007
Finance participation contra accrual for input branch (In Advance/In Arrears/Past Due)	F008
Finance auto interest settlement of interest due from customer	F020
Finance auto interest SP101 posting in settlement currency	F021
Finance auto interest SP101 posting in accrual currency	F022
Finance auto interest P&L for behalf of branch	F023
Finance auto interest suspense reversal for behalf of branch	F024
Finance auto interest receivable reversal for behalf of branch	F025
Finance auto interest P&L for input branch	F026
Finance auto interest suspense reversal for input branch	F027
Finance auto interest receivable reversal for input branch	F028
Manual posting	M001
User defined postings	U***

Creating a Customisation Project

To define customisation details, a new customisation project needs to be created. This is created using the Eclipse development environment with the help of supplied plug-in.

Setting up Eclipse

The customisation editor uses the Eclipse platform as a base. There are multiple versions available, and the one chosen should cover all of your needs. The standard, java or Java EE editions are recommended.

The latest main version of Eclipse as of writing this guide is Kepler (4.3.1), though the supplied software will work on versions from Helios (3.6.2) onwards.

For further information on setting up Eclipse, refer to the *SDK Common Facilities Guide - Trade Innovation*.

The following instructions assume that Eclipse is being run on Windows.

Copy the supplied jar files:

- com.misys.tiplus2.customisation_editor_<version>.jar
- com.misys.tiplus2.software_bundle_registration_<version>.jar

into the `eclipse/plugins` subfolder. Remove any older versions of these plugins.

Start Eclipse.

Note: In order to make sure the plugin behaves correctly after replacing an older version, eclipse must be started with the `-clean` parameter. This can be achieved by creating a shortcut to exlipse.exe, and modifying the command defined in the shortcut.

Registering a Software Bundle

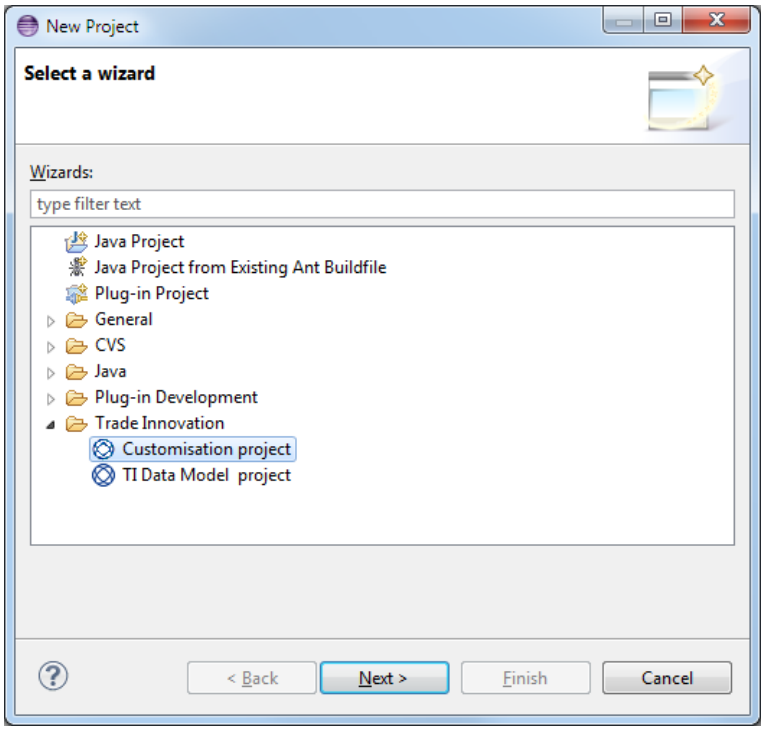
The customisation plugin relies on the software management plug-in that allows versions of Trade Innovation software to be registered as software bundles. For further information see the *SDK Common Facilities Guide - Trade Innovation.*

At least one software bundle must be registered before a customisation project can be created.

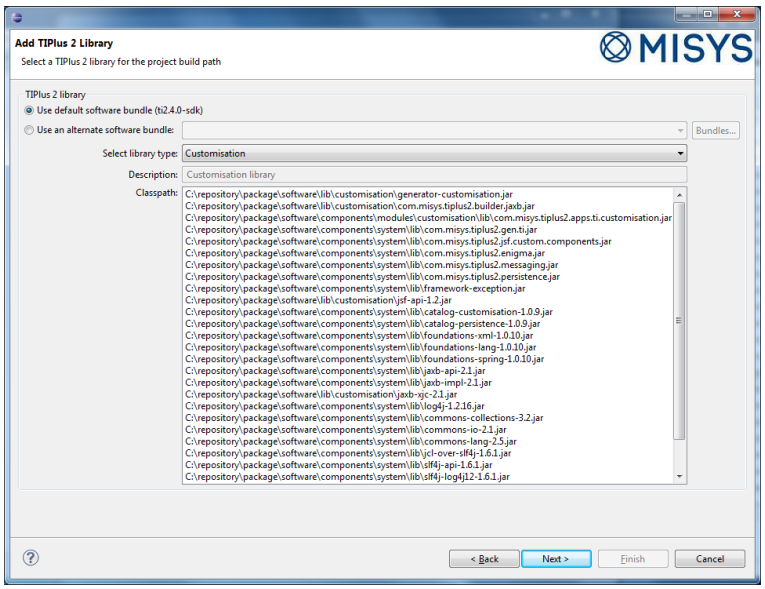
Creating a New Project

To create a new customisation project,

1. Select File|New|Project...:



2. Select the **Customisation project** entry under the **Trade Innovation** folder and click **Next**:

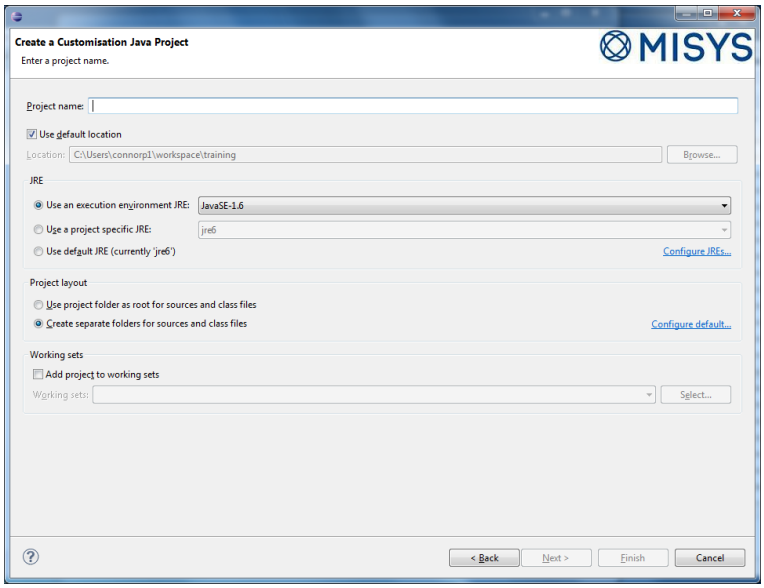


This panel allows you to select the software bundle to use with this project. This will initially reference the default bundle.

You can choose to use an alternate bundle that has been registered, or if the one you want to use has not yet been registered, click **Bundles…** to show the **Registered Software bundles** panel. The bundle can be registered, and on return will be used.

The library type will be pre-selected to be **Customisation**.

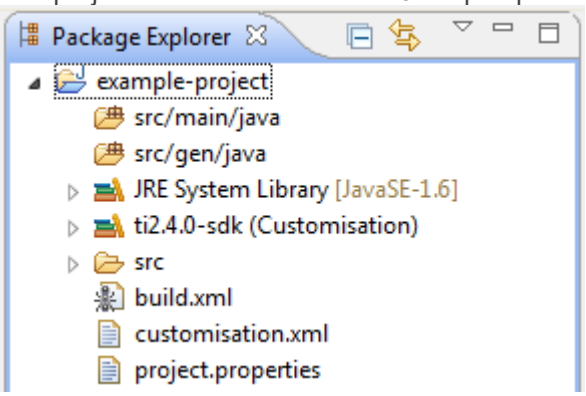
3. Click **Next**.



This panel is similar to the standard java project creation panel.

4. Enter a name for the project. The remaining information can retain their defaults.
5. Click **Next** and then **Finish**.

The project will be created. In the Java perspective, the package explorer will look like:



The first source folder `src/main/java` will contain any manually created classes.

The second source folder `src/gen/java` will contain all of the generated java classes.

During the generation process, further folders will be created under the `src/gen` folder.

The `build.xml` file is an ant script to create and deploy the customisation to the referenced software bundle.

The `customisation.xml` file will contain the customisation details required, and will be updated using the customisation editor which will be covered in the next chapter.

The `project.properties` file is used by the `build.xml` ant script.

Note: This project references the software bundle's customisation library, so all of the relevant jar files will be on the classpath when compiling any manually created or generated java.

If you change the software bundle after the project is created, the `project.properties` file contains the definition of the software bundle folder used by the build process. This will need to be changed manually.

Using the Customisation Editor

This chapter covers defining the required customisation using the customisation editor.

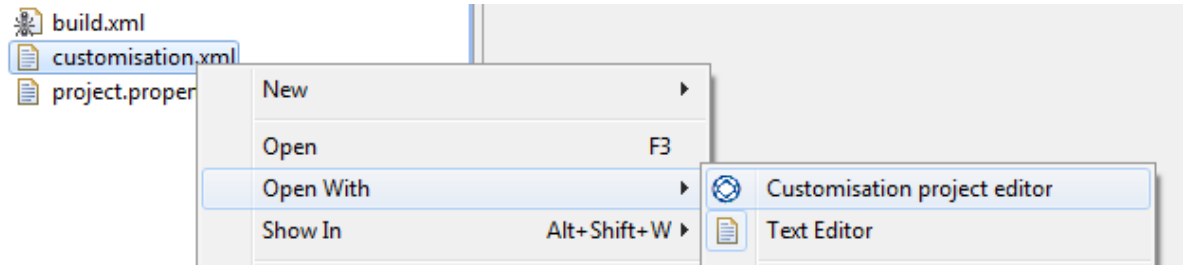
The approach taken to defining customisation details are as follows:

- define the version of the SDK that is licensed
- define the extra fields for a particular extension point (*Customer*, *Event* etc.)
- define field groups from those fields
- create common layouts for those field groups
- define linked views associated with the common layouts
- create associations between the common layouts and the context in which they will be used, or define context-specific layouts where the fields are positioned in appropriate sections on the page
- define linked or embedded views associated with the context-specific layouts
- define any new static data to be used

This is not necessarily a completely sequential process, but these are the steps involved, and may be repeated in order to build up the customisation required.

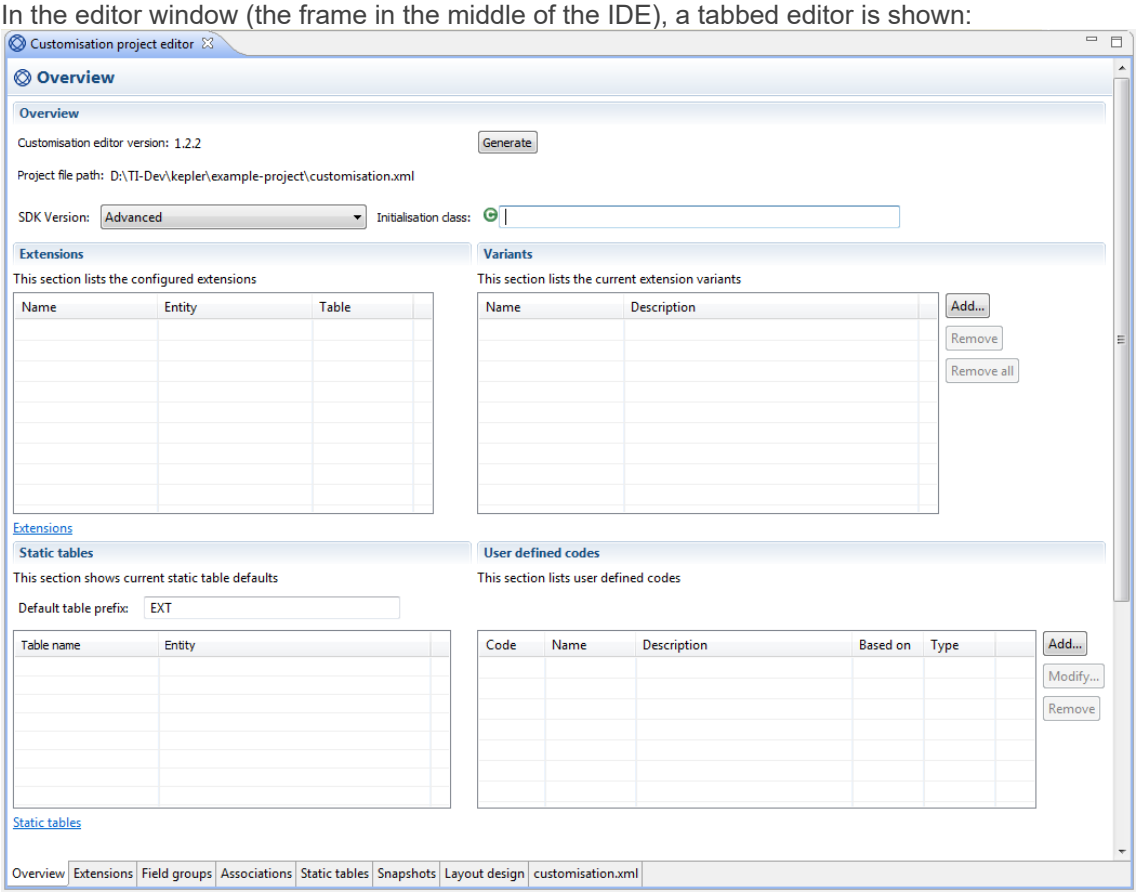
Note: Some of the features described are only available based on the version of the SDK licensed.

The editor is associated with the `customisation.xml` file, so when you double-click the file, the editor should be shown. Since the file is an xml file, it may be that by default in your setup, it will be opened in a generic XML editor. If this is the case, close any editor opened for this file, and use the context menu to select the customisation editor:



Once this has been selected, the editor will be re-opened by double clicking the `customisation.xml` file.

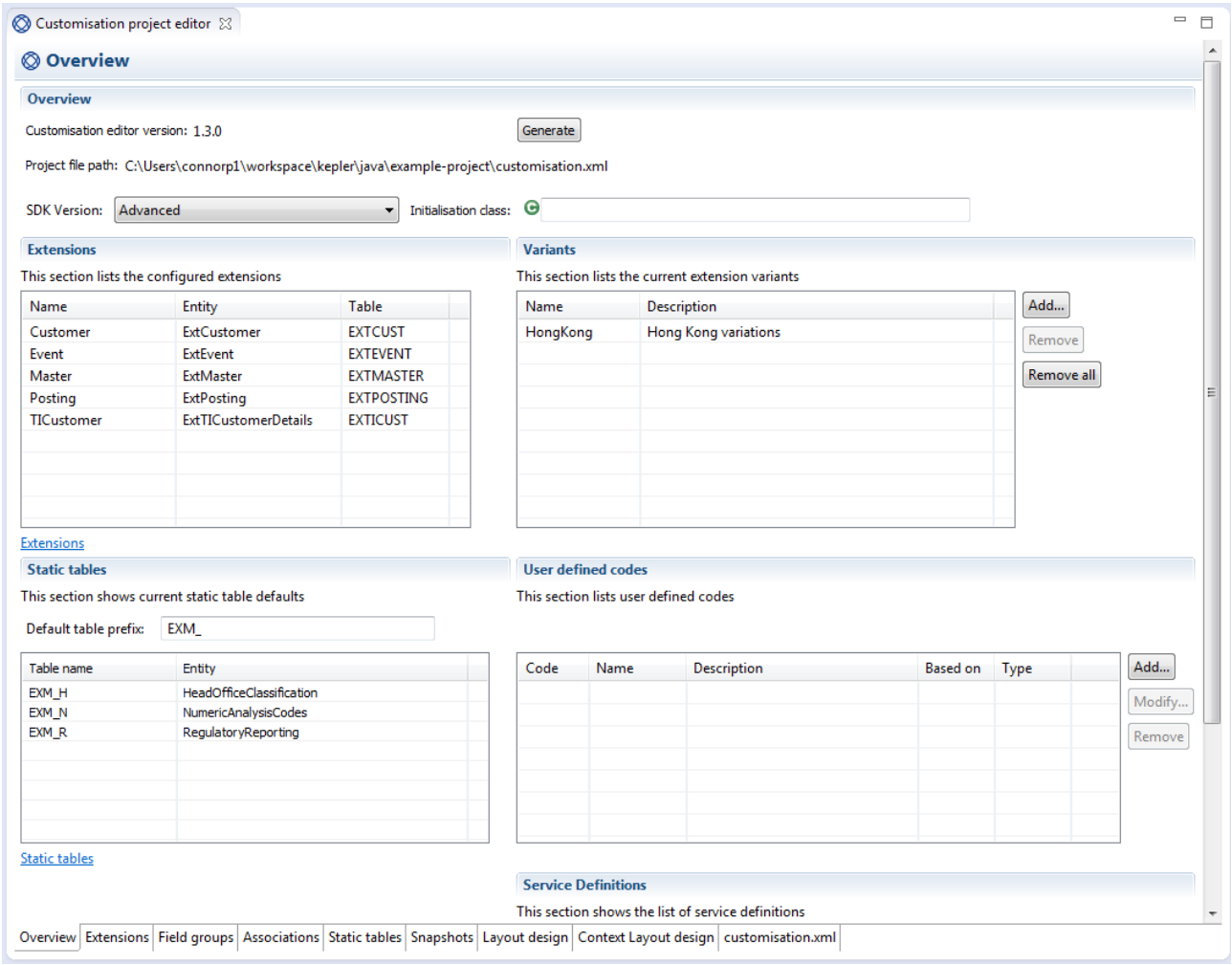
In the editor window (the frame in the middle of the IDE), a tabbed editor is shown:



This first tab allows you to declare the version of the SDK that has been licensed, as some features are not available based on the license used. While you can evaluate features of the various versions of the SDK, the customisation project used to deploy into a production system must be based on the version licensed.

The following sections cover the details provided on each tab.

Overview Tab



This first tab allows you to declare the version of the SDK that has been licensed, as some features are not available based on the license used. While you can evaluate features of the various versions of the SDK, the customisation project used to deploy into a production system must be based on the version licensed.

Also available to be defined on this tab:

- An initialisation class which will be created and called during startup and shutdown of the application server
- Variants for global processing support
- The default table prefix for static tables
- User defined codes – used for *EventStep*, allowing the step ids used in the Trade Innovation system to be defined in the customisation project, as they are used at runtime to identify the customised layout to use. This is only available with the Advanced SDK
- Service definitions – used as part of linked or embedded view definition. This is only available with the Advanced SDK

The **Extensions** section and **Static tables** list will be updated as the editor is used.

Finally the **Generate** button is used to re-generate the code after a change has been made.

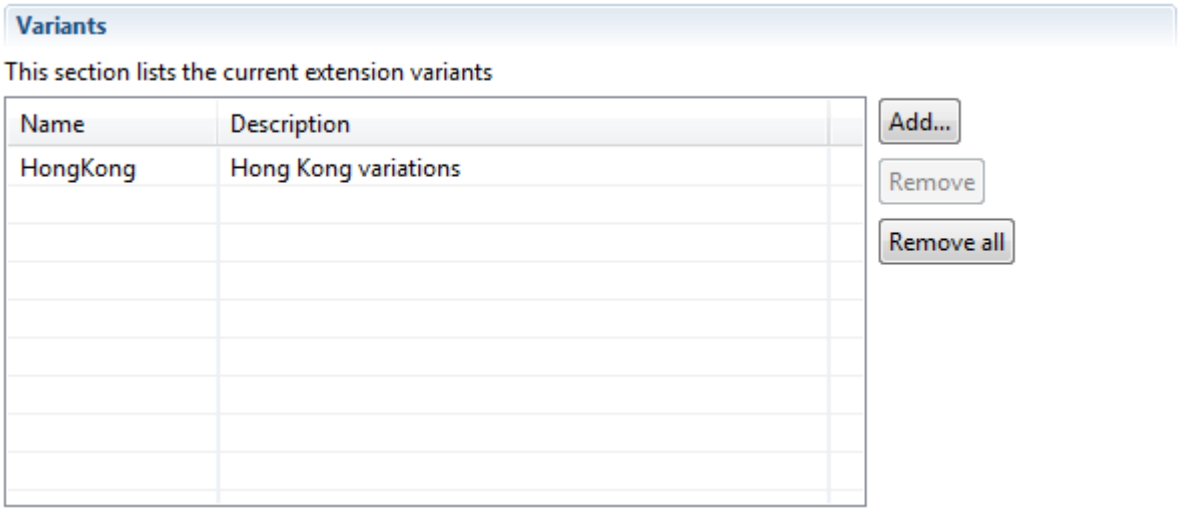
You can save the definition either by using `Ctrl+S` or selecting **File|Save**. Note that while you can choose to close the editor without saving any changes, changes cannot be undone. So if you are about to make any major changes, it is advisable to make a snapshot of the current state of the customisation file (see the later section).

Note: Some operations require the customisation.xml file to be autosaved.

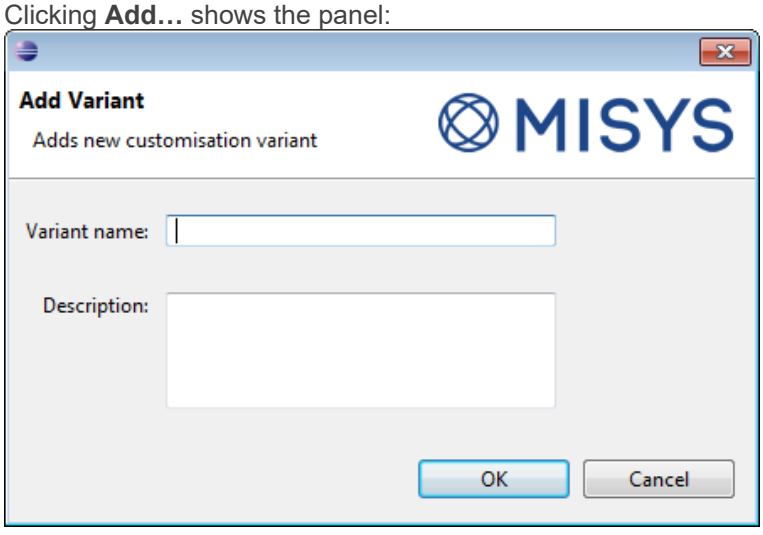
Variants

Variants allow for different customisation processing in different parts of the branch hierarchy. When defining customisation processing, it is in the context of a major and minor code. For events, these codes correspond to product and event codes. Variants can only be used for *Event* and *Master* extensions.

If this processing needs to vary, then adding a variant allows that processing to be specified in context of major, minor and variant.



Clicking **Add…** shows the panel:

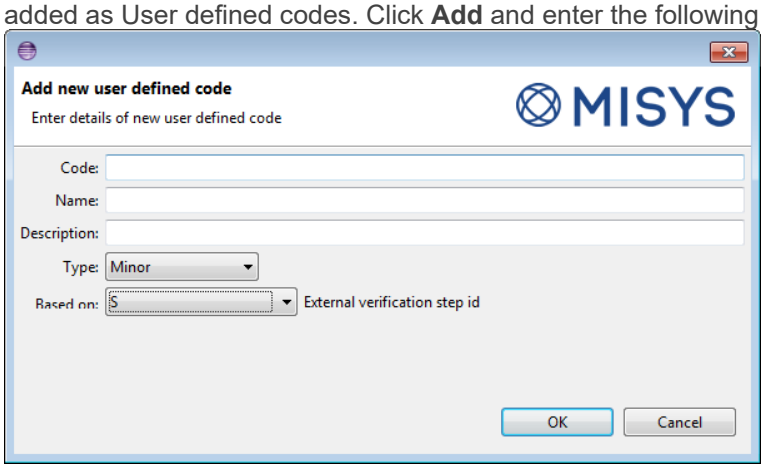


Enter a name and description.

User defined codes

This is only shown if the Advanced SDK has been selected.

For the *EventStep* extension point, the associated minor codes represent step ids for external verification orchestration steps. Once these have been defined in Trade Innovation, they may be added as User defined codes. Click **Add** and enter the following details:



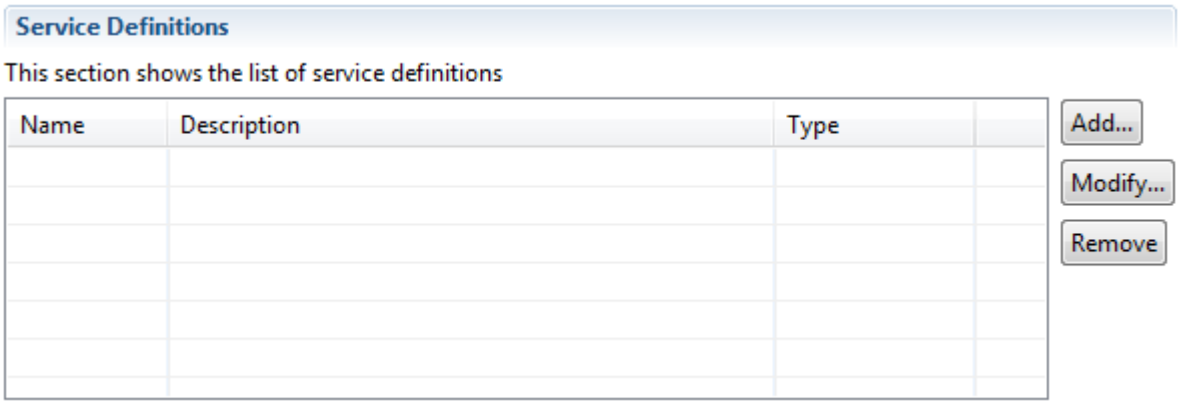
The value for Code must match the step id in Trade Innovation.

Note: The code must not have any embedded spaces, so make sure that the step id defined in Trade Innovation does not have spaces.

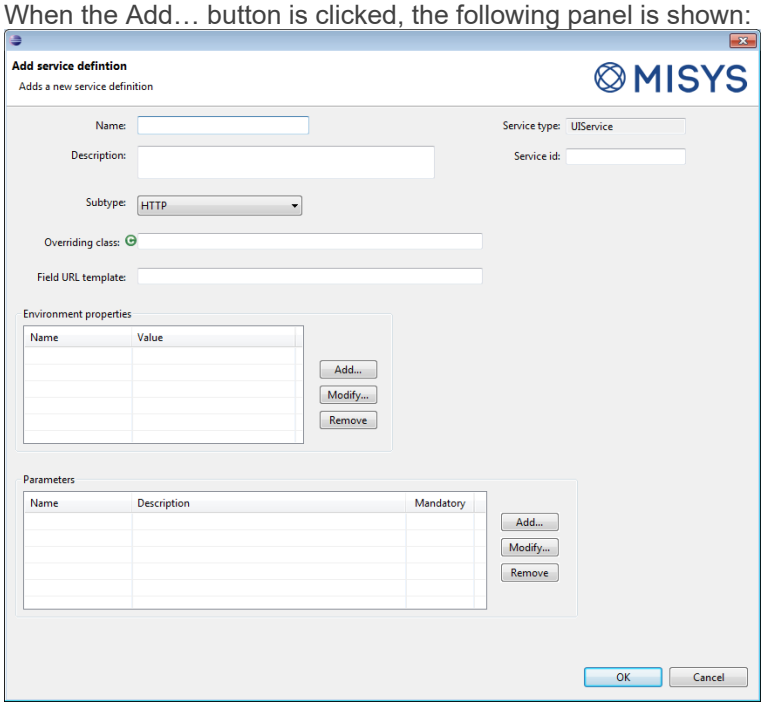
Service definitions

This is only shown if the Advanced SDK has been selected.

To show a linked or embedded view, at least one service must be defined.



When the Add… button is clicked, the following panel is shown:



Enter a name and description for the service.

Two types of service are currently supported:

- HTTP – this represents the equivalent of an HTTP ‘GET’ operation
- Other – this signifies a project-specific service

For an *HTTP* service, the standard processing may be overridden by subclassing the provided service. If this is required, enter the full class name for the Overriding class.

In order to perform the GET operation, an HTTP service requires a URL to invoke. This may be a fixed URL, or it may require data from its context in order to calculate it. The URL template field can be defined with parameter placeholders defined using the syntax `${myParameter}`. This may include the domain name of the URL, or even the root URL itself.

For an *Other* service, the full class name is mandatory.

For both types of services, environment entries and parameters may be defined.

Environment entries are fields that may change based on the particular deployment. For example if an external system needs to be called, its location may be different for a test system as opposed to the live system. An initial value may be specified, and then overridden along with the other environment variables for a zone access deployment.

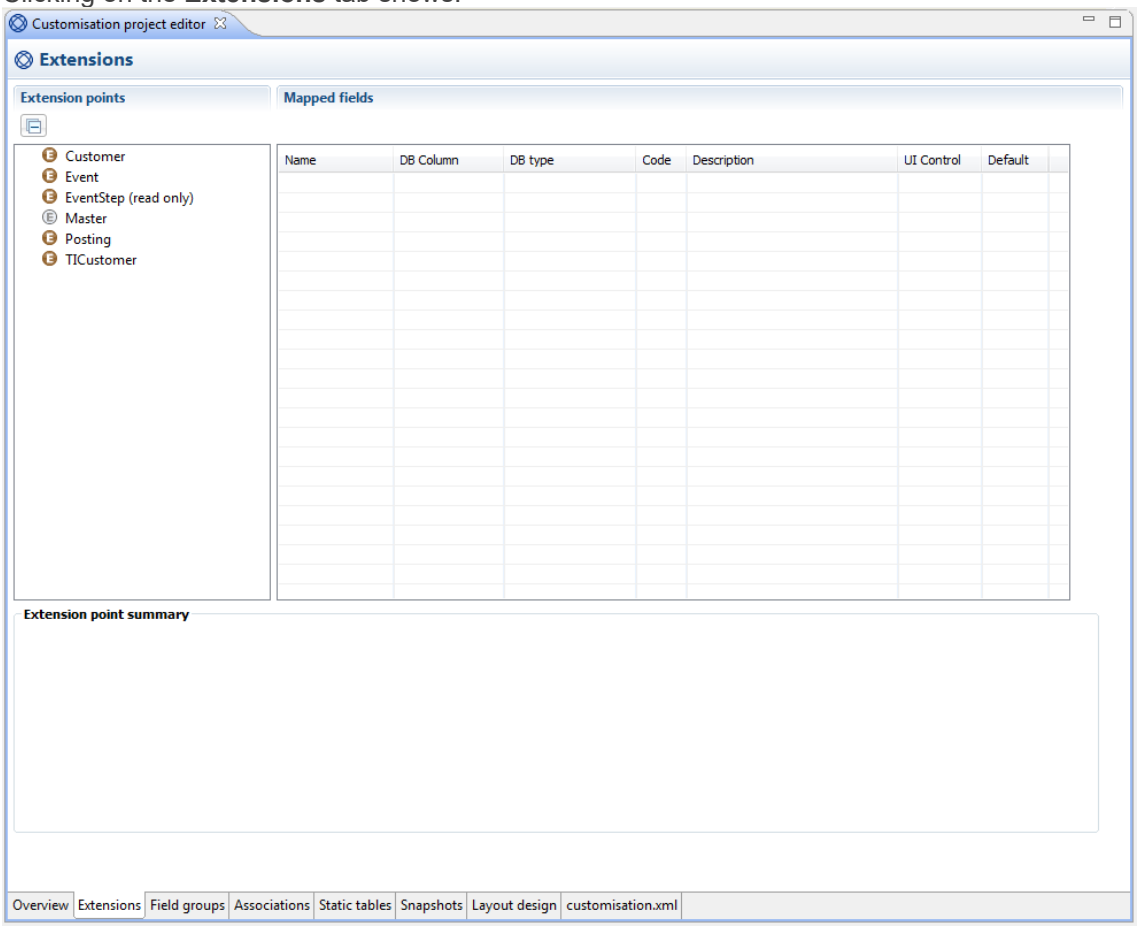
Click on **Add…** and enter a name and initial value.

Parameters are fields that are needed in order for a service to be called. A parameter may be marked as mandatory, so that a service will only be called once all mandatory parameters have values.

Click on **Add…** and enter a name, description and whether the parameter is mandatory.

Extensions Tab

Clicking on the **Extensions** tab shows:

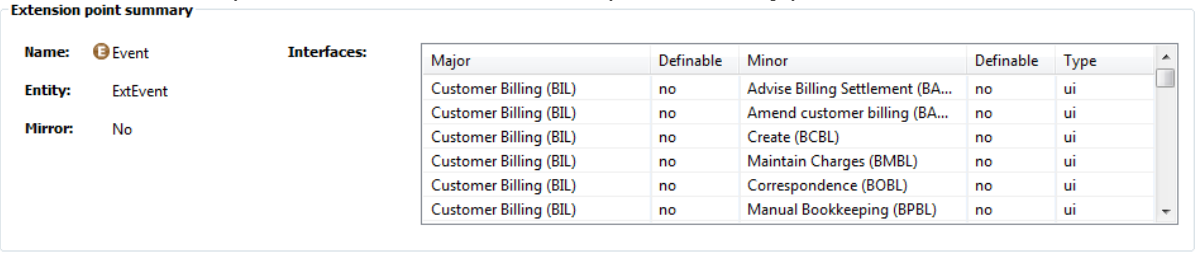


Here, the extension points are listed on the left. From this screen you can add field definitions to those extension points.

Note that the *Master* extension point has a greyed out icon. This is because it is not possible to add fields directly to the *Master* extension; they are added indirectly from the *Event* extension.

Also, the *EventStep* extension is read only – that is, any layouts defined for the extension will be for display purposes only. This is only visible with the Advanced SDK.

When an extension point is selected, the Extension point summary panel is refreshed:

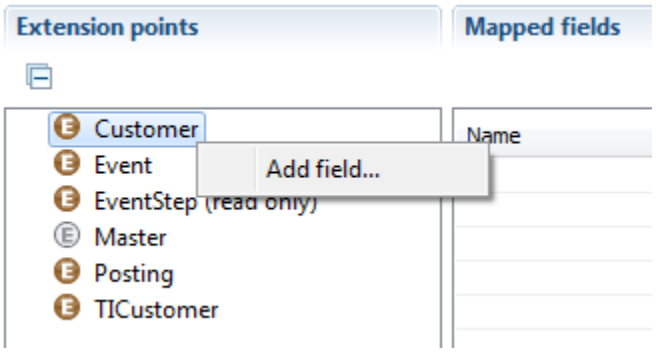


This panel shows the name of the extension point, the entity that the fields will be added to, whether the extension point is a mirror of another (in this case the *Master* is a mirror of the *Event*), and the interfaces that can be customised.

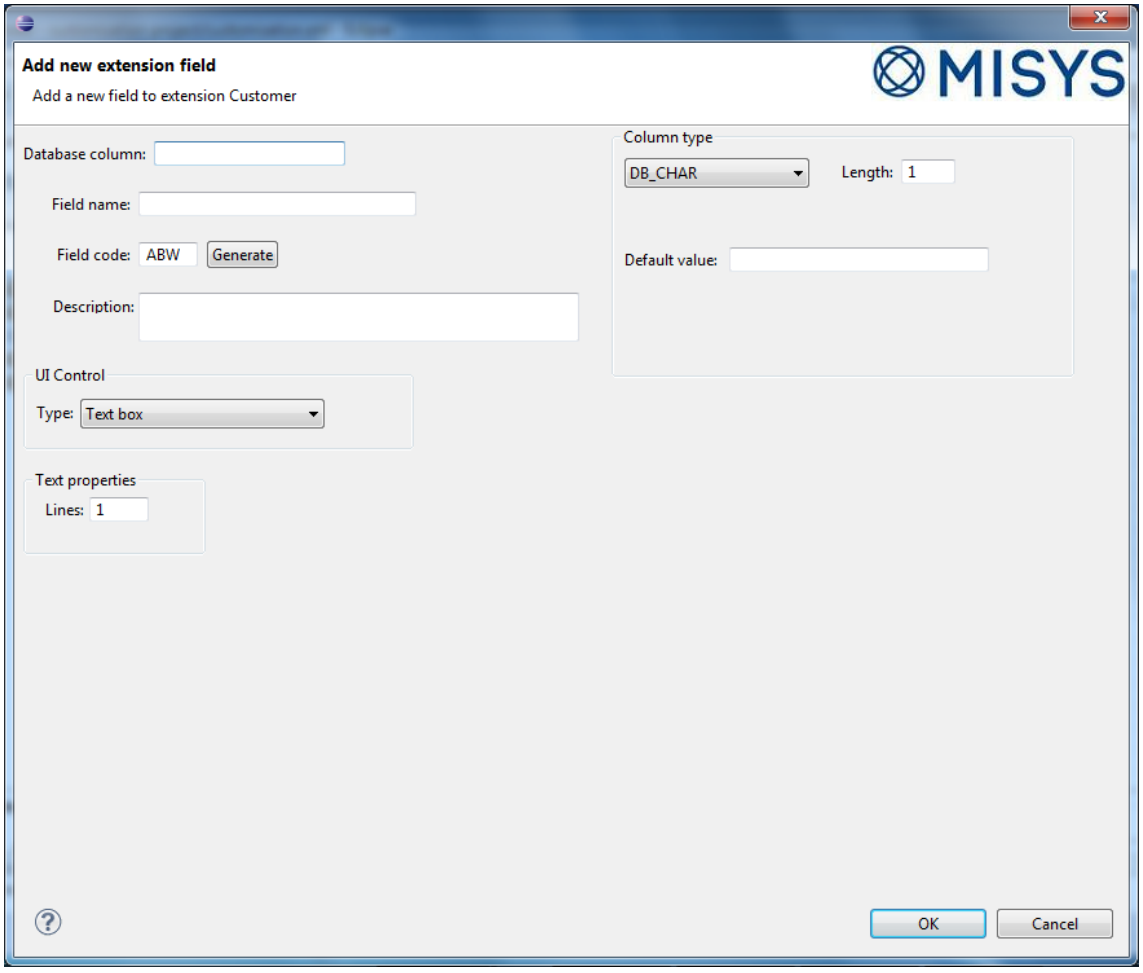
For the *EventStep* extension point, a definable minor code applies.

Adding a New Field

To add a field to an extension, right click on the extension point and select **Add field…**.



This brings up the dialog:

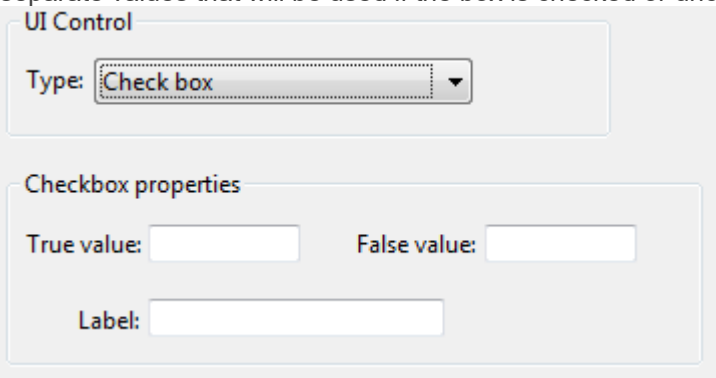


This allows you to define the field details. At the same time, you also define the UI control used for this:

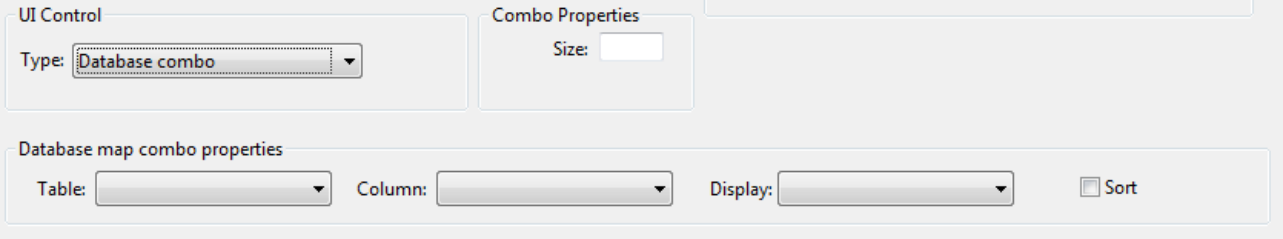
Field	What to Enter
Column name	The name of the column on the extended table (maximum 8 characters)
Mirror on Master	Check if this column should also appear on the Master extension (only applicable for the Event extension)
Logical name	A more descriptive name
Field code	A field code will be automatically allocated, but may be overwritten. If the field is blank, you can generate another value by pressing the Generate button
Mirrored field code	Field code for the mirrored column (only shown if the Mirror on Master option is checked)
Description	A description of the field
Column Type	A drop down of the data type for the column
Length	Length of the column
Scale	Number of decimal places (only shown if the data type is numeric)
Default value	A default value

The UI Control may be fixed based on the data type of the field. If the data type is `TI_MONEY`, then the control type will always be `Money`. For `TI_OFFICER`, `TI_COUNTRY` and `TI_CURRENCY`, they will be either their associated browser or a `Text box`. If it is a `TI_LOGICAL`, then it is a `Check box`. If it is a `TI_DATE`, then only a `Text box` is shown.

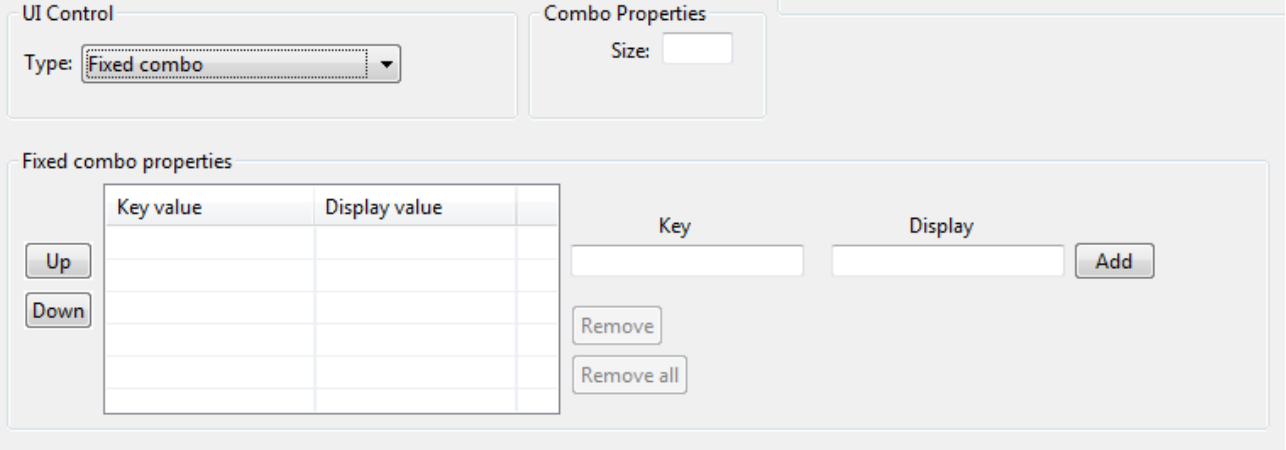
For a `Check box`, the data type does not have to be a Y/N result – it is possible to specify the separate values that will be used if the box is checked or unchecked. A label can also be provided.



For a Database combo, you may select the Table, followed by the column whose selected value will be used, and a column that will be used in the drop down for display. Only new static definition tables added to the project are eligible to be listed. When a table is selected, then only columns that have the same data type and length are available to be selected in the column drop-down.



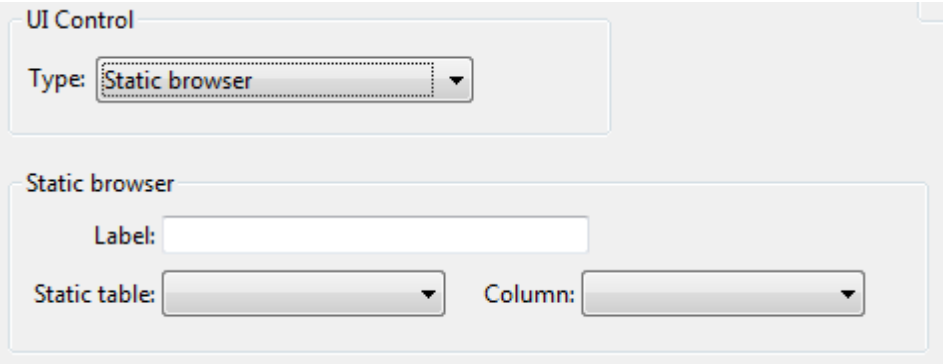
For a Fixed combo, the entries are added – providing a key value and a display value.



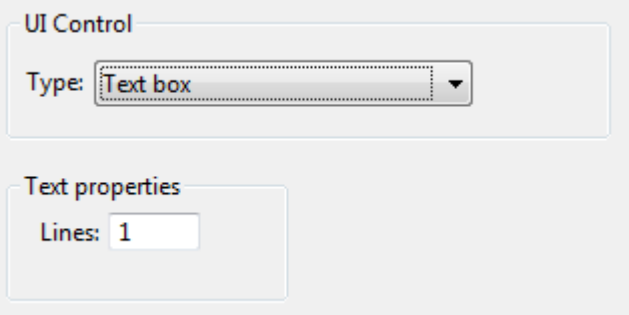
If the size field is set to a value greater than 1, then the control used will be a list box allowing multiple items to be selected. A maximum number of items allowed to be selected may also be specified. This is used during validation.

Note: The maximum number is also used to work out the size of the database column. Multiple items are stored as a comma-separated list. If no maximum is defined, then a CLOB is used to store the data, otherwise a VARCHAR is used and its length derived.

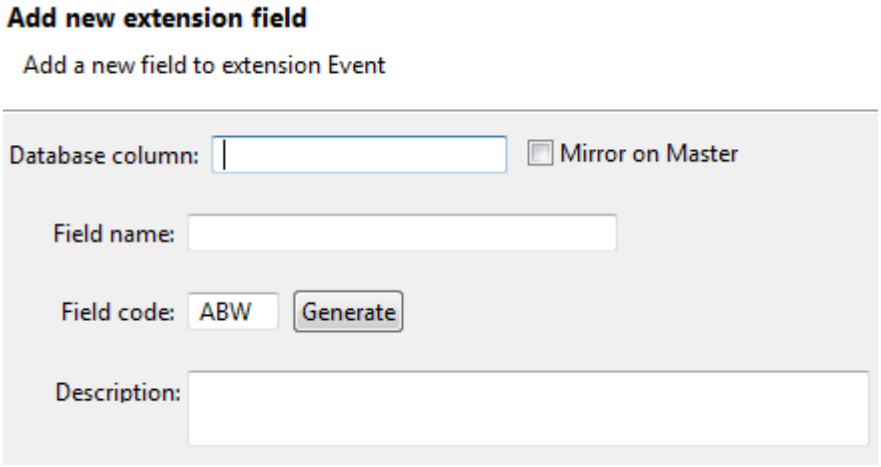
For a Static browser, the rules are similar for the database combo in terms of the table and column selected. Note that only the key field for a static table maybe selected for the column. The label here is to help differentiate it when adding the control to the layout, it is not used at runtime.



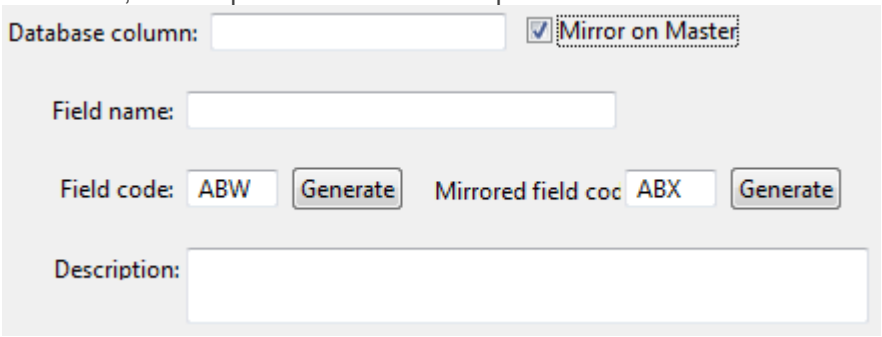
Finally for a Text box, the number of lines must be specified.



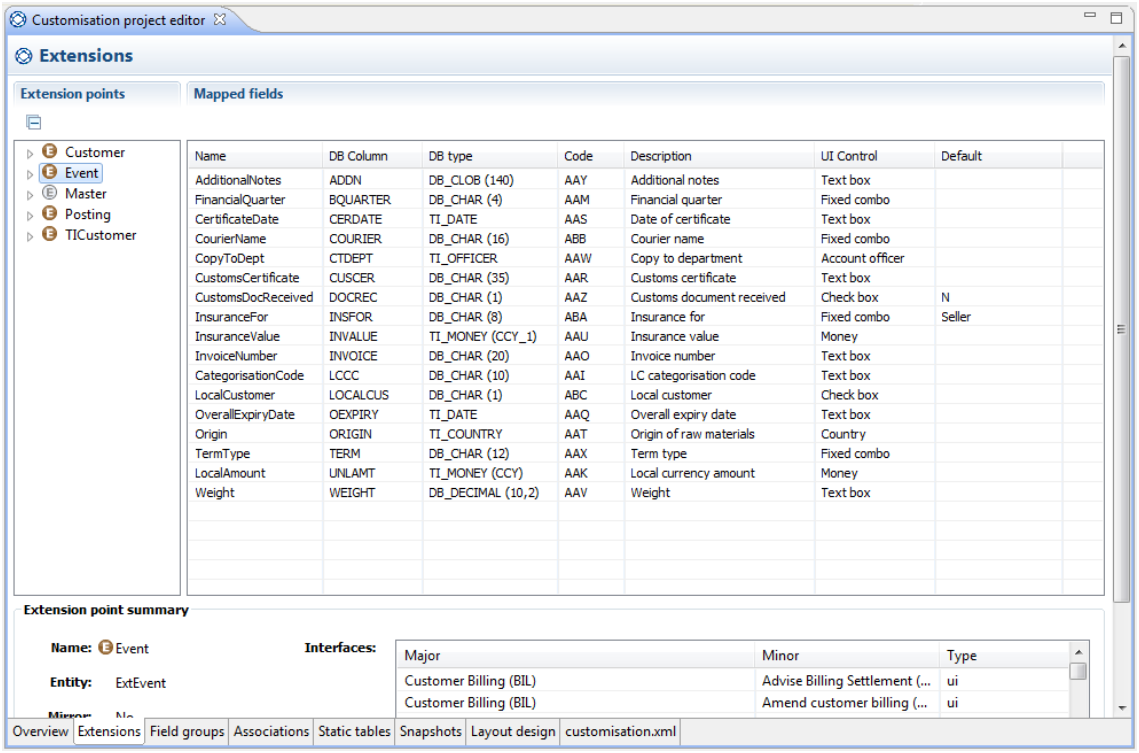
When adding a field to the *Event* extension, an extra checkbox will appear, allowing the field to be added to the *Master* extension:



If checked, it is the possible to define a separate field code for the Master extension:

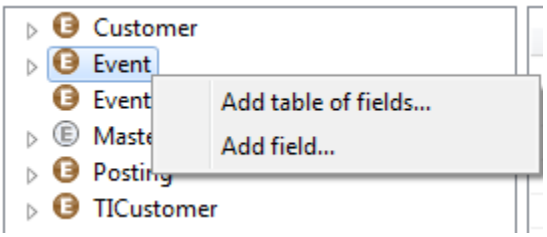


After you have added the fields you need, if you select the extension, a summary of the fields added is shown:

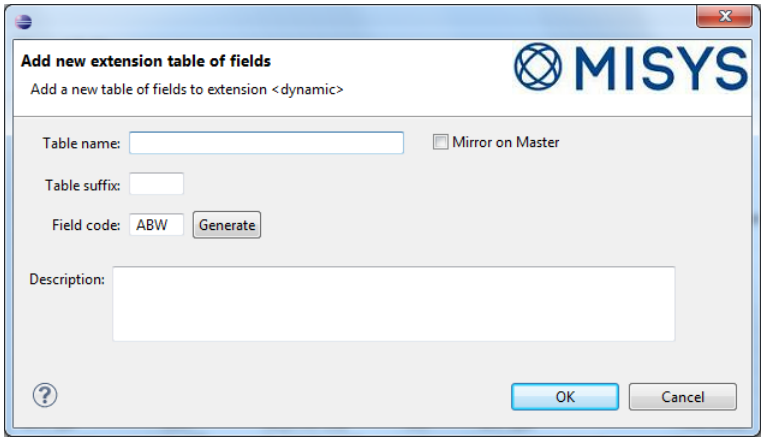


Adding a Table of Fields

For the *Event and EventStep* extension, it is possible to add a table of fields.

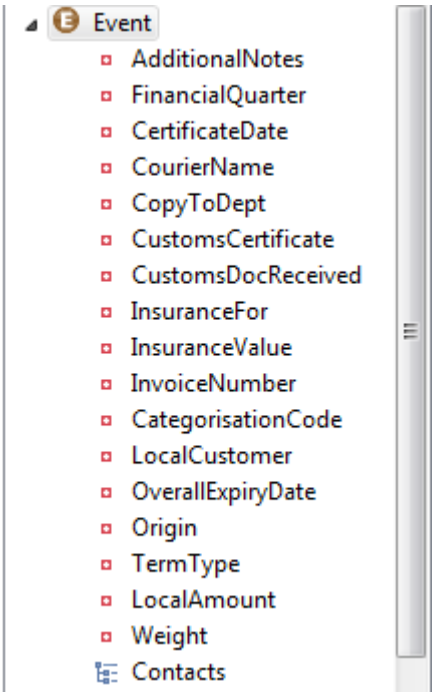


Right-click on the *Event* extension and select the **Add table of fields...** option.



As with a normal field in the *Event* extension, it is possible to mirror this table on the *Master* extension. If this is checked, then you can also specify the field code for the *Master* version.

Once these details are entered, the extension details are:

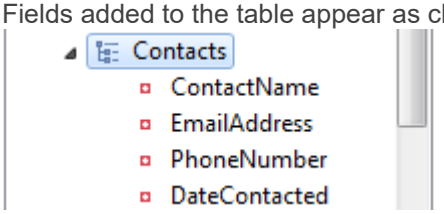


With the new table of fields at the bottom of the list.

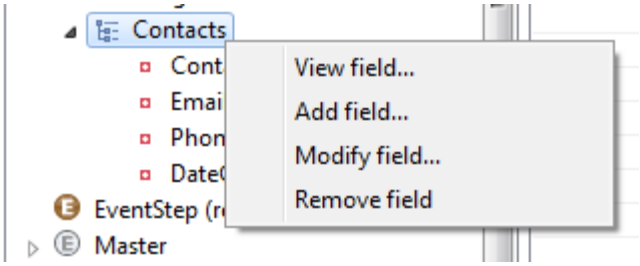
You can add fields to this table in the same way as you add to the extension itself. The only difference is that the field code is now a part code.

Note: *EventStep* extension field's codes are used to populate customised data from an incoming message, and are not available to rules and documents.

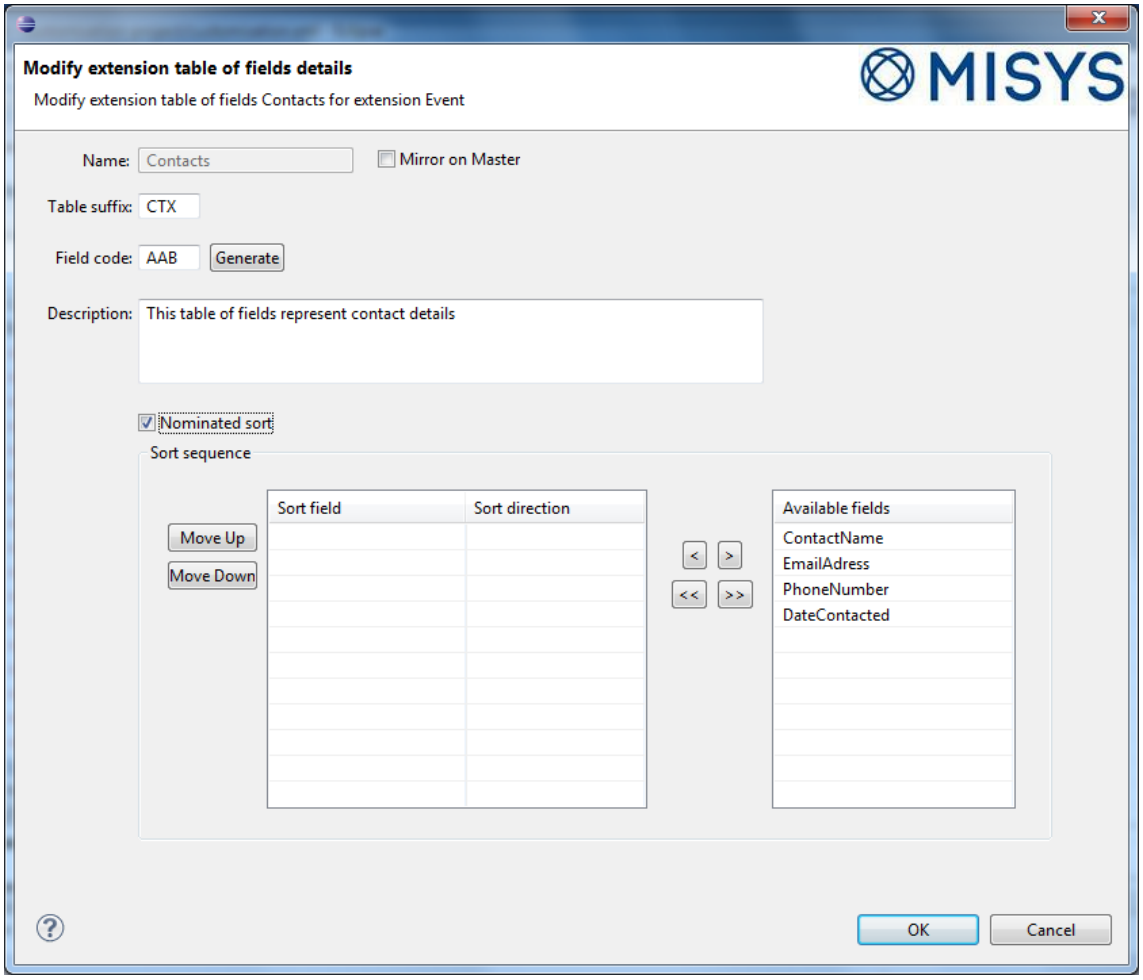
Fields added to the table appear as children:



There are two types of tables, those whose entries are sorted manually, and those whose entries are sorted by particular fields in the table. To specify which type, modify the table itself.



This shows the details of the table, providing the ability to specify the table type. It is assumed that the table is manually sorted when created.

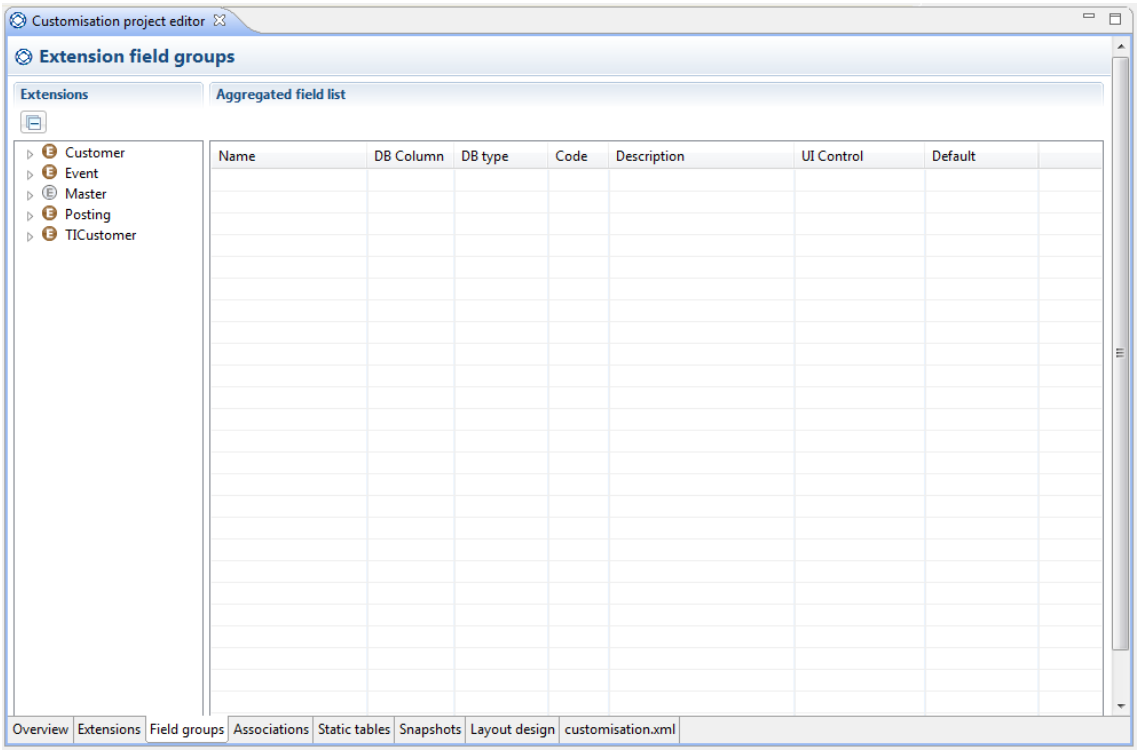


To specify the order that the entries will appear in, select the relevant field from the list on the right and use the '**<**' button to add it to the list of sort fields.

Fields are added to the list with ascending order by default – right-click on the row to change the direction.

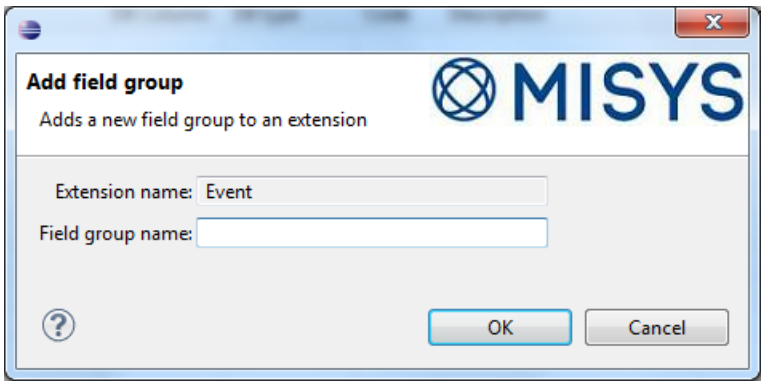
Field Groups Tab

The next thing to do is to group fields together using the **Field groups** tab.

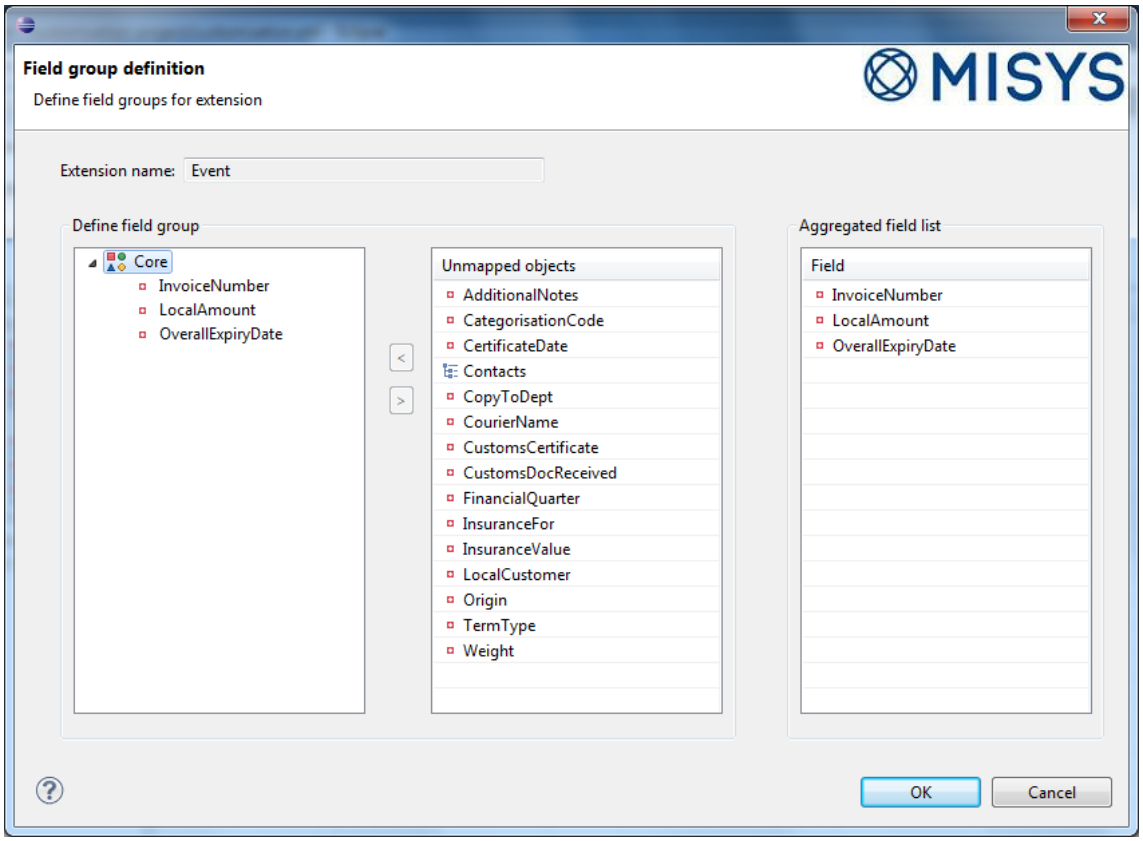


For each extension, a **Default** group is defined which corresponds to all fields associated with the extension. The only extensions that allow you to define more field groups are the *Master* and *Event* extension.

To add one, right-click on the extension and select **Add field group…**.

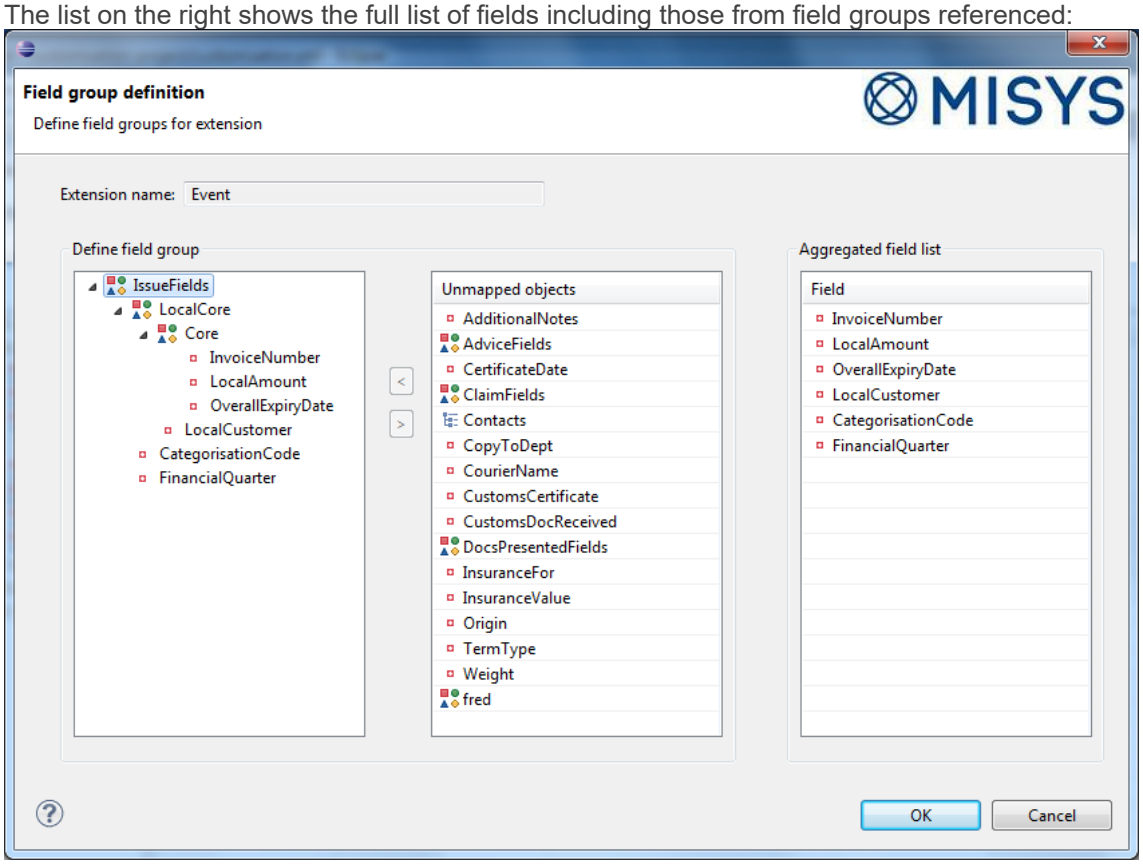


Enter a name for the field group and click **OK**.



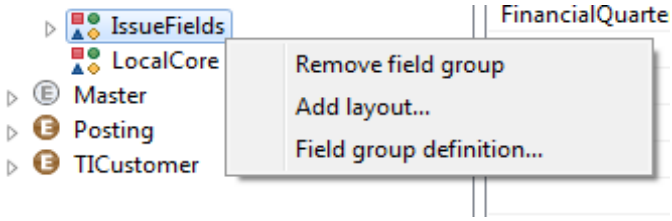
You can then add or remove other field groups or individual fields to your new field group using the < and > buttons when selecting items in the field group list or the unmapped objects list.

The list on the right shows the full list of fields including those from field groups referenced:

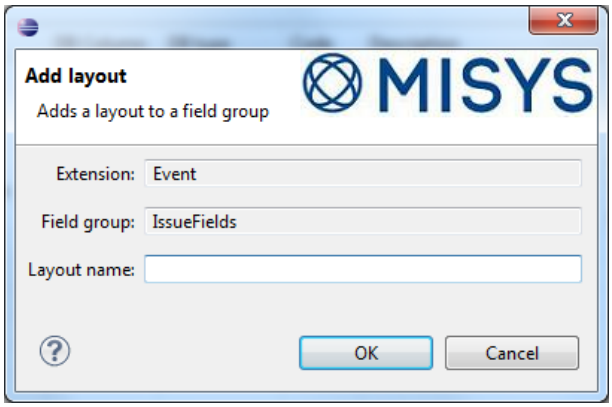


Note: A table can be included in a field group, but not its fields.

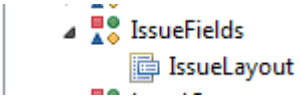
Once a field group has been defined, you can then add a common layout to it by right-clicking the field group and selecting **Add layout…**.



This then allows you to give a name to the layout.

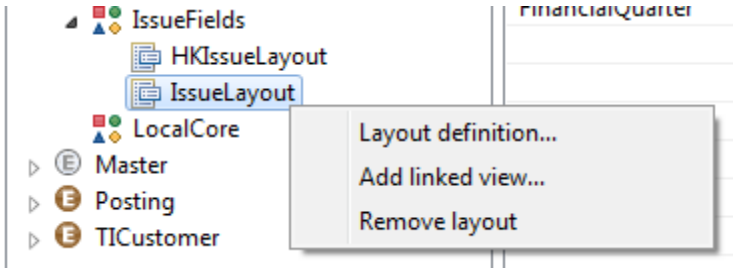


This is then shown below the field group.

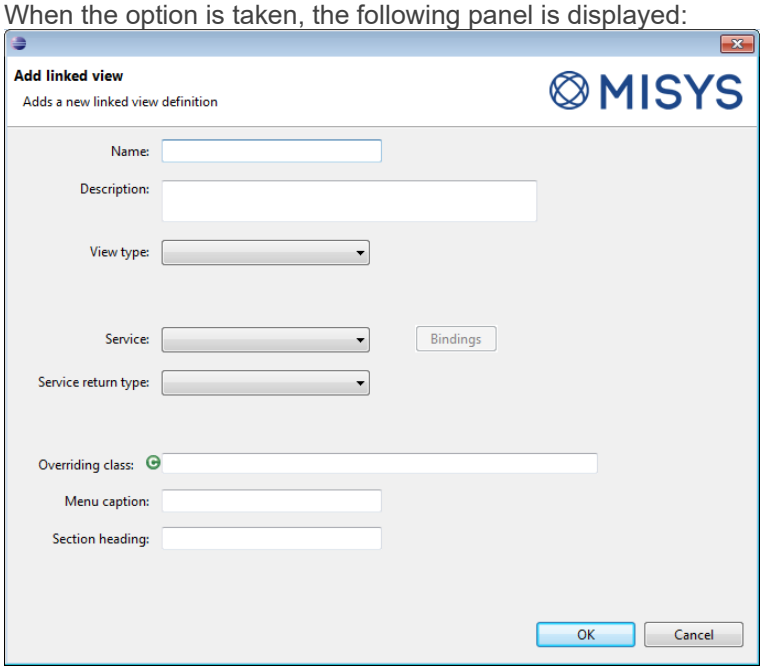


Double-clicking the layout allows you to design the associated panel (see next section).

If you are using the Advanced SDK, then if you right-click on the layout, you can define a linked view:



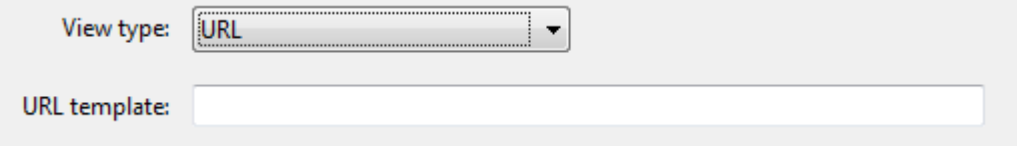
When the option is taken, the following panel is displayed:



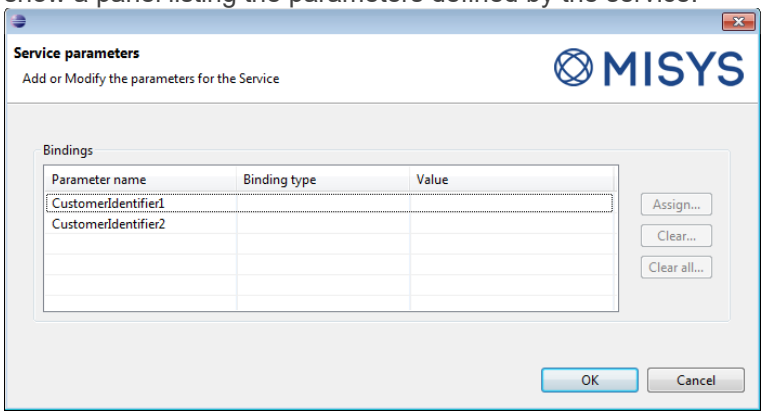
Enter a name and description.

There are two types of view – URL or content-based.

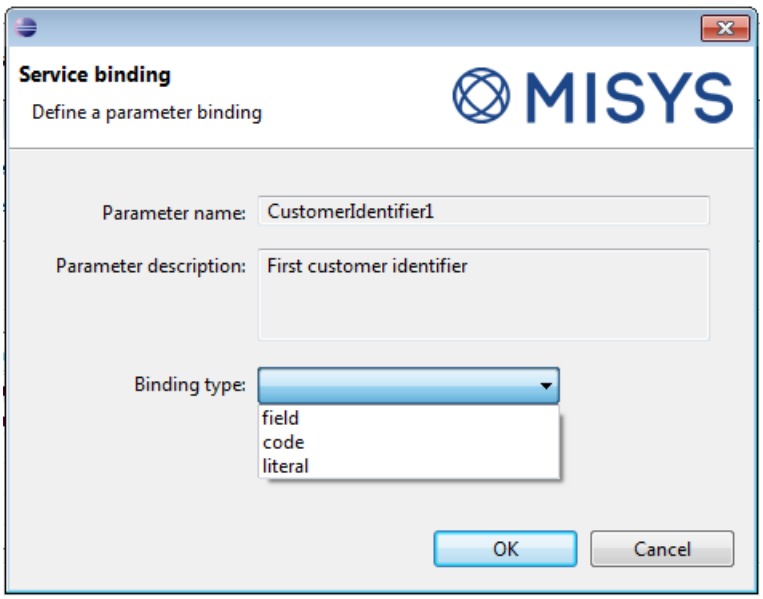
If the type selected is URL, then a URL template must be specified. The format of this field is described in the service definition section of the overview tab.



A service must be selected to use for the view. These are defined on the Overview tab. Once the appropriate service has been selected, then the Bindings button is available. Clicking this button will show a panel listing the parameters defined by the service:



Selecting a parameter, a binding can be defined by clicking **Assign...**

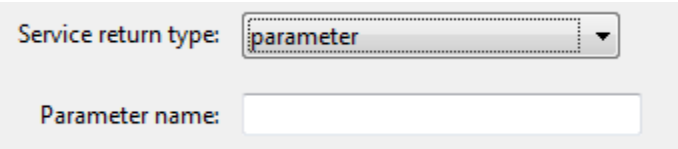


There are three types of binding – Field, Literal and Code:

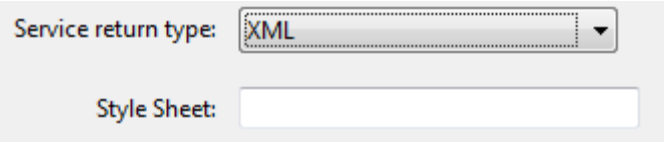
- Field allows you to select a customised field from the field group associated with the layout.
- Literal allows you to supply a specific value
- Code allows you to supply a field code:
  - For Event, Master and EventStep, this is in the form of code, type and part. For instance, the applicant's customer identifier would be APP,p,cu.
  - For Customer and TICustomer, this is a descriptive name – see later section

Back on the linked view panel definition, based on the view type and service type, you can specify what the expected return type of the service will be.

For URL views, the return type could be Parameter, Properties or Nothing. Parameter requires a name to be specified that the returned value will be associated with (this does not need to match a parameter defined by the service). For Properties, this signifies that the return value will be a list of name/value pairs.



For Content views, the return type could be Content or XML. For XML, you must specify an XSLT style sheet file that will be executed in order to transform the XML into HTML content.



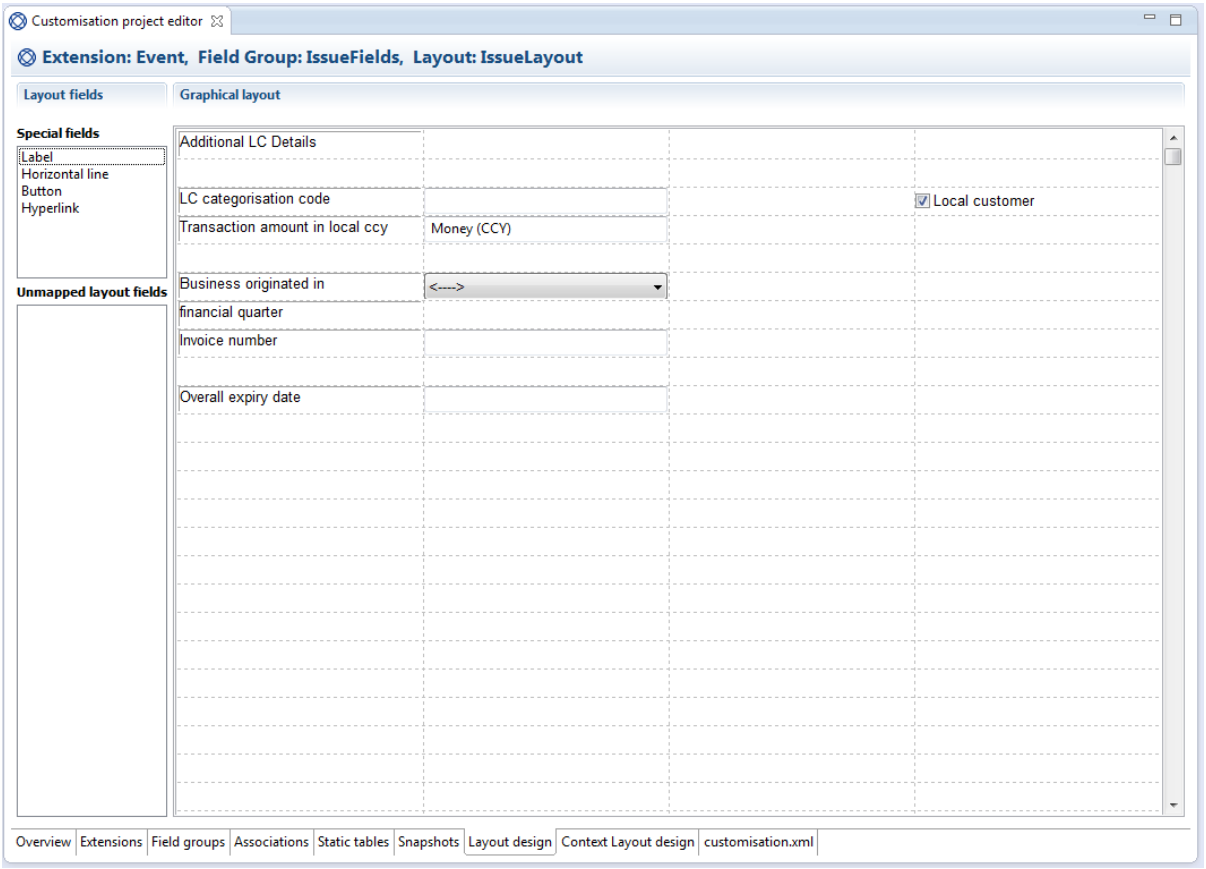
If the standard view processing is going to be overridden, then the respective full class name may be specified.

Finally, supply the menu caption, and the section title that will be displayed when the linked view page is shown.

Layout Tab

The **Layout** tab is shown when a layout is double-clicked from the field group tab or from the associations tab. If you select it explicitly, it will either show the last layout you were working on, or an empty layout editor.

The tab shows the fields available to the layout to use.

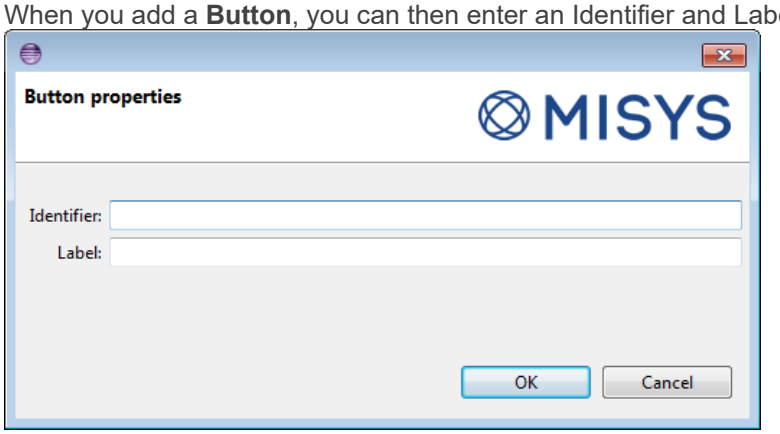


The four-column grid represents how controls will be placed in the extra data section at runtime, and is more of a logical representation rather than an explicit rendering.

Items can be added to the panel by dragging them from the lists on the left to the appropriate grid position. To remove an item right-click it and select **Remove**. If you remove a field, it will reappear in the list on the left.

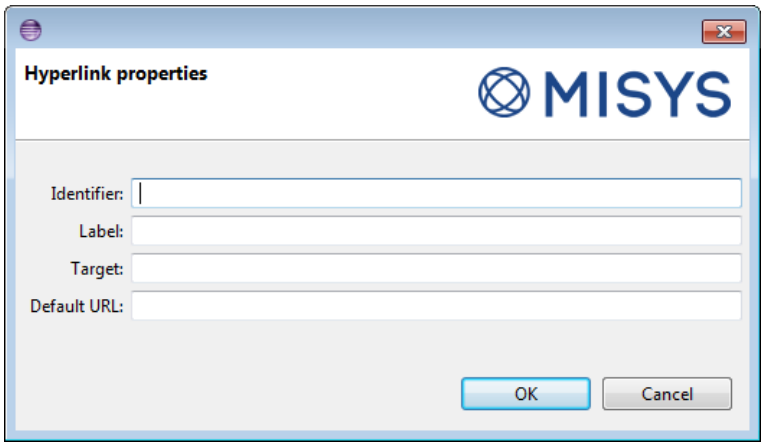
The **Button** and **Hyperlink** special fields are only available with the *Standard* or *Advanced SDK*.

When you add a **Button**, you can then enter an Identifier and Label:



The Identifier will be used to generate a supporting action method that will be called as a result of the user clicking the button.

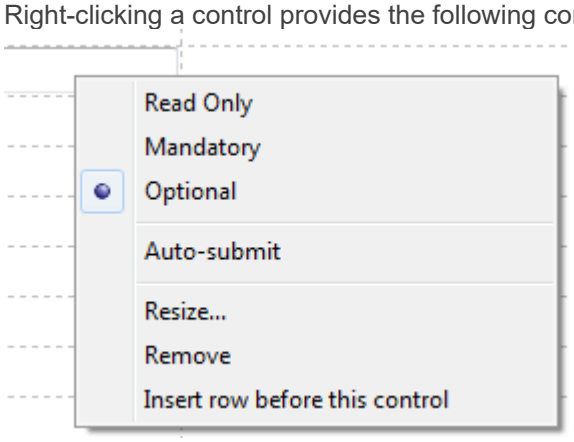
When you add a **Hyperlink**, you can then enter an Identifier, a Label, the target window to show the result of clicking the hyperlink and a default URL.



If the target field is blank, then each time the hyperlink is clicked, a new window or tab will be opened.

Continue until all of the required fields are laid out. (Note you don't have to add all or indeed any fields to the layout).

Right-clicking a control provides the following context menu:

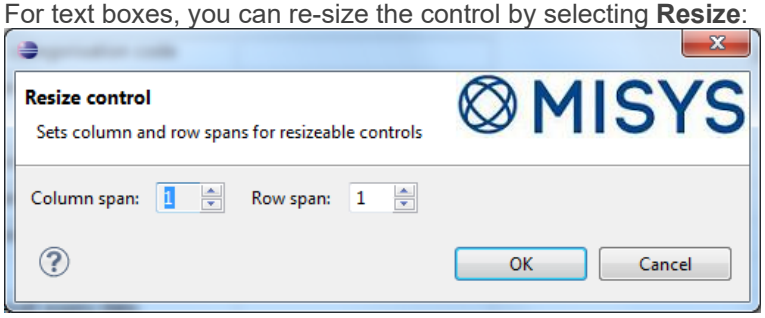


All controls are by default set to being **Optional**. You can make them **Mandatory** or **Read Only** instead.

All input controls may be flagged as **Auto-submit**, so that if a value for this control changes, the information is submitted to the server and the page redisplayed. This is useful to help condition showing a field based on the value of another.

Note: If the control is a multi-item list box, then auto-submit is not available.

For text boxes, you can re-size the control by selecting **Resize**:

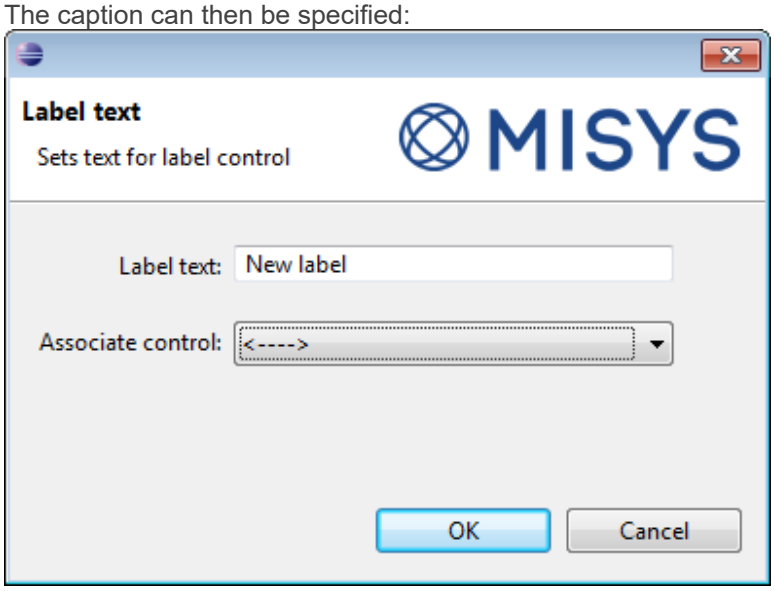


Note it is not possible to move controls that have been added to the grid – they must be removed and re-added.

This menu also allows a row to be inserted above this control – all items on the same line and following lines will move down.

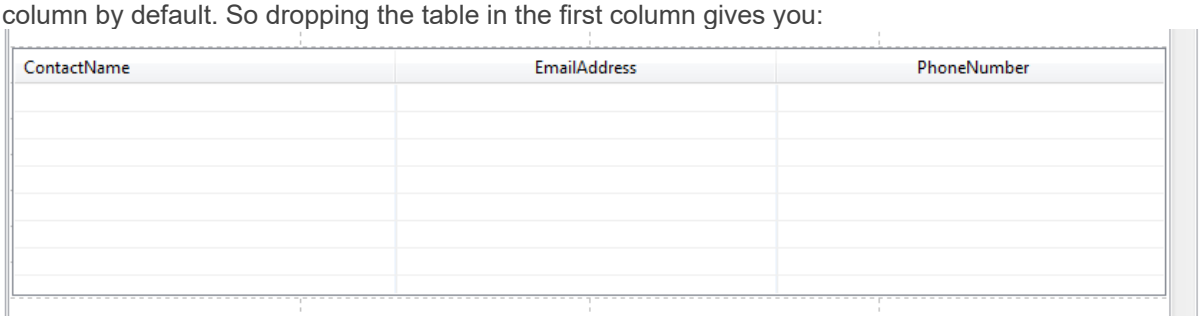
A label can be added to the layout by dragging the label entry from the Special fields section.

The caption can then be specified:

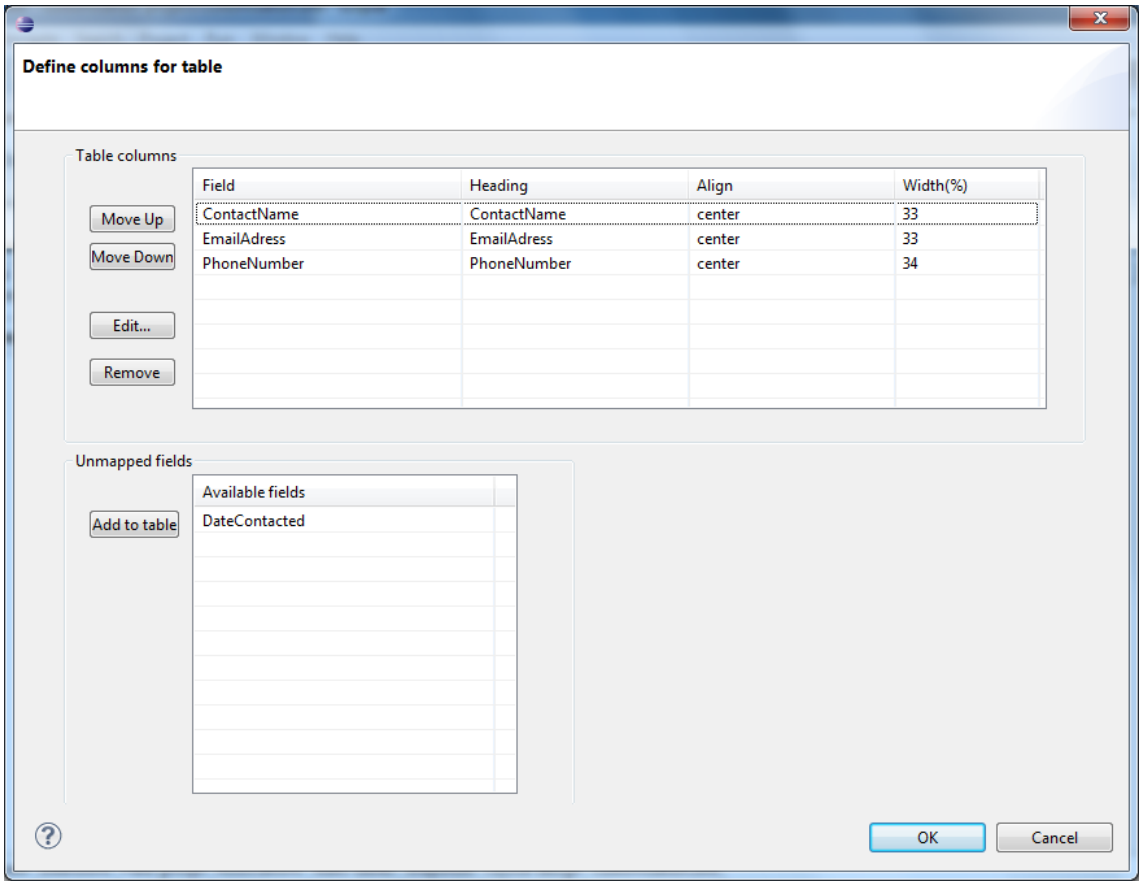


The associated control drop down allows you to select a control already on the layout. At runtime, if the associated control is hidden, then the label will be hidden as well.

When you add a table to a layout, it will take up the columns from where you place it up to the last column by default. So dropping the table in the first column gives you:

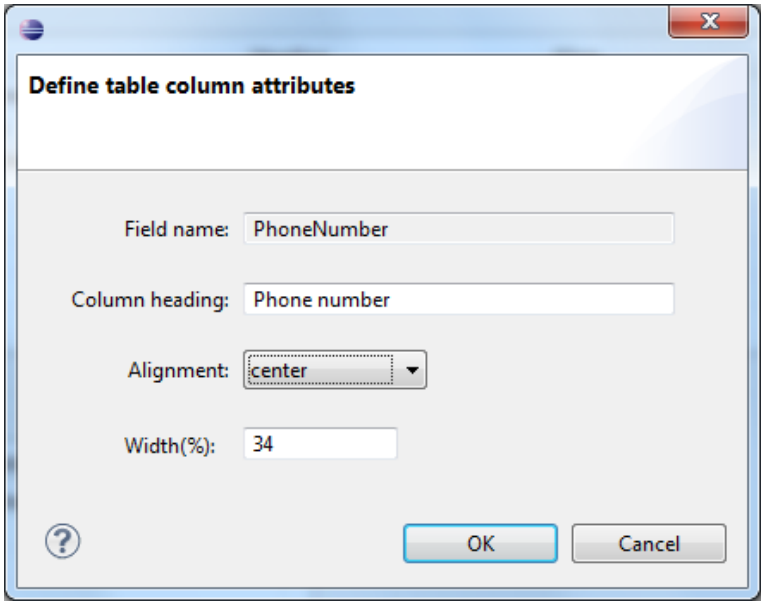


By default, the first three fields are shown as columns. These can be change by right-clicking and selecting the **Define columns...** option.



On this panel, you can add fields to the table, or remove them.

When on the table, the behaviour can be changed by selecting a field and clicking **Edit...**.



This panel allows you to change the column heading, the content alignment and the relative width.

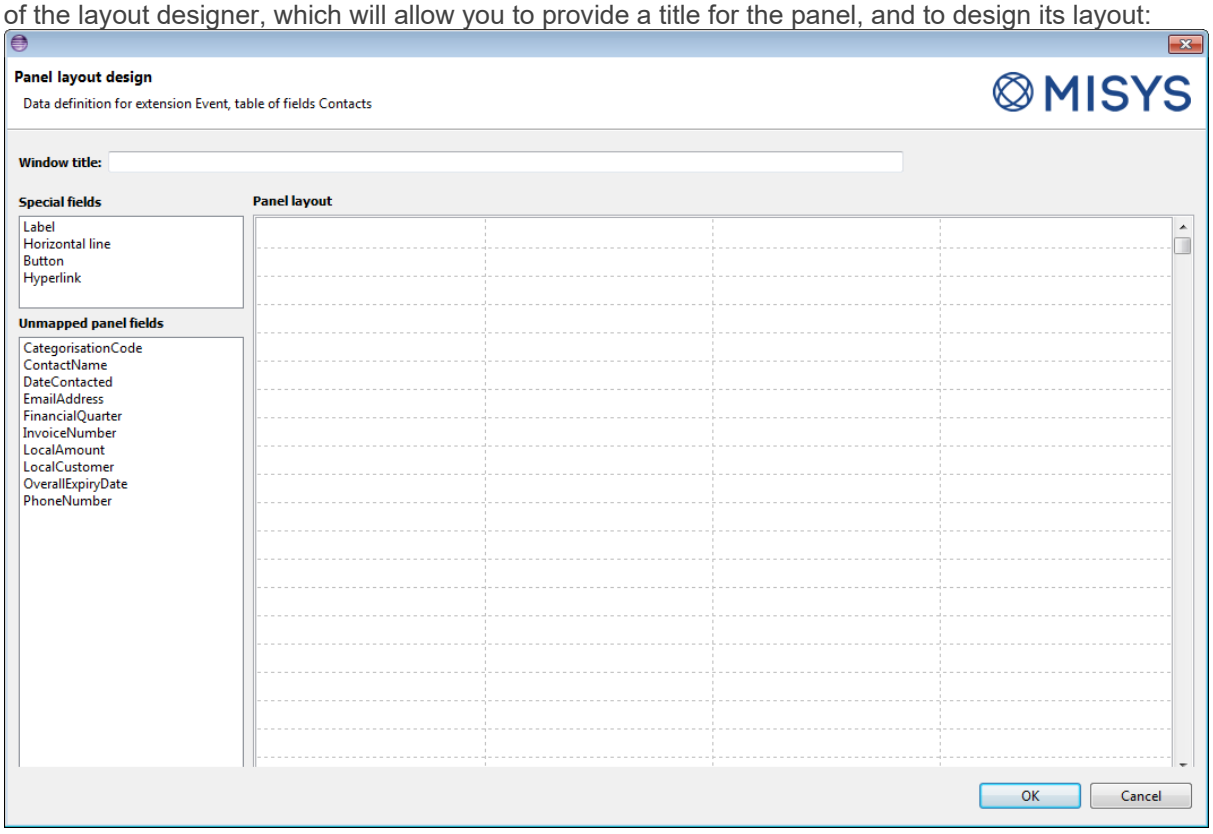
This provides a table on the layout. At runtime there will also be buttons to manage the content of the table:

- Add
- View
- Update
- Delete
- Move Up (manual sort tables only)
- Move Down (manual sort tables only)

You will see that the number of lines taken by the list will vary based on the number of columns the table spans. This is to take into account the number of rows covered by the buttons.

When one of the main buttons is pressed, a panel is shown representing a single row – either to enter a new item, or to represent an existing item.

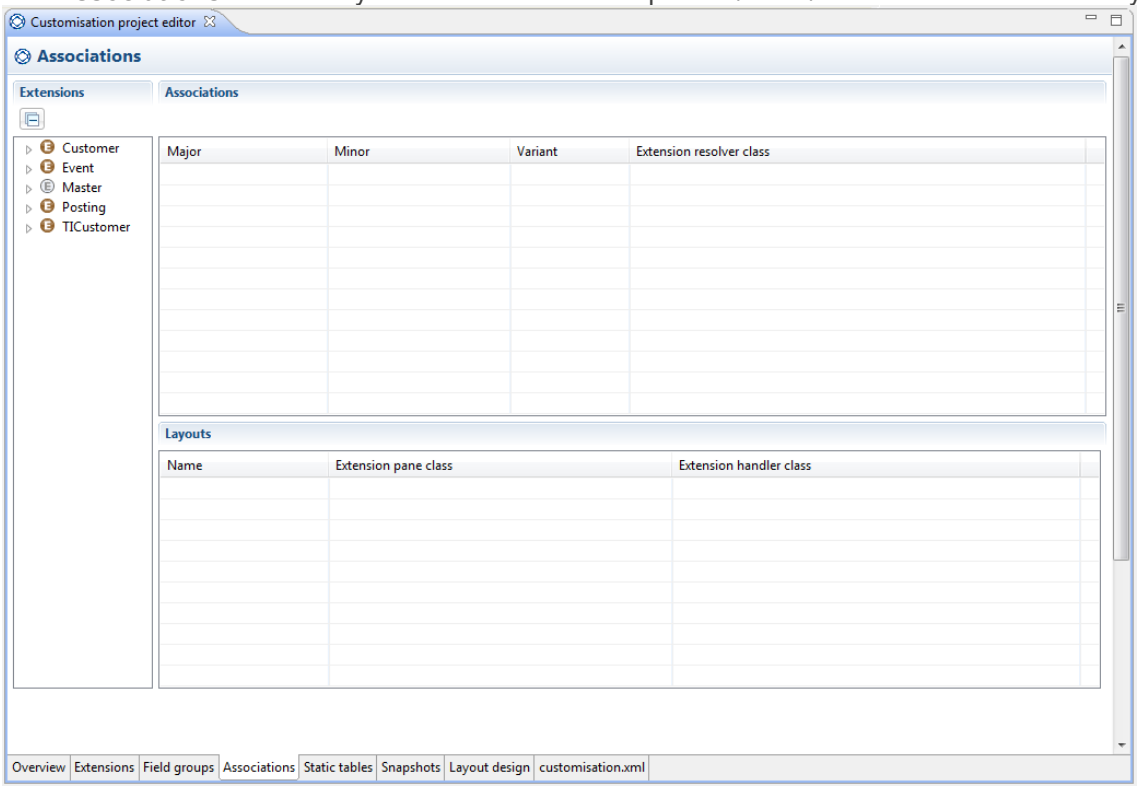
To design that panel, right-click on the table and select **Define panel...** This will open a modal version of the layout designer, which will allow you to provide a title for the panel, and to design its layout:



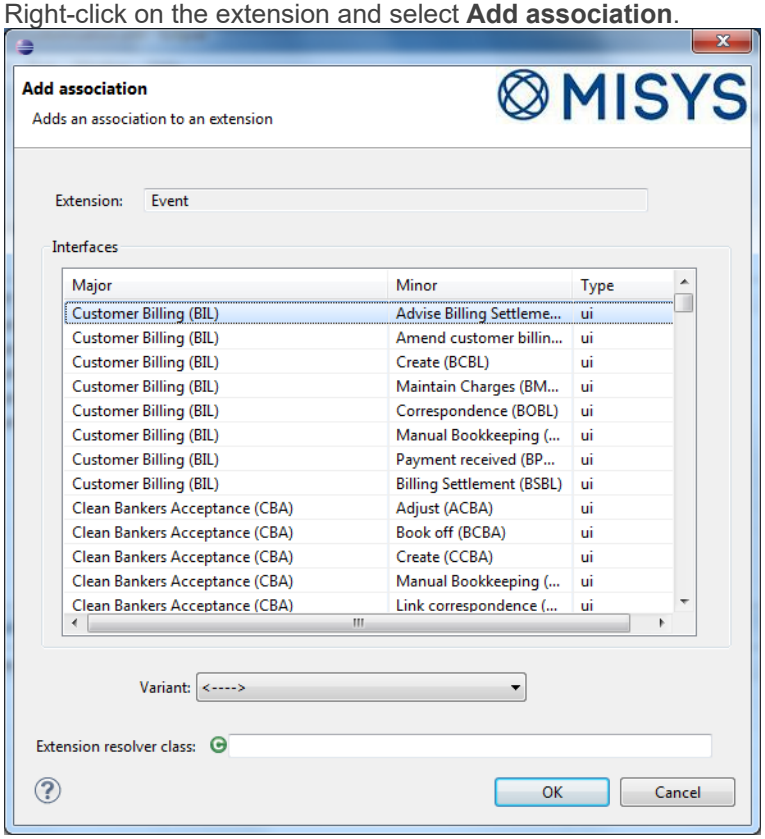
The list of fields will include the fields of the table as well as other non-table customised fields in the field group. If you use any field other than the table fields, then that field will be displayed in read-only mode.

Associations Tab

The **Associations** tab allows you to cross reference a product/event/variant combination to a layout.



Right-click on the extension and select **Add association**.

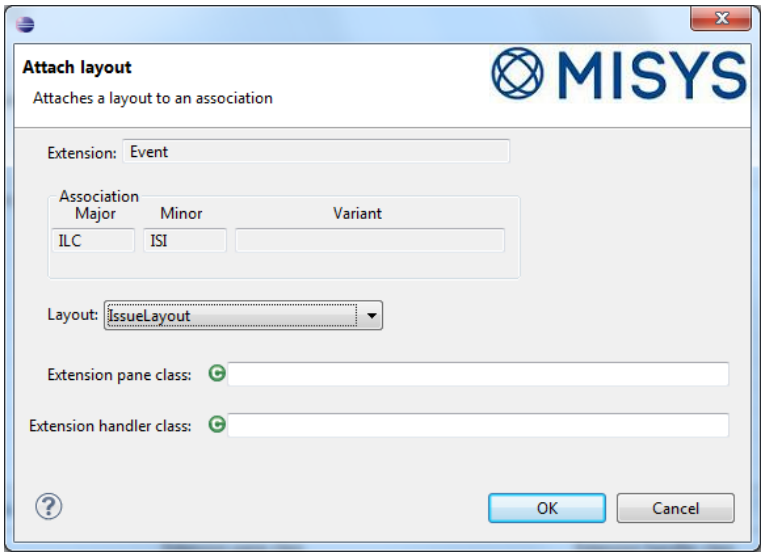


Select the major/minor combination corresponding to the product/event if applicable. If variants have been defined, you can also select a variant if applicable (*Master* and *Event* only).

For *Event* extensions, it is also possible to reference a resolver class that will be used at runtime to indicate the layout that is to be used (see later chapter). The existence of this class will not be checked for – so may be referenced in advance.

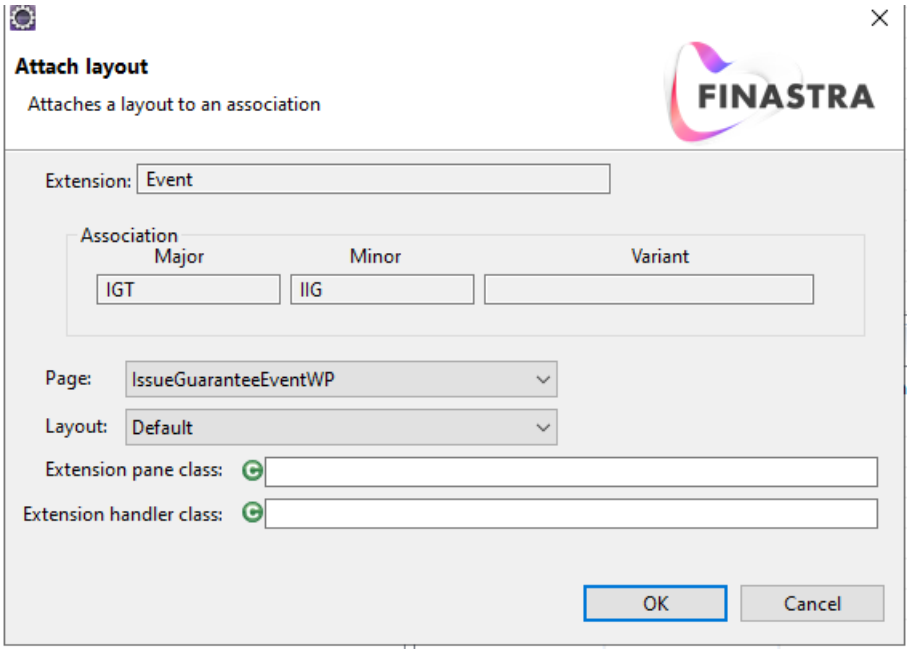
Note that this must be entered if multiple layouts are to be assigned to the association.

To attach a common layout, right-click on the association and select **Attach layout…**.



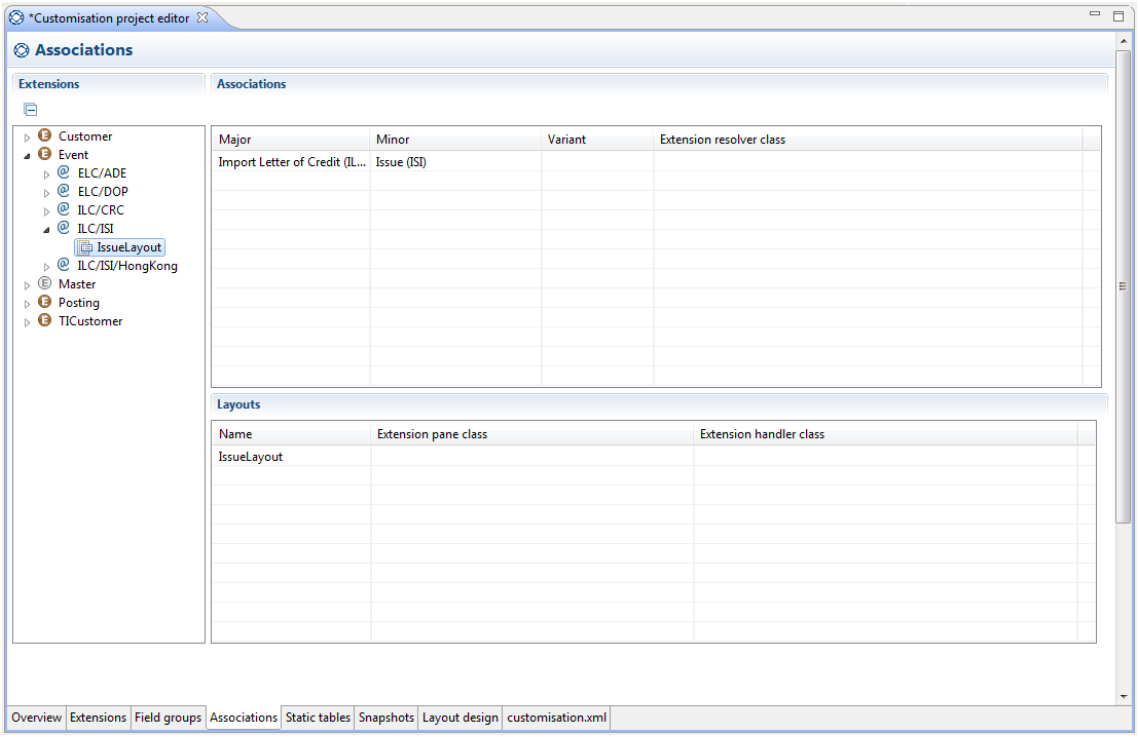
Select from the available layouts in the drop down, and if appropriate subclasses have been or are going to be created for an extension pane or extension handler, specify their full class names. These can be left blank, in which case they will default to the generated handlers.

If the association has multiple alternate pages, the relevant page must be selected



Note that if a resolver class has been specified for the association, then the handler and pane classes can only be defined on the first layout, as it is expected that the same classes will be used for all layouts for the resolver.

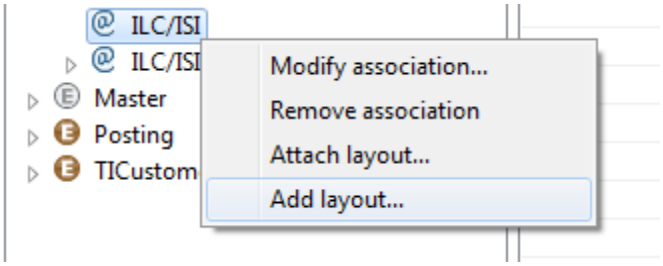
This associations screen then looks like:



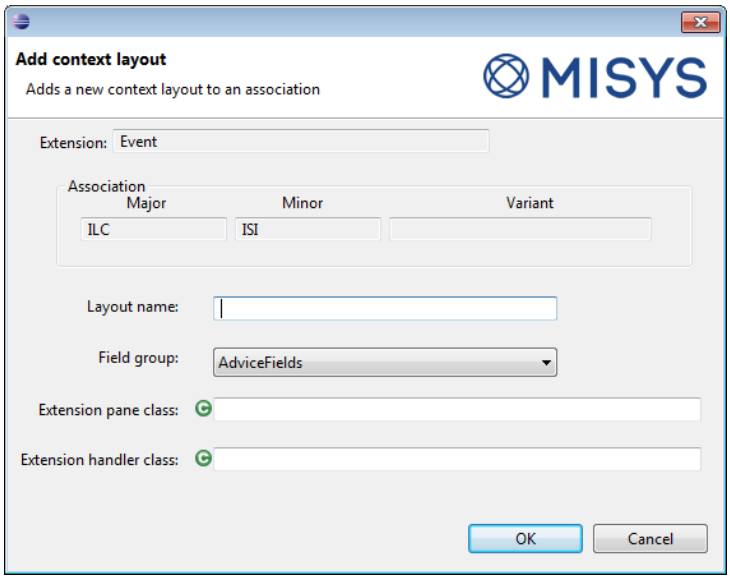
Note: For EventStep, unless you create a user-defined code that represents the step id of an external verification step, no associations will be available.

For data to move to succeeding events, add an association to the master and then add the default view to the new association.

If you are using the Advanced SDK, rather than attaching a common layout, a context-specific layout can be defined:

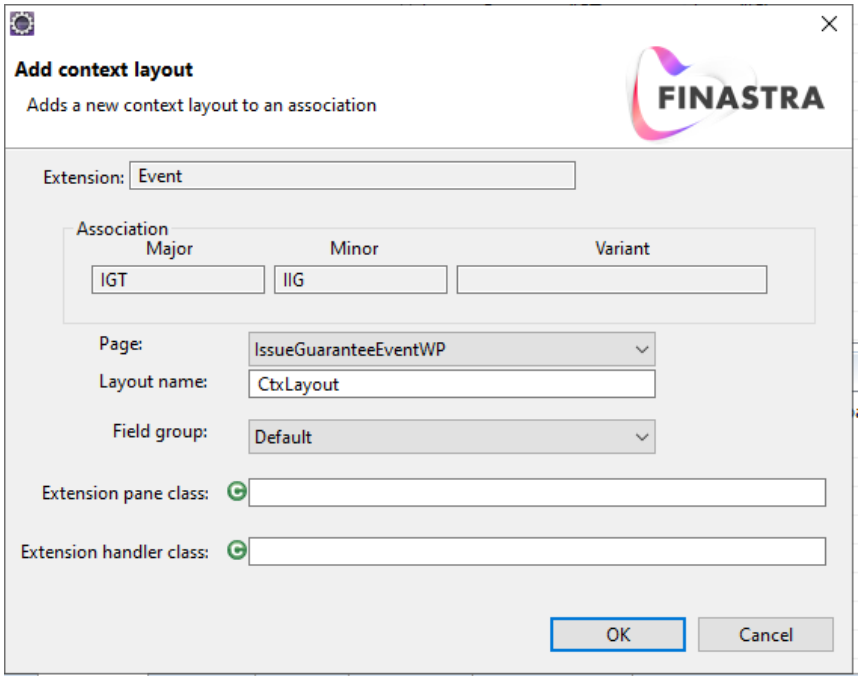


Taking the **Add layout…** option shows the following panel:

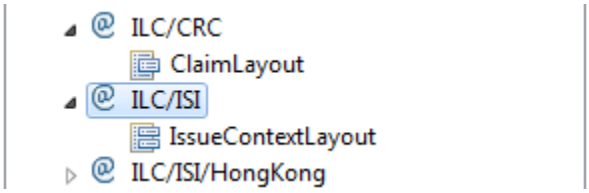


This is similar to the panel shown when attaching a common layout, except you need to supply a name for the layout and pick from the list of field groups to use.

If the association has multiple alternate pages, the relevant page must be selected



Once added, the context layout is shown as:



Right-clicking on the context layout allows you to select the **Layout definition…** option that takes you to the Context Layout Designer (see next section).

It is also possible to add a linked view to a context layout in the same way as a common layout.

Context Layout design Tab

The **Context Layout design** tab is shown when a context layout has the **Layout definition…** option selected from the associations tab. If you select it explicitly, it will either show the last layout you were working on, or an empty layout editor.

The tab shows the fields available to the layout to use.



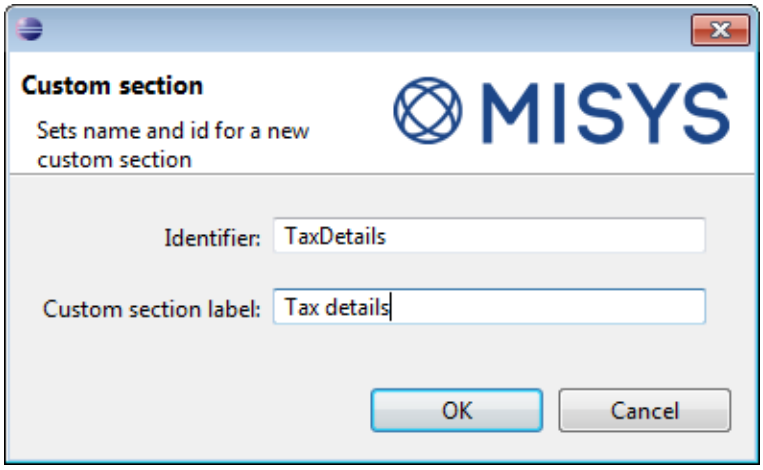
The sections shown are extracted from the page corresponding to the association.

Controls may be dropped into the grid below a section title. When this happens an extra line is added to the grid:



At runtime, some sections may not be shown within the transaction at all times. If a section is hidden, you can indicate whether the customised fields in that section should be shown regardless by right-clicking on the section title, and selecting **Always visible**. If this is set, and the section is not shown, then the fields in that section are shown in the standard extra data section at the bottom of the page.

New sections may be added to the page by dragging a **Custom section** from the **Special fields** area of the editor onto the last line of the preceding section. The following dialog is shown:

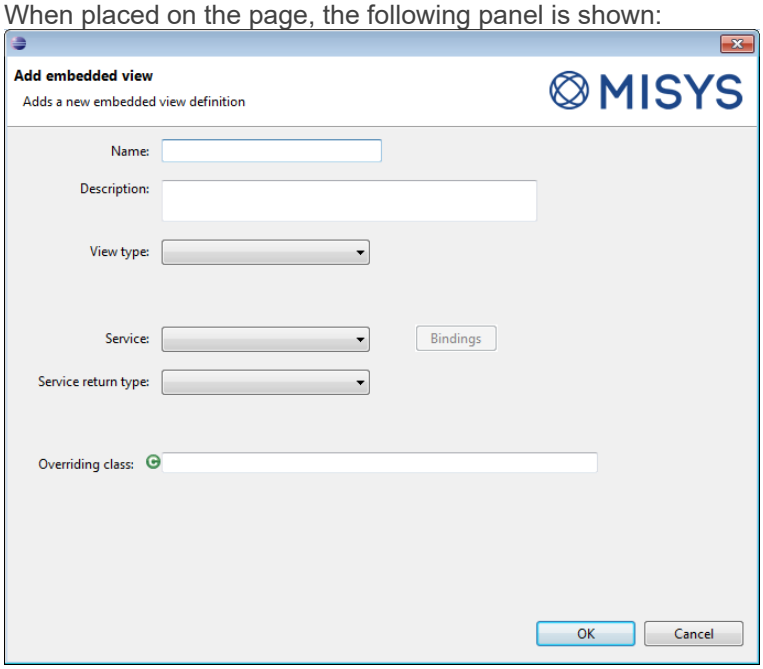


Enter a unique identifier for the page, and a section label. The new section is then shown in place:



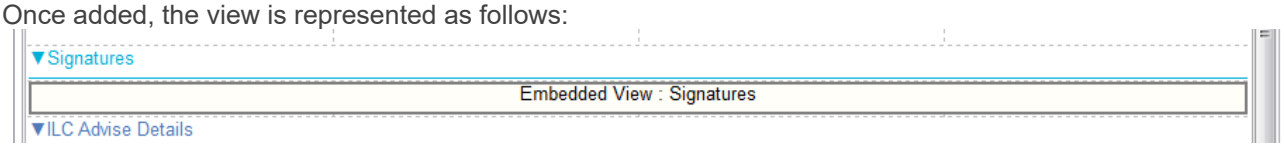
To add an embedded view, you must first add a custom section, before dragging an **Embedded View** from the **Special fields** section of the editor onto the empty line of the new section. It will fail if something already exists in the section.

When placed on the page, the following panel is shown:



This is the same panel as used by a linked view, except without the menu caption and section title. Refer to the earlier section on adding a linked view for more details.

Once added, the view is represented as follows:



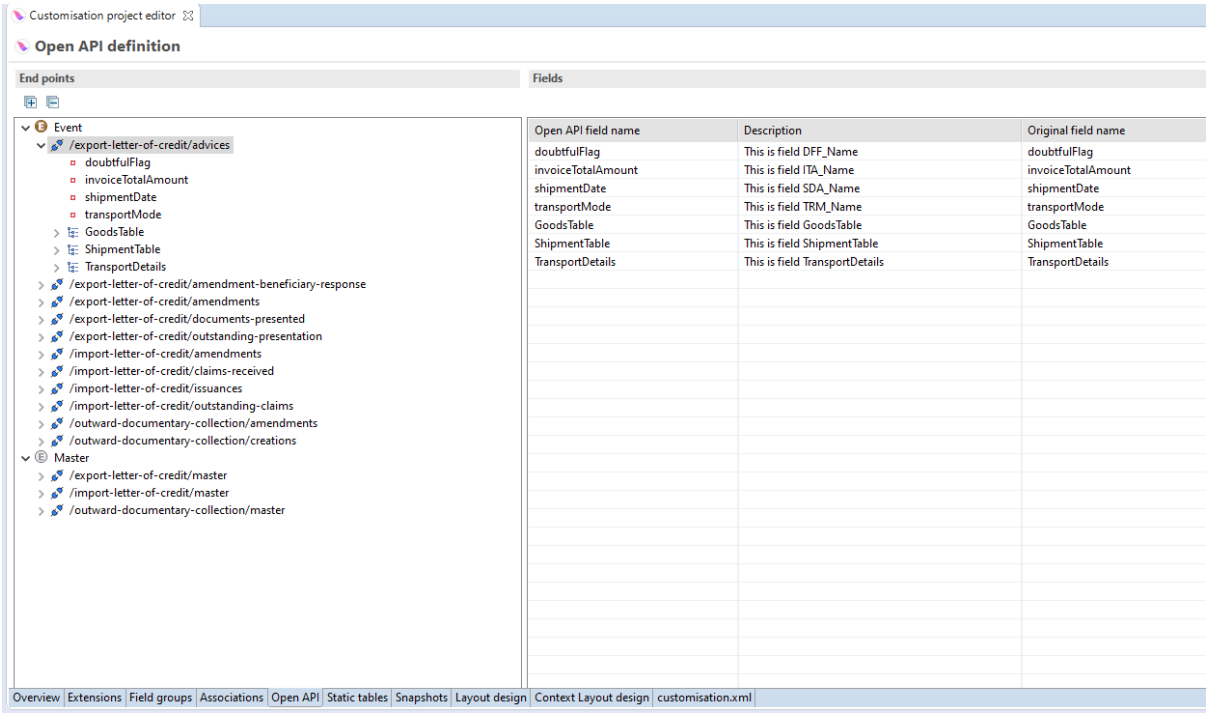
To increase the size of the section, right-click on the view and select **Resize…**

The rest of the layout editor works in the same way as the common layout editor.

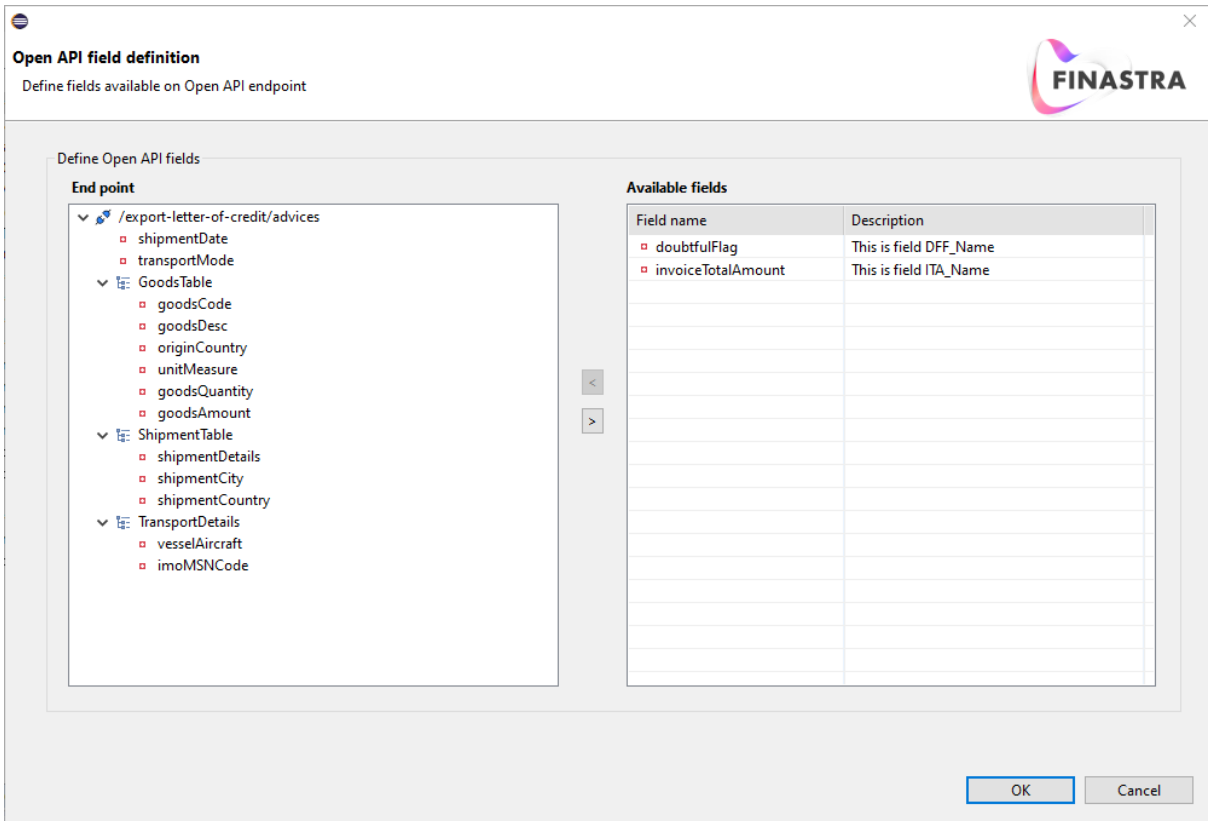
Open API Tab

For users that have implemented Trade Innovation Open APIs, customisation fields can be added to the **custom-fields** schema of the JSON payload.

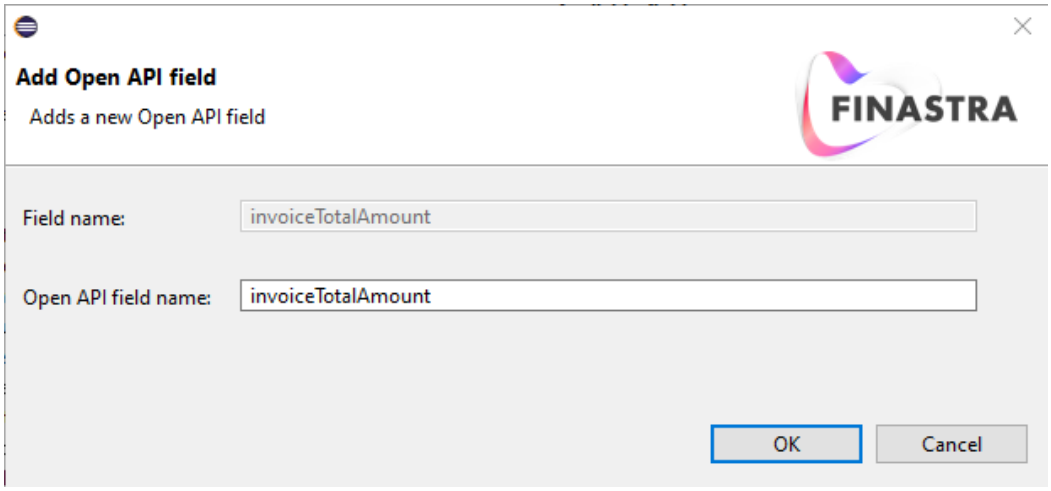
When there is an association present for the master/event combination supported by the Open APIs, the available end points will be shown. The fields to be added the Open API end points can then be defined.



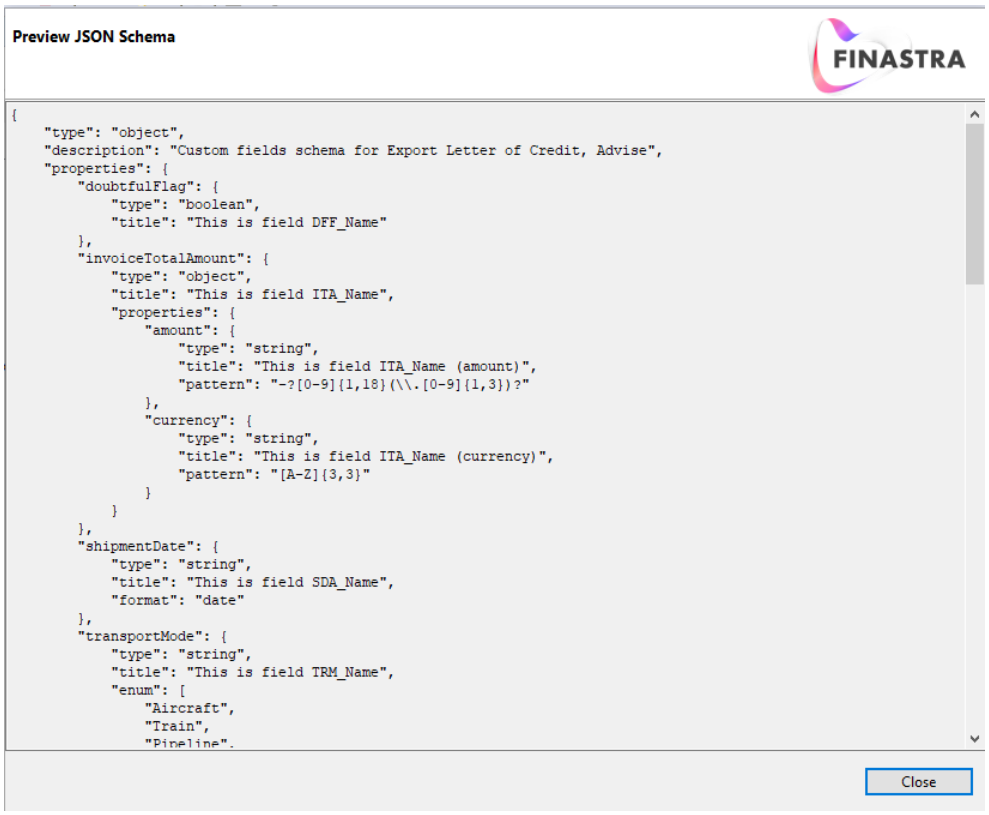
To define fields on an Open API end point, right-click on the end point and select **Define fields…**



The fields can be added to or removed from the Open API message by using the buttons. When a field is added, a dialog will pop up to allow the user to confirm the addition and if required, the name of the field on the Open API message can be changed to be different from the field name given in the customisation editor.

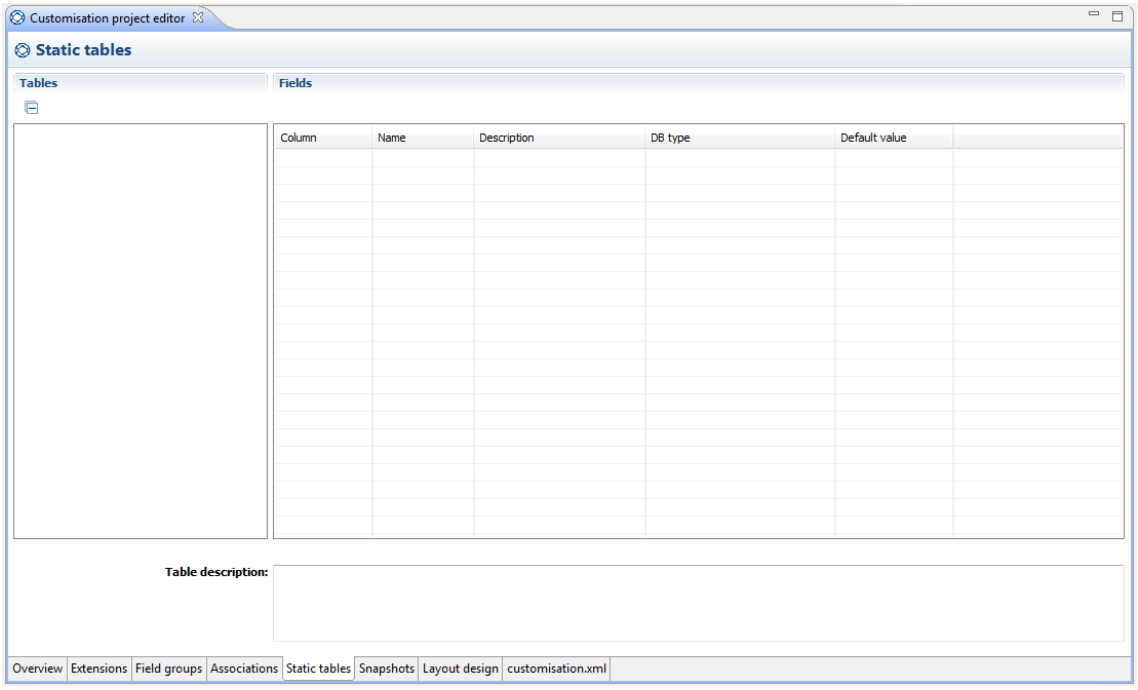


To preview the JSON schema for the Open API custom fields right-click on an entry point and select **Preview JSON schema…**

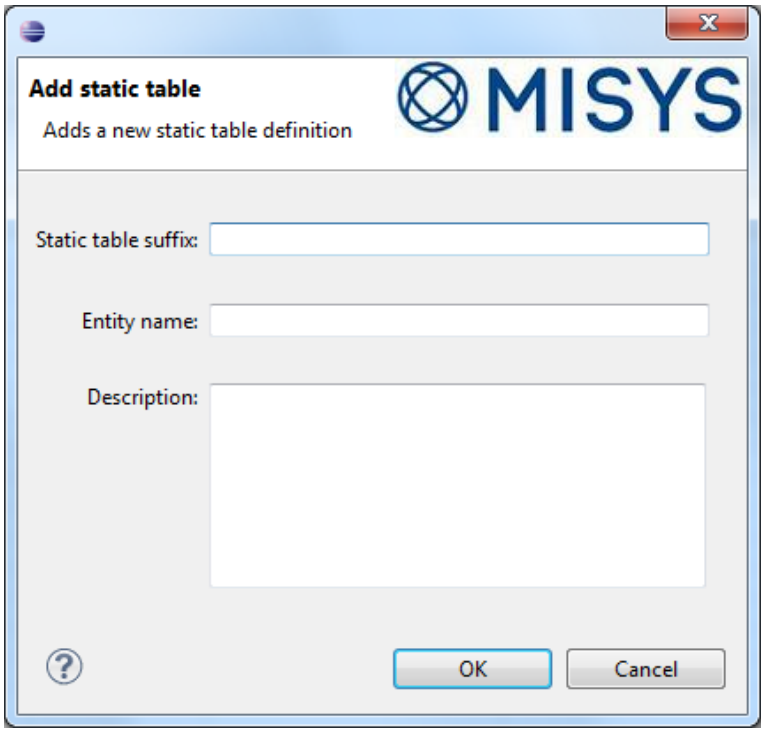

Preview JSON Schema
FINASTRA
{
"type": "object",
"description": "Custom fields schema for Export Letter of Credit, Advise",
"properties": {
"doubtfulFlag": {
"type": "boolean",
"title": "This is field DFF_Name"
},
"invoiceTotalAmount": {
"type": "object",
"title": "This is field ITA_Name",
"properties": {
"amount": {
"type": "string",
"title": "This is field ITA_Name (amount)",
"pattern": "-?[0-9]{1,18}(\\.[0-9]{1,3})?"
},
"currency": {
"type": "string",
"title": "This is field ITA_Name (currency)",
"pattern": "[A-Z]{3,3}"
}
}
},
"shipmentDate": {
"type": "string",
"title": "This is field SDA_Name",
"format": "date"
},
"transportMode": {
"type": "string",
"title": "This is field TRM_Name",
"enum": [
"Aircraft",
"Train",
"Pipeline",
Close

Static Tables Tab

New static data tables can be defined from the static **Static tables** tab.

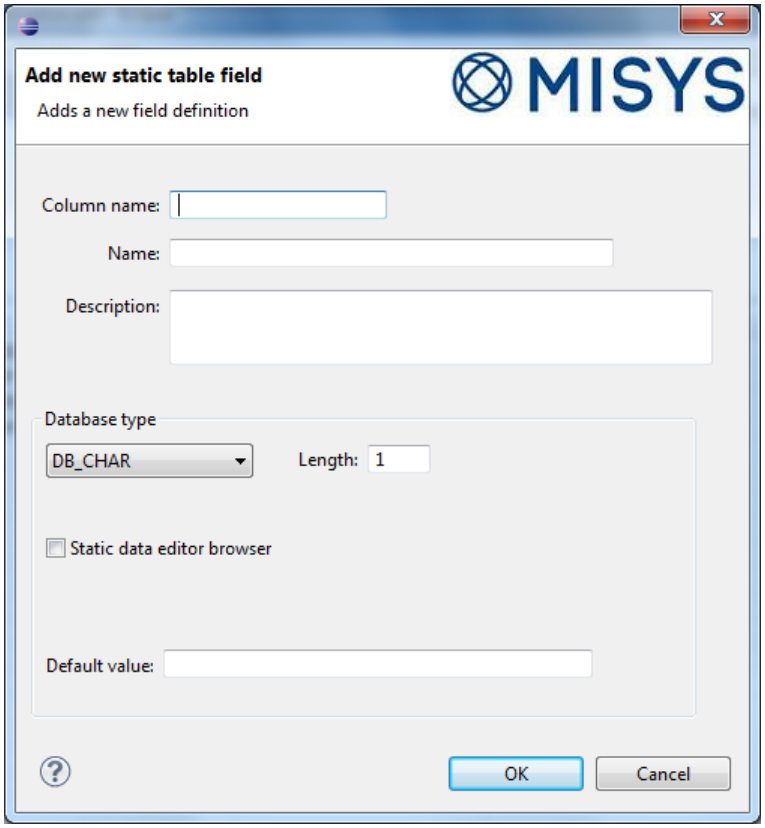


From the **Tables** list, right-click and select **Add table…**.



Which allows you to enter the table name suffix (this is combined with the prefix form the **Overview** tab).

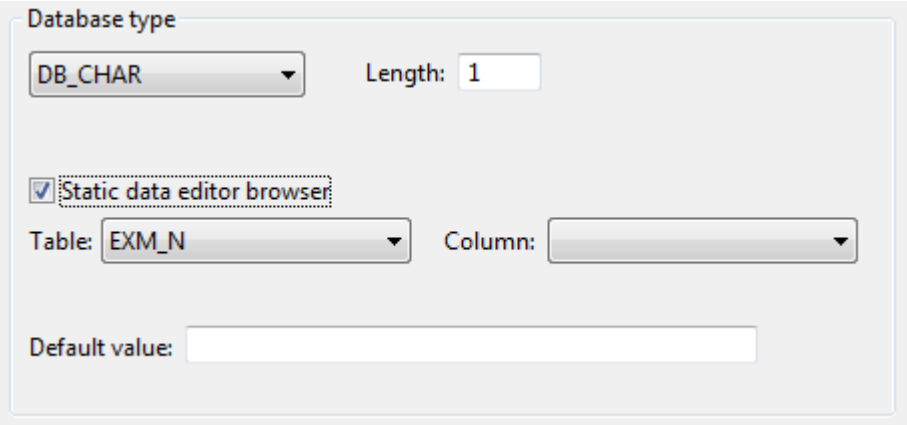
To add a field, right-click on the relevant table entry and select **Add field**.



The fields are defined similarly to the extension fields.

Note that the first field defined becomes the key field for the table.

If a column can use another static table to enter the data, then check the static data editor browser checkbox



Then specify the table and key column name accordingly.

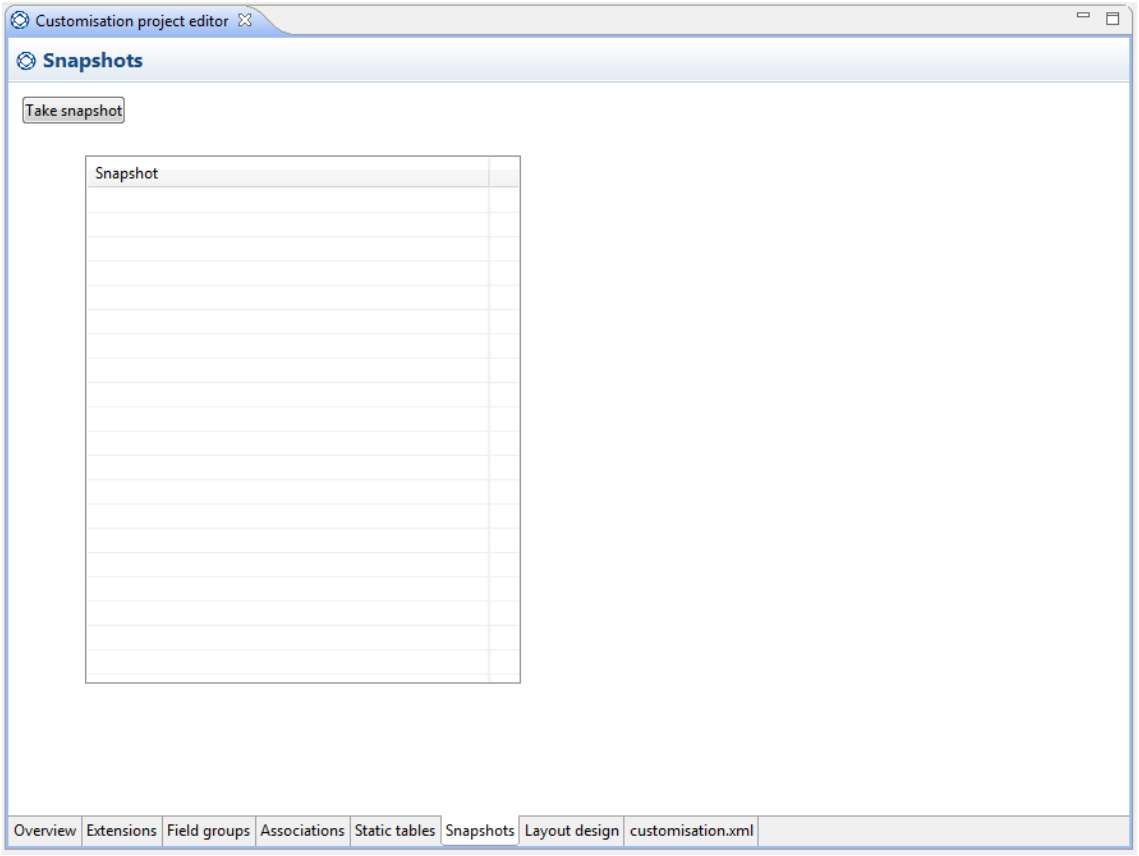
Snapshots Tab

This tab allows you to take a snapshot of the customisation.xml project file. This is done for two reasons:

- Since there is no unlimited undo facility, it is advisable to take a snapshot of the `customisation.xml` file before making a set of significant changes.
- Control how database upgrade scripts are derived

For a full explanation on how the database scripts are generated, see the later section on building customised components.

From the **Snapshots** tab:

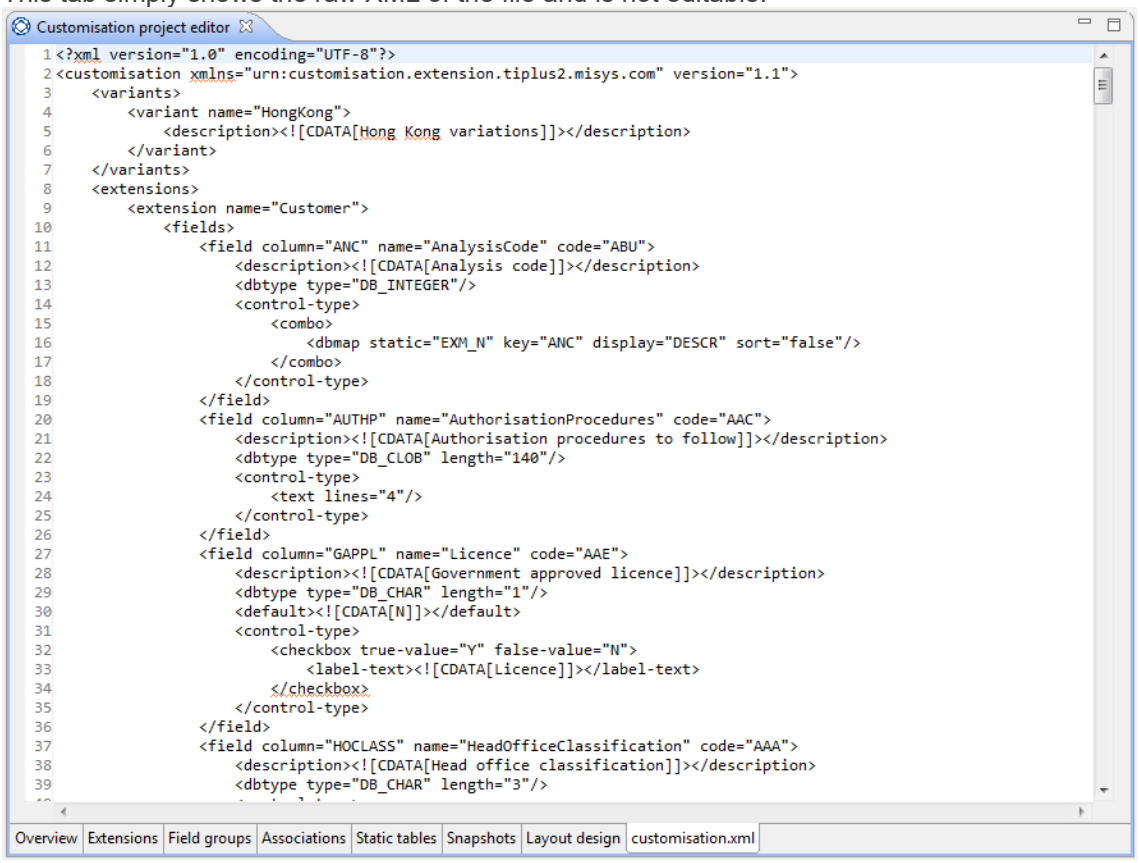


Click the Take snapshot button. A new version and timestamp-based entry will be listed. These entries correspond to files held in the snapshots folder of the project.

You can recover to recover to a particular snapshot by closing the customisation editor, delete the `customisation.xml` file, and copy the relevant file to the project root, renaming it to `customisation.xml`. However there are knock-on effects if you have already tried to deploy the project before reverting.

Customisation.xml Tab

This tab simply shows the raw XML of the file and is not editable.




```
<?xml version="1.0" encoding="UTF-8"?>
<customisation xmlns="urn:customisation.extension.tiplus2.misys.com" version="1.1">
    <variants>
        <variant name="HongKong">
            <description><![CDATA[Hong Kong variations]]></description>
        </variant>
    </variants>
    <extensions>
        <extension name="Customer">
            <fields>
                <field column="ANC" name="AnalysisCode" code="ABU">
                    <description><![CDATA[Analysis code]]></description>
                    <dbtype type="DB_INTEGER"/>
                    <control-type>
                        <combo>
                            <dbmap static="EXM_N" key="ANC" display="DESCR" sort="false"/>
                        </combo>
                    </control-type>
                </field>
                <field column="AUTHP" name="AuthorisationProcedures" code="AAC">
                    <description><![CDATA[Authorisation procedures to follow]]></description>
                    <dbtype type="DB_CLOB" length="140"/>
                    <control-type>
                        <text lines="4"/>
                    </control-type>
                </field>
                <field column="GAPPL" name="Licence" code="AAE">
                    <description><![CDATA[Government approved licence]]></description>
                    <dbtype type="DB_CHAR" length="1"/>
                    <default><![CDATA[N]]></default>
                    <control-type>
                        <checkbox true-value="Y" false-value="N">
                            <label-text><![CDATA[Licence]]></label-text>
                        </checkbox>
                    </control-type>
                </field>
                <field column="HOCLASS" name="HeadOfficeClassification" code="AAA">
                    <description><![CDATA[Head office classification]]></description>
                    <dbtype type="DB_CHAR" length="3"/>
```

Opening an Existing Cutomisation Definition

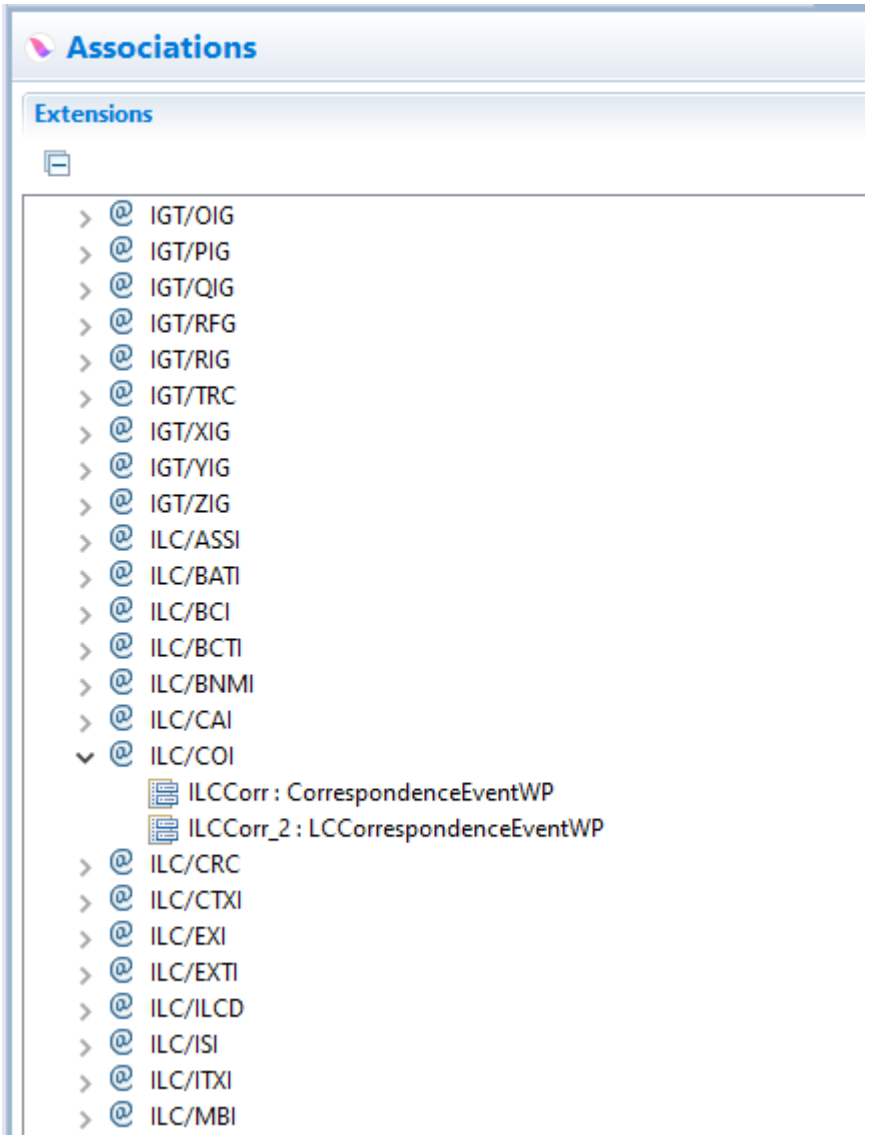
If a new version of the customisation editor is delivered, a project's definition may go through some automatic upgrades to make the definition consistent with the new software.

Version 1.6.2

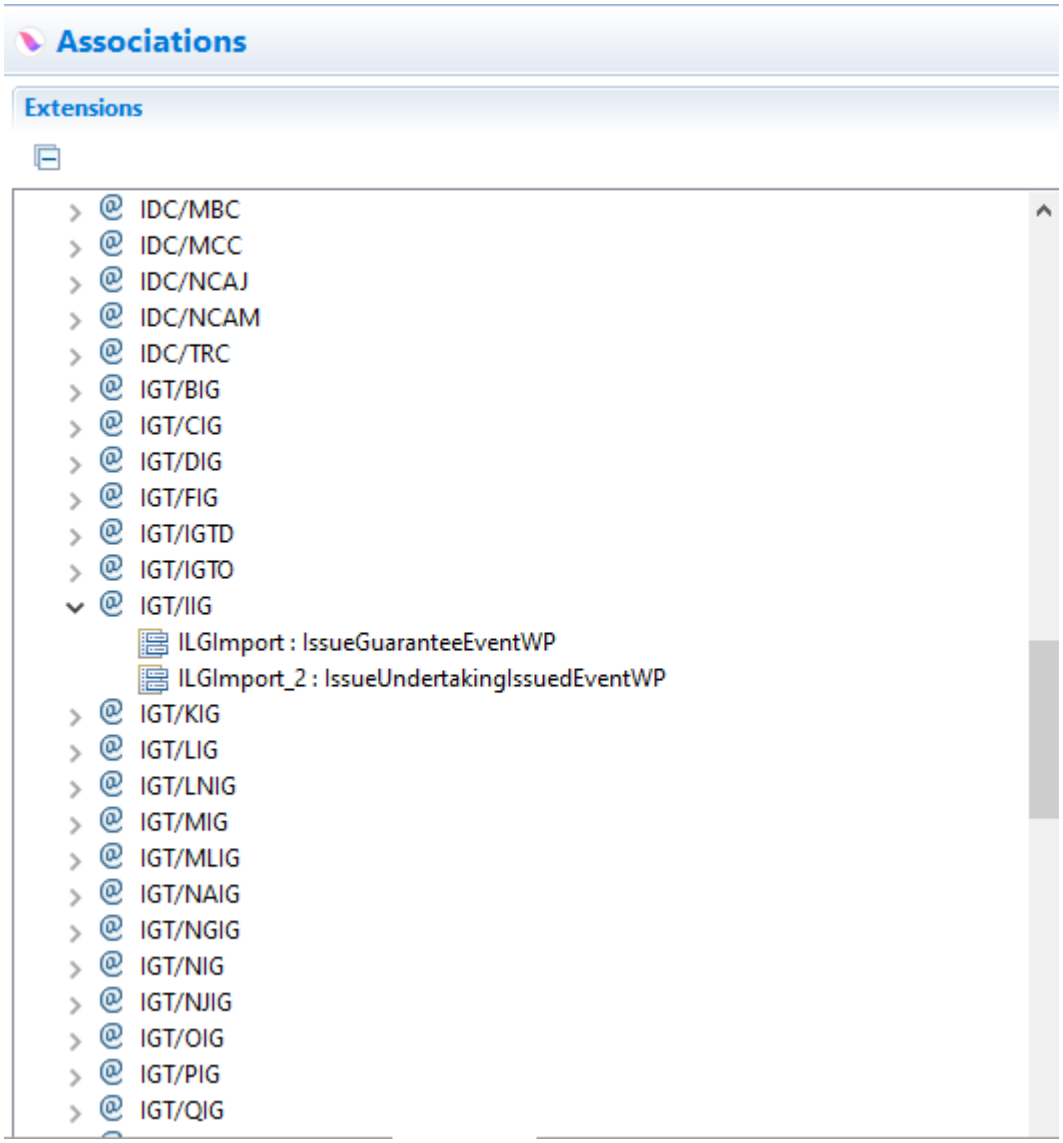
When a project definition is opened for use in Trade Innovation 2.9.1.0 or above, the following adjustments will be made.

- The following Correspondence associations are now invalid: CRCS, CEGS, COES, CESS, COFS, CIGS, COIS, CISS, RCOS.

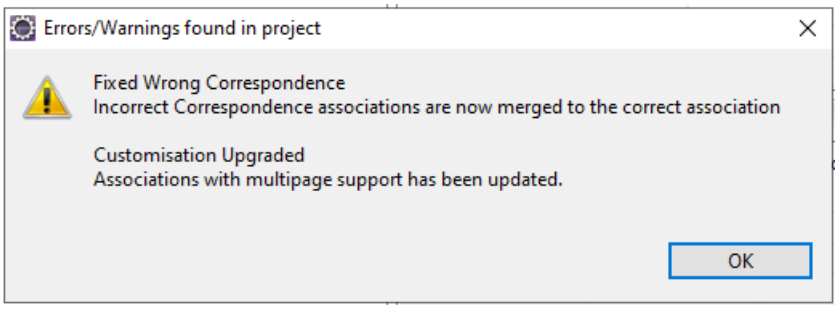
  Layouts created for these associations will be automatically moved to the correct association.



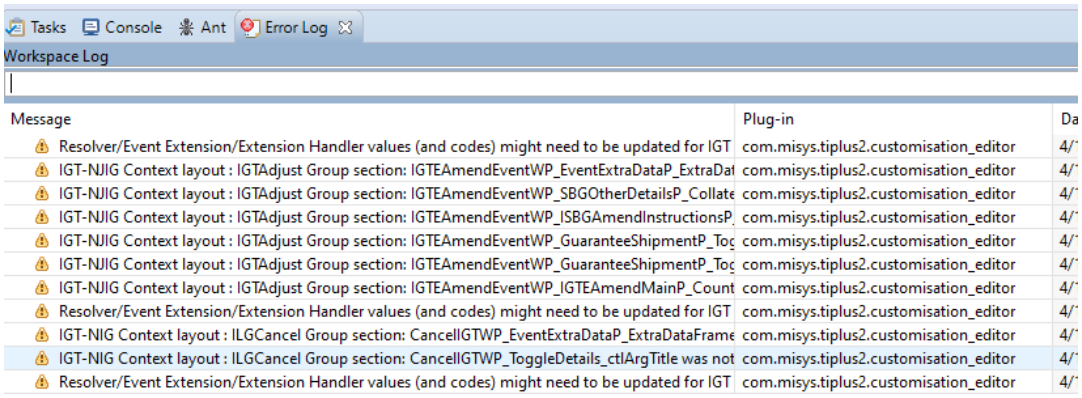
- Existing customised layouts for associations that now have multiple alternate pages (currently Events related to Undertakings) will now have layouts generated for the new page (for Undertakings)
  - Generated simple layouts will have the same fields under Extra data
  - Generated context layouts will try to match Section titles where possible and move the fields to same sections, otherwise the section will go under Extra data as a Custom section



Note: After the upgrade an Errors/warning dialog may be shown if any association was updated.



More information can be seen in Error Log view about the updates done, and if any user action is needed


Tasks
Console
Ant
Error Log
Workspace Log
Message
Plug-in
Da
Resolver/Event Extension/Extension Handler values (and codes) might need to be updated for IGT
com.misys.tiplus2.customisation_editor
IGT-NJIG Context layout : IGTAdjust Group section: IGTEAmendEventWP_EventExtraDataP_ExtraDat
com.misys.tiplus2.customisation_editor
IGT-NJIG Context layout : IGTAdjust Group section: IGTEAmendEventWP_SBGOtherDetailsP_Collate
com.misys.tiplus2.customisation_editor
IGT-NJIG Context layout : IGTAdjust Group section: IGTEAmendEventWP_ISBGAmendInstructionsP_
com.misys.tiplus2.customisation_editor
IGT-NJIG Context layout : IGTAdjust Group section: IGTEAmendEventWP_GuaranteeShipmentP_Toc
com.misys.tiplus2.customisation_editor
IGT-NJIG Context layout : IGTAdjust Group section: IGTEAmendEventWP_GuaranteeShipmentP_Toc
com.misys.tiplus2.customisation_editor
IGT-NJIG Context layout : IGTAdjust Group section: IGTEAmendEventWP_IGTEAmendMainP_Count
com.misys.tiplus2.customisation_editor
Resolver/Event Extension/Extension Handler values (and codes) might need to be updated for IGT
com.misys.tiplus2.customisation_editor
IGT-NIG Context layout : ILGCancel Group section: CancelIGTWP_EventExtraDataP_ExtraDataFrame
com.misys.tiplus2.customisation_editor
IGT-NIG Context layout : ILGCancel Group section: CancelIGTWP_ToggleDetails_ctlArgTitle was not
com.misys.tiplus2.customisation_editor
Resolver/Event Extension/Extension Handler values (and codes) might need to be updated for IGT
com.misys.tiplus2.customisation_editor

Enhancing Customisation

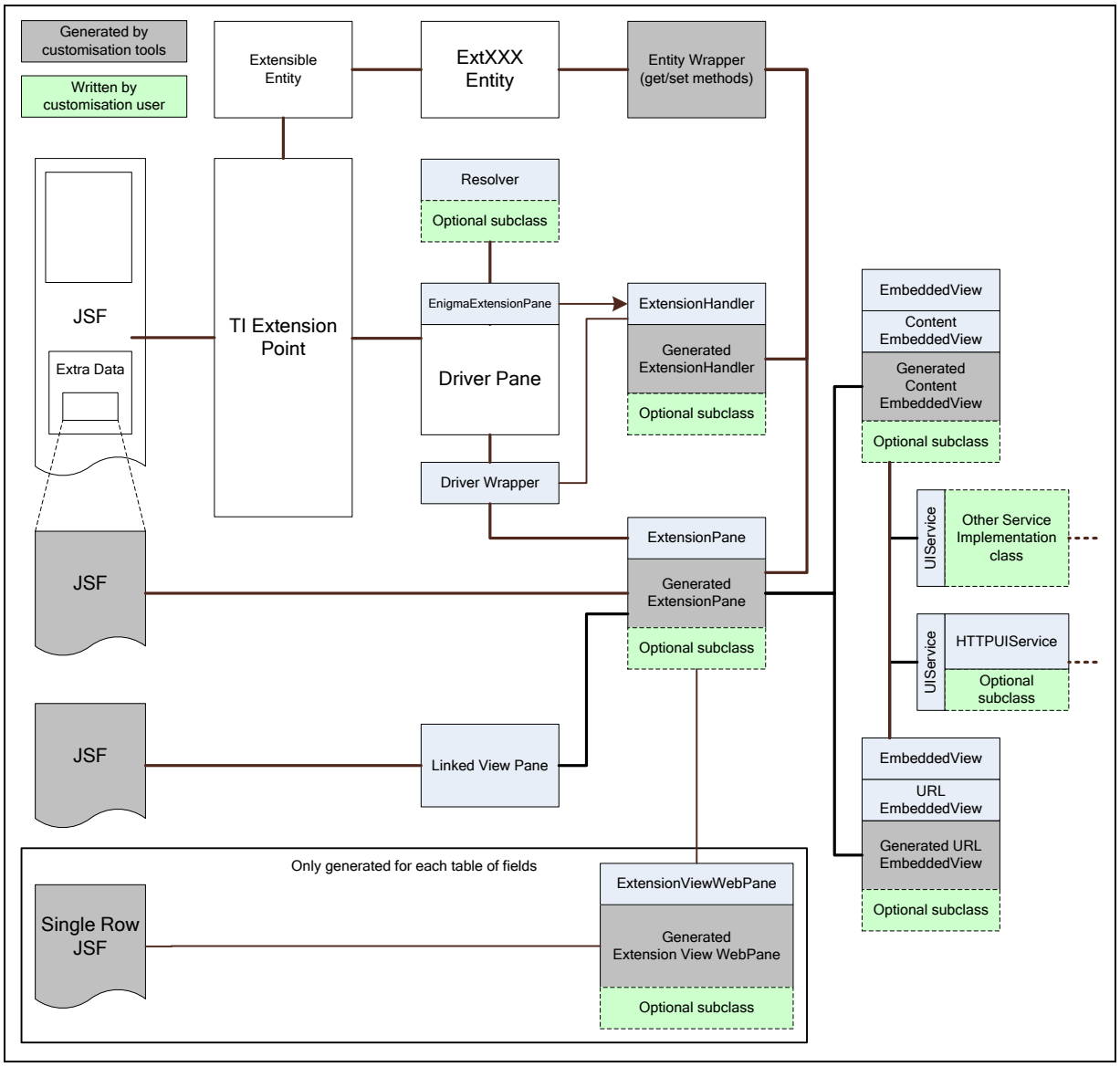
Once the details associated with the required customisation have been defined using the customisation editor, creating new Java classes can enhance functionality.

Overview

The following are types of enhanced customisation behaviour that can be achieve by writing subclasses of the generated code:

- Resolver – where multiple layouts have been defined for a particular association, the resolver class must provide the ability to choose the layout.
- Extension handler – This allows extra functionality to be added during initialisation, validation, saving, and is where the majority of posting functionality must be implemented.
- Extension pane – This allows individual field validation to be defined.
- Extension view pane – This allows individual field validation for a row in a table
- Embedded views – to derive a URL or content (Advanced SDK only)
- UI Service – to optionally interact with an external system to retrieve information for an embedded view (Advanced SDK only)

The following diagram provides an overview of the components involved:



A Trade Innovation extension point uses a driver pane to initially work out which `ExtensionHandler/ExtensionPane/Layout` to use using a `Resolver` class. From then on, the driver pane is used to communicate with the extension handler as part of the data entry processing. It also serves as a callback object via the `DriverWrapper` class.

The page content is served via a generated JSPX file to be included in the extension point's page. For common layouts, the fields are shown in a section at the bottom of the page. If context layouts are used, the fields are served in the same way via a single JSPX file, however javascript in the browser moves the respective fields to their targets.

Where a table has been defined, an extra JSF is generated, as well as an `ExtensionViewWebPane` to handle the validation of the data for a single row of the table.

Where linked views have been defined, a common page is displayed that uses the specific embedded view to show in isolation. In this way, all views ultimately rely on embedded view components.

Access to the extended entity is via an entity `Wrapper` that has getters and setters for the individual extended fields.

Any subclasses created must be defined in the `src/main/java` source folder of the customisation project. Thought should be given as to which package these classes should reside in.

Resolver

Where an association has multiple layouts assigned, there needs to be some logic at runtime deciding which layout to show. It is only possible to define the use of a resolver class for *Event* extensions, and it must be specified in order to attach multiple layouts to the association.

The class to subclass is `MultipleLayoutAssociationResolver` in the `com.misys.tiplus2.enigma.customisation.extensions` package.

If the Association has multiple alternate pages, the class to sub-class is `MultiPageLayoutAssociationResolver` in the `com.misys.tiplus2.enigma.customisation.extensions` package.

The methods to override are:

`<constructor>()`

The default constructor should be used to set a default layout by calling the `setDefaultLayout()` method passing the layout name. If no default layout is specified, then the first layout attached to the association is used.

```
String resolveLayout(EventFieldAccess eventFieldAccess)
```

If using `MultiPageLayoutAssociationResolver`:

```
String resolveLayout(String pageId, EventFieldAccess eventFieldAccess)
```

The `eventFieldAccess` object has a single method called `getEventFieldAsText()` with parameters as described in the previous section. This method must return the layout to use. If the layout returned is not recognised, then an error is logged, and the existing layout is used.

Extension Handler

Extension handlers may be subclassed and particular methods overridden that will be invoked at the appropriate points in the runtime processing. These points are:

- Initialisation
- Validation
- Calculated event field retrieval
- Saving
- Posting-related (adding extra fields, netting etc.)

The default extension handlers are called `CustomerExtension`, `EventExtension`, etc., and are located in the `com.misys.tiplus2.customisation.extensions` package.

Initialisation

There are common initialisation methods for each extended entity type. The generated code will perform all of the necessary initialisation appropriate to the extension. At the end of the initialisation, a method called `onPostInitialise()` is invoked. This method may be overridden in the extension handler to perform any extended initialisation.

The return value is a Boolean – return `true` if initialisation succeeded, return `false` otherwise.

For the *Event* extension, during the initialisation of a subsidiary event, extended master details that were mirrored from the event that are in the field group associated with the layout selected for the association are automatically copied from the extended master to the extended event details. This is automatic and cannot be overridden.

Validation

As part of the validation processing, the generated code will call the validation methods on the extension pane, and collect any errors. At the end of the validation processing, the method `onValidate()` will be called. This method may be overridden to perform any more complex validation.

A single `ValidationDetails` parameter is passed. This object is available to have errors and warnings added to it to be returned to Trade Innovation and be displayed.

The methods on this object to use are:

```
void addError(ErrorType type, String message)
```

```
void addWarning(WarningType type, String message)
```

The type enumerations are defined in the `ValidationDetails` class.

In the following example, the validation performed ensures that the user has entered at least one of two fields - `InstructionsToReceivingBank` and `ObjectOfContract`:

```
@Override
public void onValidate(ValidationDetails validationDetails) {
    if ((getPane().getObjectOfContract() == null ||
        getPane().getObjectOfContract().equals("")) &&
        (getPane().getInstructionsToReceivingBank()==null ||
         getPane().getInstructionsToReceivingBank().equals("")) {
        validationDetails.addError(ErrorType.MustBeEntered,
              "One of instructions to receiving bank or object of contract must be
entered.");
    }
}
```

It is not necessary to call the parent `onValidate()` method, however if you have created a hierarchy of extension handlers, then it may be necessary.

During an end of day process, an event may be input and released in one process. For an *Event* extension, an extra processing method is available to recognise when this occurs. The method is called `onAutoEventProcessing()`. This method may be overridden. It returns a Boolean – return `true` if processing was successful, otherwise return `false`.

Calculated Event Field Retrieval

There are two types of event field code within customisation extensions:

- Standard codes defined using the customisation editor that correspond to data held on the extension details
- User-defined codes, which you can add yourself programmatically. These can be set up to return calculated values which may or may not involve the extra fields

The runtime calls into the generated code to get a list of field codes. The codes defined using the customisation editor are filled in automatically. Once they had been set, a call is made to the `initialiseUserFieldCodes()` method. This method may be overridden to add the field codes corresponding to non-stored calculated fields. A field code may be added to the list by calling the `addFieldCode()` method which takes the following parameters:

```
addFieldCode(String code, String description, FieldCodeType type)
```

The `FieldCodeType` field is an enumeration that can take the values of `Money`, `Boolean`, `Date`, `Integer`, `List` and `String`. An example of using this method is as follows:

```
@Override
public void initialiseUserFieldCodes() {
    addFieldCode("cXYZ", "description of new field", FieldCodeType.String);
}
```

Note that all customisation field codes start with a lower case '`c`'. This also applies to the codes defined in the customisation editor – the prefix is added in any generated code.

When Trade Innovation needs to access a customised event field, it calls into the generated code which will handle the codes defined in the customisation editor, and for those it does not recognise will call a method based on the field code type:

```
String getUserFieldCodeString(String code)
String getUserFieldCodeList(String code)
Integer getUserFieldCodeInteger(String code)
Boolean getUserFieldCodeBoolean(String code)
String getUserFieldCodeAmount(String code)
Date getUserFieldCodeDate(String code)
```

Note that any other data type used, such as a non-money related decimal value (for instance a rate) is treated as a string.

These methods can be overridden to return the calculated values. It is only necessary to override the methods where there are user field codes of the associated type.

The format to return for the `getUserFieldCodeAmount()` is a string that has the currency mnemonic either as the first three characters, or the last three.

Saving

For the *Event*, *Customer* and *TICustomer* extensions, Trade Innovation calls into the generated code when the fields from the pane need to be copied to the extended entity. After this is done, a call is made to the `onSaving()` method. This method may be overridden to perform any extra appropriate processing. The method returns a Boolean – return `true` if your code was successful, otherwise return `false`.

In the Event extension, details from the extended master entity are updated from the event on release. They are also updated each time a full defining event is submitted or pended.

The methods are `updateMaster()` and `updateMasterOnRelease()`. These methods may be overridden and any appropriate processing added. Note in these cases, the overridden method **must** be invoked as part of this additional processing. Also, it is not possible to update the extended entity of the Master – it is simply a trigger point.

Posting Related

The extra fields for postings are defined in the usual way using the Customisation Editor. However, there is no data entry facility for postings. There is only a read-only pane for displaying the values. You must provide the code that sets up the data in the extra fields.

Two methods can be overridden to set up the postings extra fields:

```
void addExtraFields()
```

```
void addExtraSettlementFields()
```

`addExtraFields` is the method you should use for adding simple data that might be used to influence netting, and `addExtraSettlementFields` should be used when extra data needs to reflect the actual settlement of the amount, such as the actual account being posted to (see Chapter 2).

From these methods you have access to information about the current posting and information about the current context of the event using the event field mechanism.

The `addExtraFields` method is used to generate raw postings or accrual postings. The code below provides several examples of its use:

```
@Override
Public void addExtraFields() {
    getWrapper().setMasterCategorisationCode(
        getDriverWrapper().getEventFieldAsText("cAAJ ", "s", "", "E"));

    getWrapper().setMasterOverallExpiryDate(
        getDriverWrapper().getEventFieldAsText("cABE ", "d", "", "E"));


    getWrapper().setPartyTaxApplicable(
        getDriverWrapper().getPostingFieldAsText("PTY", "cAAH"));

    String tCcy = getDriverWrapper().
        getEventFieldAsText("FOA", "v", "c", "E");
    String tAmt = getDriverWrapper().
        getEventFieldAsText("FOA", "v", "a", "E");
    getWrapper().setPartyQuotaAmountCurrency(tCCy);
    if (tAmt != null && !tAmt.equals("")) {
        getWrapper().setPartyQuotaAmount(
            getDriverWrapper().convertFromToDBFormat(tAmt, tCcy, "T"));
    }
}
```

In these examples `MasterCategorisationCode`, `MasterOverallExpiry`, `PartyTaxApplicable`, `PartyQuotaAmount` and `PartyQuotaAmountCurrency` are four extra fields for the Posting extension. Note that since `PartyQuotaAmount` is of type `Money`, a derived field of `PartyQuotaAmountCurrency` is generated automatically.

`MasterCategorisationCode` and `MasterOverallExpiry` are set up from extra event fields by making use of event field mechanism.

`PartyTaxApplicable` is set up with a different method called `getPostingFieldAsText`. This is similar to `getEventFieldAsText`. `getPostingFieldAsText` is a different method under the event field mechanism and it is used to get access to the current posting details.

From the *Event Field Calculator*, you should be able to tell that the code `PTY` is used to get hold of the posting party. Because party is a complex data type in Trade Innovation and has many different parts (fields or data members), the second parameter (part code) in the above code of line 3 is used to access a particular part of the posting party. Before you use a part code (like `cAAH`), make sure that you can see it from the *Event Field Calculator*. Note that because field code `PTY` is a party and `cAAH` is prefixed with the letter `c`, it is indicating that an extra field from the *Customer* and/or *TICustomer* extension is being accessed.

The `addExtraSettlementFields` method can be coded in a similar way.

To help you with adding extra fields to postings, a special posting code is provided to be able to recognise what type of posting this is. To get this posting code for any raw posting, you can use:

```
getDriverWrapper().getPostingCodeAndDescription()
```

This returns a string with the code and description separated by a ‘`:`’.

Raw postings will be netted into netted postings, and accrual postings could be bulked together. The method `checkDoNettingOrBulking` is used to check whether the extra posting fields are compatible for netting or bulking purposes.

The default implementation simply checks whether the extra fields on the netted posting (`getWrapper()`) are the same as the comparison raw posting (`getCompWrapper()`) – if they are then this method returns true to net or bulk, otherwise it will return false. Override this method to perform your own check. It is passed a String parameter of the posting type – the posting types can be:

- `ADV` – Advance
- `CHA` – Charge
- `COL` – Collateral
- `DIS` – Discounted
- `EVE` – Event
- `FX` – FX
- `FIN` – Financing
- `FIR` – Financing repayment
- `MAN` – Manual
- `PAP` – Part pay
- `REP` – Repayment
- `PCA` – Periodic charge accrual
- `FAP` – Financing accrual

To indicate whether the posting should net you should pass true or false by calling the following:

```
getParameters().getBSuccess().assignEn(true|false);
```

If a posting is to be netted, then the method `netWith(String postingType)` is called.

The default implementation simply copies the data from the comparison raw posting to the current netted posting. Override this method if you want to change this.

When postings are released, just before they are copied to the staging tables for transfer to the relevant back office system, it is possible to update netted posting fields. This can be done by overriding the method `updateNettedPostingOnRelease()`.

Access Methods

To access the extended entity, call `getWrapper()`.

To access the extension pane, call `getPane()`.

To access the Trade Innovation component, call `getDriverWrapper()`

For the *Posting* extension, you can also call `getCompWrapper()` for the comparison posting extension.

Driver Wrapper

To access information from Trade Innovation, the following methods are available from the Trade Innovation Component using the `getDriverWrapper()` method.

convertFromToDBFormat

```
String convertFromToDBFormat(String amount, String currency, String fromTo)
```

Used to convert an amount to or from the database format where it will be stored as minor denominations. The `fromTo` parameter can be either `T` or `F`.

getAmountInBaseAtFXRate

```
String getAmountInBaseAtFXRate(String fxRateCode, String currency,
        String amount, String buySellMid)
```

Returns the base currency equivalent amount of that passed in `amount / currency`. The amount is converted to base currency equivalent using the fx rate code passed in `fxRateCode` / `buySellMid` if passed – if not, the conversion is via the spot rate. The `buySellMid` parameter can be `B`, `S` or `M`.

Only available for *Event*, *Master* and *Posting* extensions.

getAmountInBaseAtSpotRate

```
String getAmountInBaseAtSpotRate(String currency, String amount)
```

Returns the base currency equivalent amount of the amount passed in `amount / currency` using the spot rate.

Only available for *Event*, *Master* and *Posting* extensions.

getAmountInCurrency

```
String getAmountInCurrency(String amount1, String currency1,
        String currency2, String standardisedRate,
        String fxRatecode, String buySellMid)
```

Returns the equivalent amount in `currency2` of the amount passed in `amount1 / currency1`. The amount is converted to base currency equivalent using the standardised rate passed in `standardisedRate` or the fx rate code passed in `fxRateCode / buySellMid`, otherwise the conversion is via the spot rate. The `buySellMid` parameter can be `B`, `S` or `M`.

Only available for *Event*, *Master* and *Posting* extensions.

getBaseAtFXRate

```
String getBaseAtFXRate(String fxRateCode, String currency, String buySellMid)
```

Returns the normalised exchange rate for the currency passed in `currency`. The rate is determined by using the fx rate code passed in `fxRateCode / buySellMid`. The `buySellMid` parameter can be `B`, `S` or `M`.

Only available for *Event*, *Master* and *Posting* extensions.

getBaseAtSpotRate

```
String getBaseAtSpotRate(String currency)
```

Returns the normalised spot exchange rate for the currency passed in `currency`.

Only available for *Event*, *Master* and *Posting* extensions.

getCurrencyAtFXRate

```
String getAmountInCurrency(String currency1, String currency2,
        String fxRatecode, String buySellMid)
```

Returns the normalised cross currency rate for the currencies passed in `currency1` and `currency2`. The rate is converted using the fx rate code passed in `fxRateCode / buySellMid` if passed, otherwise the conversion is via the spot rates.

Only available for *Event*, *Master* and *Posting* extensions.

getCustomerFieldAsText

```
String getCustomerFieldAsText(String fieldName)
```

Return a specific piece of information related to Customer field `fieldname` (include spaces):

```
Mnemonic
Date Maintained
Mail Branch
Reference
Analysis Code
Customer Number
Customer Type
Full Name
Group
Account Officer
Parent Country
Resident Country
Risk Country
Local Clearing Number
Source Banking Business
```

Only available for *Customer* and *TICustomer* extensions.

getTICustomerFieldAsText

String getTICustomerFieldAsText(String fieldName)

Return a specific piece of information related to TICustomer field `fieldName` (include spaces)

```
Charge Group
Charge Group Description
Customer Mnemonic
Special Instructions
Work Group
Work Group Description
```

Only available for *Customer* and *TICustomer* extensions.

getEventFieldAsText

```
String getEventFieldAsText(String eventFieldCode, String eventFieldtype
        String part)
```

Return a specific piece of information related to a given event field code (`eventFieldCode`) and part code (`part`) when applicable by making use of the event field mechanism. The second parameter, `eventFieldType`, represents the event field type you are looking at and should use one of the constant names given on the list of Event Field Types:

```
a – account
v – amount
p – party
d – date
r – reference
n – narrative
l – logical
b – Branch
i – integer
s – string
'-' - address
f - System account
z – currency
y – country
c – customer
q - Party settlement
x - Watch list checker party report
u – User
t – Timestamp
+ - List
```

Only available for *Master* and *Event* extensions.

Note: This method allows the retrieval of event fields in the product/event context – as seen in the event field viewer. It does not allow for the retrieval of event fields in other contexts.

In some cases there may be alternative means of access. For instance, the fields pScu and pSno in the Party context are not accessible, as the fields repeat. However, you can retrieve part fields from party event fields. For instance, to retrieve the customer mnemonic of the beneficiary:

```
getDriverWrapper().getEventFieldAsText("BEN" ,
                        FieldCodeType.Party.getTypeString(), "cu");
```

where

`BEN`	the event field code for the Beneficiary
`cu`	is the part of the party field type corresponding to customer mnemonic

```
String getEventFieldAsText(String eventFieldCode, String eventFieldtype
        String part, String flag)
```

Same as previous method, except when `flag` is `M`, the information is retrieved directly from the Master, whereas if `flag` is `E`, then the information is retrieved from the Event first, and if not found, will retrieve it from the Master.

Only available for *Posting* extensions.

getProductType

```
String getProductType()
```

Return the product type of the current event.

Only available for *Posting* extensions.

getEventType

```
String getEventType()
```

Return the event type of the current event.

Only available for *Posting* extensions.

getPostingFieldAsText

```
String getPostingFieldAsText(String code, String partCode)
```

Similar to the method `getEventFieldAsText`, but the information is for the current posting.

Only available for *Posting* extensions.

isSubsidiary

```
boolean isSubsidiary()
```

Returns `true` if this is a subsidiary event.

Only available for *Event* extensions.

logError

```
void logError(String message)
```

Writes an error message to the log.

Accessing Static Tables Directly

At some points during your input of data for extra fields, you may want to access some data from tables directly. This can be achieved by the `createQuery` method provided by the Trade Innovation run-time system. An example of utilising the `createQuery` method is as follows:

```
AdhocQuery<? extends com.misys.tiplus2.customisation.entity.EXM_R>
    queryRCODE = getDriverWrapper().createQuery("EXM_R", "ID='" + id + "'");
EXM_R registrationCode = queryRCODE.getUnique();
if (registrationCode != null) {
    /* Record found */
}
```

The `createQuery()` method is overloaded and can be passed one, two, three or four parameters:

- Name of the table to query
- Where clause using column names
- Order by clause using column names
- `Map<String, Object>` of parameters if where clause is parameterised

The methods available on the result are:

- `getUnique()` – returns a single entity (the first if more than one item is returned form the database)
- `iterator()` – returns an `Iterator<T>` for the result set.
- `getMoreRows()` – returns true if there were more rows above the system limit

This can only be used to read some data from static tables; getting data from transaction tables should be carried out by means of the event fields mechanism.

Extension Pane

The extension pane can be subclassed, and a series of `validateXXXX()` methods may be overridden. One method per field is generated using the logical name – for instance `validateCategorisationCode()`.

These methods should only validate the individual field. A single parameter of `ValidationDetails` is passed (refer to `onValidate` in the previous section). Access to the field value should be performed through a getter – for instance `getCategorisationCode()`.

All standard validation for the field based on its type, and whether it is mandatory is performed before these validation methods are called.

Where you have defined a table in an extension point, there is also an `ExtensionViewWebPane` that needs to be created per table for handling a row of the table. To be able to add to the validation for handling a single row, a subclass may be written, however, in order to create an instance of the subclass, the method:

createExt<extension><repeating field>ViewPane(String layoutName)

must be overridden to return it based on the layout name. The default implementation will simply create the default generated class.

The default extension panes are called `CustomerPane`, `EventPane`, etc., and are located in the `com.misys.tiplus2.customisation.pane` package.

Managing Control State

The extension pane provides access to the controls on a layout. For controls associated with fields, the same control name is used regardless of the layout. So to access the control for the field `CategorisationCode`, then the method `getCtlCategorisationCode()` is available, and it returns an `ExtendedXXXControlWrapper`, based on the control type.

The common utility methods on these objects are:

- setEnabled(boolean enabled)
- boolean isEnabled()
- setVisible(boolean visible)
- boolean isVisible()

For buttons and hyperlinks, they are specific to a particular layout, so the method is based on the Identifier and the layout name.

For a button with an identifier of `Retrieve` on a `Default` layout, the getter method has the name of `getBtnRetrieveDefault()`. The returned control allows the method `setCaption(String caption)` to change the caption of the button.

For a hyperlink with an identifier of `Signature` on a layout called `IssueLayout`, the getter method has the name of `getCtlSignatureIssueLayoutHyperlink()`. For a hyperlink, the methods `setURL(String URL)` and `setCapiton(String caption)` are also available.

Note: Hyperlinks are only available with the *Standard* and *Advanced* SDK.

Any labels that have been associated with a control that is not visible will also be hidden.

The layout name used can be retrieved by calling `getLayout()`.

Action Methods

For fields that are flagged as auto-submit, and for any buttons that are used, action methods are generated on the relevant extension pane.

Auto-submit action methods are named based on the field name and layout that they are on. This allows for different behaviour on different layouts. If this is not required, then one method can be implemented with the behaviour and the others simply call this one. For a field called `CategorisationCode` on a layout called `IssueLayout`, the method to override is `onCategorisationCodeIssueLayoutAutoSubmit()`.

When buttons are clicked, the same principal is followed – the method name called is based on the button identifier and layout name. So, for a button with the identifier `Recalculate` on the layout called `IssueLayout`, the method that will be invoked is `onRecalculateIssueLayoutButton()`.

Note: Buttons are only available with the *Standard* and *Advanced* SDK.

The expectation of these action methods would be to change the state of other controls, or may be call an API on an external system. These methods have no access to `ValidationDetails`, so if an error occurs, it should be thrown as an exception – the sort of errors expected is that an external system is unavailable. This will then be shown on an error page that can then be dismissed.

Extension View Web Pane

An extension view web pane represent the panel associated with repeating groups of fields. The naming convention of the generated pane is:

<extension><Layout>Ext<extension><Repeating field>ViewPane

So, for the event extension, for the layout IssueLayout and the repeating field Contacts, the name would be:

EventIssueLayoutExtEventContactsViewPane

The extension view pane can be subclassed to override:

- initialisation – postInitialisePane()
- Field validation - `validateXXXX()` – one method per field is generated using the logical name – for instance `validateContactName()`
- Row validation – `onValidate()`
- Action methods – see next section

The validation methods are passed a single parameter of `ValidationDetails` (refer to `onValidate` on the extension handler).

Access to a field value should be performed through a getter – for instance `getContactName()`.

All standard validation for a field based on its type, and whether it is mandatory is performed before the field level validation methods are called.

When all of the individual validation methods have been called, then the `onValidate()` method will be called.

Getter methods for controls are also available on the extension view.

Note: To create an instance of the subclassed pane, the main extension pane must be subclassed and the `createExt<extension><repeating field>ViewPane()` method implemented. Refer to the earlier extension pane.

Action Methods

For fields that are flagged as auto-submit, and for any buttons that are used, action methods are generated on the relevant extension pane.

Auto-submit action methods are named based on the field name and layout that they are on. For a field called `ContactName` on a layout called `IssueLayout`, the method to override is `onContactNameIssueLayoutAutoSubmit()`.

When buttons are pressed, the same principal is followed – the method name called is based on the button identifier and layout name. So, for a button with the identifier `Recalculate` on the layout called `IssueLayout`, the method that will be invoked is `onRecalculateIssueLayoutButton()`.

Note: Buttons are only available with the *Standard* and *Advanced* SDK.

The expectation of these action methods would be to change the state of other controls, or maybe call an API on an external system. These methods have no access to `ValidationDetails`, so if an error occurs, it should be thrown as an exception – the sort of errors expected is that an external system is unavailable. This will then be shown on an error page that can then be dismissed.

Embedded Views and UI Services

These features are only available if you are using the Advanced SDK.

In the following sections, the phrase embedded view will be used to include linked views, since linked views are standalone pages with a single embedded view.

There are two types of embedded view, one that generates a URL, and one that generates HTML content. In either case, the result is shown in an `<iframe>` in the browser.

URLEmbeddedView

To generate a URL, a template for that URL is defined with parameter substitution specified. These parameters are referenced using `${myField}` notation.

Parameters are defined by the service that the embedded view is associated with. The service, once called may also add to the list of parameters.

When the URL is requested as part of creating the page to show, the parameters defined for the UIService associated with the embedded view are resolved based on their bindings. If all mandatory parameters have values defined, then the service is retrieved, and if it is not null, is invoked.

The value returned by the service may be a single parameter value to add to the list under a pre-configured name, or may be a list of name/value pair properties that will be added to the list of parameters.

These parameters are then used to resolve the URL template. Any referenced parameter that does not have a value will be replaced with an empty string.

The resulting string after this parameter substitution is returned as the URL.

On subsequent page displays, the URL will only be re-calculated if any of the service parameter values have changed.

If this processing needs to be overridden, then a subclass can be written that extends the generated class:

`com.misys.tiplus2.customisation.view.<extension><layout><view>EmbeddedView`

The following methods may be overridden:

- `<constructor>` - this accepts the extension pane, and driver wrapper – simply call the parent implementation

- `getBoundParameters()` – returns the parameters after they have been bound
- `getUIService()` – instantiates and returns the UIService referenced
- `getUIServiceName()` – returns the name of the UIService referenced
- `resultIsParameters()` – return true if the result of the service is a set of properties
- `getResultParameterName()` – return the name of the parameter is the service returns a single parameter value
- `processResults()` – this method is passed the `Object` returned from the service and the parameters as a `Map<String, Object>` return the resulting URL as a `String`.

ContentEmbeddedView

For a content view, the process is similar as far as resolving the parameters before calling the referenced service.

In this case, the result of calling the service is expected to be content (HTML) or XML. If it is XML, then a referenced XSLT stylesheet will be used to transform the XML into HTML.

The XML can be returned as a:

- String
- javax.xml.transform.Source
- org.w3c.dom.Node.

If this processing needs to be overridden, then a subclass can be written that extends the generated class:

com.misys.tiplus2.customisation.view.<extension><layout><view>EmbeddedView

The following methods may be overridden:

- `<constructor>` - this accepts the extension pane, and driver wrapper – simply call the parent implementation
- `getBoundParameters()` – returns the parameters after they have been bound
- `getUIService()` – instantiates and returns the UIService referenced
- `getUIServiceName()` – returns the name of the UIService referenced
- `isResultXML()` – return true if the result of the service is XML
- `getStyleSheet()` – returns the name of the XSLT stylesheet used to transform the returned XML into HTML
- `processResults()` – this method is passed the `Object` returned from the service and the parameters as a `Map<String, Object>`. Return the resulting content as a `String`.

UIService

All services implement the UIService interface that has a single method:

```
Object call(Map<String, Object> parameters)
```

The parameter map will contain:

- all of the bound parameters
- the environment entries
- two standard parameters of `CurrentUser` and `CurrentZone`.

The `HTTPUIService` class supplied performs the equivalent of a simple HTTP GET operation on a resolved URL template defined for the service. This is performed by:

- creating a `java.net.URL` object
- opening a connection on the URL
- setting the connection request properties
- opening an input stream
- reading the input stream until the stream finishes

Given those steps it can be seen that redirect-responses are not detected or actioned. If you wish to allow this, then you may create a subclass to override this behaviour.

The following methods can be overridden:

- `<constructor>` - accepts the URL template as a string
- `call()` – the interface method
- `addRequestProperties()` – this method is passed the URLConnection that has been opened so that request properties can be added to it
- `callHTTP()` – this method is passes the URL as a string, and expects to return the result from the GET operated on the URL.

For an 'other' service, the implementation must be supplied. The constructor is expected to be the default constructor. If more information needs to be passed to the other service, the `getUIService()` method of the embedded views that use the service needs to be overridden to pass relevant values to the constructor.

Displaying Results

Whether the embedded view generates a URL or HTML content, an `<iframe>` is used to display the results. The `sandbox` attribute of the `<iframe>` will be set to `allow-scripts` by default, so that javascript can be executed within the frame. If a URL is used, then `allow-forms` is also added to the `sandbox` attribute.

This means that URLs used in the frame can provide a level of interactivity with the referenced system. Make sure that the interaction does not update information in the referenced system, as it will not be synchronised with any updates in the customised extension.

Note: If you are using a URL embedded view, make sure that the session cookies for Trade Innovation and the remote system do not overwrite each other.

Security

For URL-based views, it is expected that security is covered by the SSO environment shared by Trade Innovation and the system referenced by the URL.

Note: This doesn't include the embedded CAS server as that is not currently available to be used by a third party system.

For content-based view, any utility service that interacts with an external system is responsible for covering security.

Initialisation Class

If an initialisation class is declared, then it must implement the `CustomisationInitialiser` interface. This has two methods:

- contextInitialized()
- contextDestroyed()

These methods will be called during application startup and shutdown respectively.

The initialise method is the last one called, so the zone details and deployment contexts should be available if needed.

For more information on the contexts, refer to the SDK Common Facilities Guide – Trade Innovation.

External System Interaction

It may be necessary to call out to an external system in order to display something in context, or to update customised fields with values returned from that external system.

One way that this could be achieved is during the `onValidation()` method, when the relevant required data is available in order to call the external system. The remaining question is whether there is a need to call the external system again. This may be the case if the source data required to make a call to the external system has changed.

Two features have been provided to expand the conditions under which a call to an external system is made. They are:

- using a button
- using a hyperlink

A button click can be used to trigger a call to the external system if the required data is available, and can be used to repeat the call on demand.

Constructing a hyperlink that references a URL whose contents will be opened in another browser window or tab also provides the means to show contextual information (e.g. a signature image). The constraint on the URL would be that the security access needs to be compatible with the same single-signon realm as Trade Innovation.

Note: Buttons and hyperlinks are only available with the *Standard* or *Advanced* SDK.

Calling an External System

The code to support calling an external system is expected to be defined in the hand written code provided in the customisation project, as the type of call required may differ from system to system.

It is possible to use configured transport client components already part of the system to help with the communication.

For further details on transport client components and message formats, refer to the *SDK Systems Interfacing Guide – Trade Innovation*.

The interface components are registered in the configuration as a spring bean under the identifier of `<service>.<operation>.transport.client`. The service name of `Custom` is reserved, and can be used by users of the customisation facilities. The operation values are unique within the service, and so can be allocated as needed.

The customisation code is still responsible for creating the XML message to pass to the `TransportClient` and interpret the response XML message.

So the steps involved would be:

- Lookup the transport client component using `Custom.<operation>.transport.client`

- Create the `ServiceRequest`-based XML message with the required payload
- Call the `invoke()` method, passing the message
- Parse the returned `ServiceResponse` message to extract any required information to set customised fields

The payload of the request and the response should be based on the operation name chosen, using `<operation>Request` and `<operation>Response` as the root elements.

For more details on looking up spring beans, refer to the SDK – Common Facilities Guide – Trade Innovation.

Message Processing

For those associations in an extension point that allow customised data to be provided on incoming gateway messages, the processing of auto submit methods, and validation methods need to be thought through depending on the complexity of data involved.

Customised fields on an incoming gateway message are all defined as subfields of the `<ExtraData>` tag. The subfields take the form of the logical names defined. So for instance if you have a field called `CategorisationCode`, then the `<ExtraData>` section of the message would look like:

```
<ExtraData>
    <CategorisationCode>XXX</CategorisationCode>
</ExtraData>
```

For repeating data, then the repeating field name is used as a containing element. So for the repeating field `ContactDetails` with subfields of `ContactName` and `DateContacted`, the `<ExtraData>` section would look like:

```
<ExtraData>
    <CategorisationCode>XXX</CategorisationCode>
    <ContactDetails>
        <ContactName>XXXX</ContactName>
        <DateContacted>YYYY-MM-DD</DateContacted>    </ContactDetails>
    <ContactDetails>
        <ContactName>XXXX</ContactName>
        <DateContacted>YYYY-MM-DD</DateContacted>
    </ContactDetails>
    ...
</ExtraData>
```

Note: Those fields whose control is a multi-item select list, the value is specified as a comma-separated list.

If customisation fields have been mapped to Open API end points using the customisation editor, then, if required, the customisation fields can be provided in the `custom-fields` tag of the JSON payload of the Open API messages. The customisation data format is JSON and the payload must adhere to the JSON schema returned in the call to the appropriate `custom-fields-schemas` API as published in the Open API swagger document.

An example of a customisation fields payload in an Open API message is shown below:

```
    "custom-fields": {
        "StringField": "XXX",
        "NumberField": 100,
        "BooleanField": true,
        "DateField": "YYYY-MM-DD",
        "RepeatingField": [
            {
                "StringField": "XXXX",
                "DateField": "YYYY-MM-DD",
                "BooleanField": false
            },
            {
                "StringField": "YYYY",
                "DateField": "YYYY-MM-DD",
                "BooleanField": true
            },
            {
                "StringField": "ZZZZ",
                "DateField": "YYYY-MM-DD",
                "BooleanField": false
            }
        ]
    }
```

When received, this data will be:

- Loaded into the extension pane
- Any relevant auto-submit methods will be called (if data is supplied for fields that are marked auto-submit)
- For each repeating set of fields:
  - Each row of data will be loaded into the relevant view pane
  - Any relevant auto-submit methods will be called
  - The row of data will be updated to the entity wrapper
- Non-repeating data updated in entity wrapper

If Trade Innovation invokes validation, then this validation will be performed on the main non-repeating extension fields only by default.

Since validation of a row when entered via the user interface is based on rows being built up, rather than all of the rows being added at once, the validation during message processing may need to be different. If you wish to validate all of the associated repeating data for the message, then a utility method is provided on the extension pane called:

```
validateRepeatingFieldData(ValidationDetails validationDetails)
```

This may be called from the `onValidate()` method of the extension handler.

If you wish to condition this call to only be used for message processing, then call the method `isInMessageProcessing()` on the extension pane to use as a condition.

This utility method could be called unconditionally – however it is not called automatically.

Button action methods are not called automatically – these would need to be called explicitly if required during the `onValidate()` method – though again the code could be conditioned based on whether message processing was being performed. This also applies to the individual rows for repeating data – the overridden `onValidate()` methods on the `ExtensionViewWebPane` should call the appropriate button action methods – though this would only happen if the utility method described previously were actually used.

For *EventStep* message handling – there should be little or no validation, as the data is information from an external system, and is therefore correct – Trade Innovation would not have a say as to whether the data was valid.

Note: For the customer-based messages, the sections are `<CustomerExtraData>` and `<TICustomerExtraData>`

Handling Exceptions

It is entirely up to the user as to the level of exception handling they wish to implement in their extension classes. All overridable methods from the extension handler or extension pane are always surrounded by exception handling methods from the generated java code. In this way the user can just perform the basic checked exception handling required by java to ensure compilation, or they are free to add their own exception types and throw these to the calling methods when required.

Building Your Customised Components

This chapter explains how to build a project, once you have completed defining the static tables, extra data fields and associated user interfaces it will use.

Generating the Code

If you wish to just generate the code from the customisation definition, then use the button on the overview tab on the customisation editor. This will generate everything, and refresh the project, automatically compiling the generated java.

This is useful during the development of any extra java code.

Deploying the Customisation

The `build.xml` file provided in the project, has two main targets `deploy` and `undeploy`.

The `deploy` target will re-generate everything, compile both the generated and hand-written java, and package the classes into a jar file.

This jar file, together with the other resources are copied into the customisation module in the software bundle configured for the project. The location of which is taken from the `project.properties` file.

Once this has occurred, the software is ready to be assembled and deployed in the usual manner.

The `undeploy` target will examine the software bundle customisation module, and will restore the default components, to effectively remove any customisation.

Note: A feature has been added to ensure that repeating fields in the customisation project will only make available the ones that are defined to the event rather than all defined within the customisation project's extension.

Database Scripts

The generation of database change scripts is driven by the snapshots that have been taken and the current `customisation.xml` file.

A version number is allocated to each snapshot, and the current version is incremented. So to start with the customisation.xml file has a snapshot version of 1. When a snapshot is taken, the snapshot keeps the version number 1, and the version of the current customisation.xml file is set to 2. This continues as snapshots are taken.

Based on the snapshot version, database change scripts are generated as follows:

- For the snapshot with version 1, generate drop/create statements for each table
- For subsequent snapshots, generate version-based changes comparing this version with the previous
- Finally, compare the last snapshot and the current customisaation.xml, and generate version-based changes accordingly

The scripts are generated into the `src/gen/changes` folder.

If we have taken a single snapshot, then the folder structure would be as follows:

```
/changes
    /customisation
        /new
            changelog-new.lb.xml
```

```
/v002
    changelog-v002.lb.xml
    changelog-v002-data.lb.xml
changelog-main.lb.xml
changelog-data.lb.xml
```

The tables are dropped and created in `/new/changelog-new.lb.xml`, and any differences between the first snapshot and the current `customisation.xml` are in the `/v002/changelog-v002.lb.xml` file.

The customisation control data is generated in the changelog-data.lb.xml file. This control information replaces what was there before, so it doesn't apply to the versioned scripts.

If you don't have a snapshot, then you would only get the `/new` folder generated.

When this is deployed, the `/customisation` folder is copied to the `/database-scripts/changes` folder and can be executed using the `dbmaintain` tool. For further details on `dbmaintain` see the *Database Management and Upgrade Guide – Trade Innovation*.

When you run the `dbmaintain` tool, you are likely to use the configuration file set up for the zone that you need the scripts to update, so the command would look like:

```
dbmaintain customisation update -propertyFile configuration/zone.properties
```

If you have more than one zone, then it is likely that multiple zone property files have been defined, so chose the appropriate one.

If you already have customisation deployed, and this is the first time using `dbmaintain` for customisation, then the approach is as follows:

- Locate the customisation project that corresponds to your current deployment
- Take a snapshot
- Re-deploy
- Run the `dbmaintain` tool, but use the sync command rather than update
    - This synchronises the database changes in the target database with your project – so any future enhancements will lead to the tables being updated rather than dropped and recreated.
- Run the `dbmaintian` tool using the update command
    - This will skip all of the structure-based SQL, and simply replace the control data (which runs every time you update)

Note: When the zone schema is created, a number of the EXT tables are created as part of this process. This includes EXTMASTER and EXTEVENT. The customisation tables are not part of a schema in their own right, they are part of the Zone schema. Consequently, this should be considered as an update and not an initialisation.

Worked Example

If you create a new customisation project and take a snapshot, and deploy, then you simply get a set of drop/create statements in the `/new` changelog. The control data will simply be the initial customisation version being updated in the `CHPFVER` table.

If you now add a field to an extension point, and then re-deploy, there will be in addition to the original `/v002` changelog file that will contain a directive to add the column.

Enhancing a Table Upgrade

A version-based changelog will add everything to start with:

- New tables
- New columns
- If a column size or type is maintained, then the old column is renamed with '_OLD' as a suffix and the original one added with the new size or type
- A data changelog is included at this point, which is generated with data assignment SQL statements for where a column size or data type has changed
- Redundant columns (including old maintained ones) are dropped
- Redundant tables are dropped

If you want to do some enhanced data manipulation, then copy the data changelog to `/src/main/change/customisation/vNNN` and update it. When you deploy, this will overwrite the generated one.

The format of the file is LiquiBase XML format. For more detailed information on what directives can be defined, refer to the *`liquibase.org`* website.

End of Day User Control Actions

A User Control Action is an object that can be schedules as an action as part of an end of day for a daily processing cycle.

Before a user control action may be defined in the Batch Setup application, the code behind the action must be defined and registered as a spring bean.

This chapter describes what is involved in creating an action.

Implementing a User Control Action

The java class must implement the com.misys.tiplus2.enigma.job.BatchUserExitAction interface:

```
public interface BatchUserExitAction {

    public UserExitActionStatus run(Long stepId, String stepName,
            StepContext context);

    public UserExitActionStatus checkStatus(Long stepId, String stepName,
            StepContext context);
}
```

`UserExitActionStatus` is an enumeration that can take the values `EXECUTING`, `COMPLETE` and `FAILED`.

The `run()` method will be called by Trade Innovation to launch the user control action. Unless the task is quick to run, it should be launched asynchronously so that the `run()` method can return quickly.

If the `run()` method returns `UserExitActionStatus.EXECUTING`, Trade Innovation will then monitor the task by calling the `checkStatus()` method at regular intervals. This method should be able to query the task and return its current status to Trade Innovation.

Returning `UserExitActionStatus.COMPLETE` from either the `run()` or `checkStatus()` method will allow Trade Innovation to proceed with the end of day.

The `StepContext` object provides access to information defined for the job and step associated with the end of day. It has two methods, `getAttribute(String name)`, and `addAttribute(String name, String value)`. For example, to get the processing cycle identifier, use the attribute `ProcessingCycle` – for the phase of end of day, use the attribute `phaseCode`.

Note: The previous version of this interface – `UserExitAction` – is still available and may still be used.

Configuring the User Control Action

Once the class is created, it should be included in the local module as part of the application assembly process. See the *SDK Common Facilities Guide – Trade Innovation* for more details.

The java class should be packaged in a jar file, and a spring configuration file created to represent the class. The spring file should look something like:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans-2.0.xsd"
       default-lazy-init="false">

   <bean id="<User Exit Name>.user.exit"
         class="<full name of java class>"
         scope="prototype"/>

</beans>
```

where `<User Exit Name>` name is a name for your user exit task (max 10 characters).

Include your jar file in the `/local` module and edit the `build-tiplus2.xml` file to register the spring bean, by including the spring file in the application context:

```
    <springbridge file="${tiplus2.folder}/conf/path/conf/enigma.bridge.spring.xml">
        <import resource="classpath:conf/enigma/<Your Spring File>.xml"/>
    </springbridge>
```

This user control action can then be configured in the **Batch Setup** application using the **Custom control actions** option to add your user control action definition. Use the **Batch phase / Action mapping** option to map the user exit into a phase.

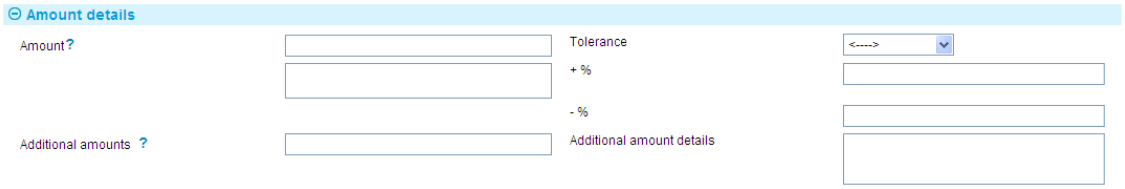
Two example tasks called `TESTUE` and `FAILUE` are predefined in the system. The `TESTUE` task runs successfully and the `FAILUE` task will fail try adding these into your system to help understand the configuration.

Translating Amounts

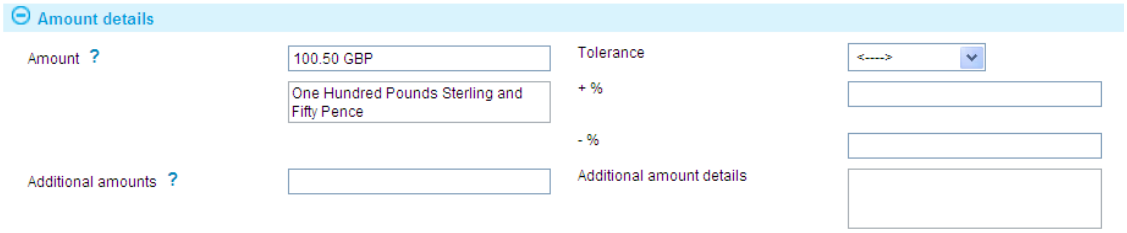
This chapter provides instructions on developing a plug-in to allow the text description of amounts to be displayed in a language other than your system's default language.

Overview

At various points during transaction processing the Amount Details panel is displayed. It is used to enter the amount, tolerance and tenor of a transaction. Amounts are entered as a figure, followed by the three-character code of the currency. A browser button allows the input clerk to select the currency.



Once the amount and currency are entered, when you press OK or Validate, Trade Innovation displays a text description of the amount and currency beneath these fields. The input clerk can edit this text, if necessary.



The wording used to describe the major currency unit and the minor currency unit is supplied from code descriptions held in the Trade Innovation database. (In the above illustration the major currency unit is 'Pounds Sterling' and the minor currency unit is 'Pence'.)

The text description is provided by a default implementation. Your bank, however can configure Trade Innovation to use a different implementation.

It is also possible to provide language-based implementations which will be used when producing documents, based on the language code used when creating the document.

The rest of this chapter explains how to implement a plugin to provide this extra functionality.

Implementing the Plug-in

To provide a language-specific implementation of this functionality, a java class must be written that implements the following interface:

`com.misys.tiplus2.enigma.external.interfaces.sayas.SayAs`

This interface defines a single method:

```
public String sayAsWords(final double majorUnits,
                         String currencyCode,
                         final int numberOfDecimals,
                         final String currencyMajorUnit,
                         final String currencyMinorUnit);
```

The parameters are:

Parameter	Description
majorUnits	Amount to convert to words expressed in major currency units
currencyCode	Currency mnemonic
numberOfDecimals	Number of decimal places for minor currency units
currencyMajorUnit	Description of the currency major unit
currencyMinorUnit	Description of the currency minor unit

The returned string is the amount converted to words.

The jar file `com.misys.tiplus2.enigma.jar` must be included on the plug-in's compilation classpath in order to pick up the interface definition.

Note: Ensure that the encoding of the result is correct, and that the fonts set up for PDF production are capable of supporting any characters output.

Configuring the Plug-in

Trade Innovation will load the plug-in based on the language code. The spring bean with the id `ti_say_as.[Language]` will be loaded. If it is not found, then the default will be used (`ti_say_as.GB`).

For further information on adding components to the application assembly process, refer to the *SDK – Common Facilities Guide – Trade Innovation.*

To configure the new plug-in, you need to supply a spring definition based on the language supported. This spring file must be included in the standard configuration, and the jar file contained the implementation included in the deployed software.

The jar file should be placed in the following folder:

`/components/modules/local/lib`

The spring file should be placed in the following folder:

`/components/modules/local/conf/path/conf/enigma`

Make sure that the name of the spring file does not clash with an existing file.

Review the following file:

`/components/modules/local/build-tiplus2.xml`

This ant file should already contain the steps to copy the contents of the configuration and lib folders to their appropriate location. To include the spring file in the standard configuration, use the `<springbridge>` ant task as follows:

```
<springbridge file="${tiplus2.folder}/conf/path/conf/enigma.bridge.spring.xml">
    <import resource="classpath:conf/enigma/<your-spring-file>.xml "/>
</springbridge>
```

If your jar file also contains an alternate default implementation, then add the following `<alias>` tag to the `<springbridge>` task:

```
...
    <alias alias="ti_say_as_default" name="alternate-default-implementation"/>
...
```

substituting the bean ID declared in your spring configuration file.

Make sure that the `local.overrides` property is set to yes in the appropriate `tiplus2.configuration.properties` file. The application EARs can then be rebuilt and deployed.

Overriding Default Currencies Example

Trade Innovation has a default implementation of the SayAs plugin for Arabic. The implementation supports a default number of currencies that are converted to Arabic words.

To override the default set of Arabic currencies, follow the steps:

- Create a properties file in software/components/modules/local/conf/path. The file should be named <language code>.currencies.properties, so, for Arabic, it will be ar.currencies.properties.
- Override the plugin's spring configuration by `software/components/modules/local/conf/path/conf/enigma/sayAsAR.xml` with the below content

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi=http://www.w3.org/2001/XMLSchema-instance
       xmlns:util="http://www.springframework.org/schema/util"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd
       http://www.springframework.org/schema/util
http://www.springframework.org/schema/util/spring-util-2.0.xsd">
    <bean id="ti_say_as_arabic" class="com.misys.tiplus2.enigma.lang.sayas.SayAsAR"
scope="singleton">
        <property name="currencies">
            <util:properties location="classpath:ar.currencies.properties"/>
        </property>
    </bean>
</beans>
```

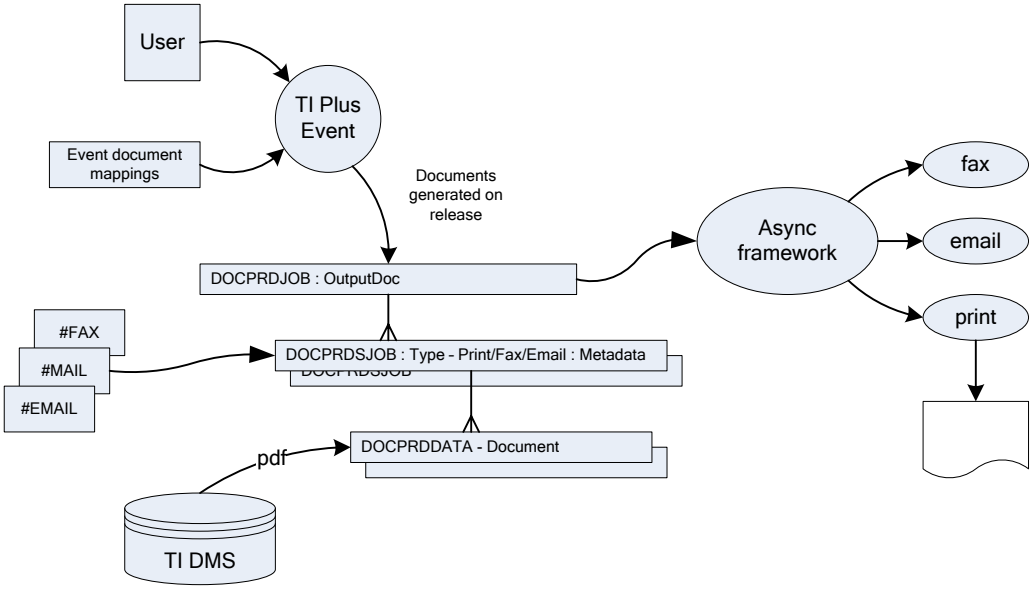
Rebuild the software ears with `local.overrides=true` set in the `tiplus2.configuration.properties` file of the zone access application.

Developing a FAX Interface

This chapter provides information on developing an outgoing FAX plug-in for Trade Innovation.

Overview

When print, fax or e-mail documents are produced by Trade Innovation the relevant `.pdf` documents are stored in the document management system (DMS) you run in conjunction with Trade Innovation and the details written to the `DOCPRDJOB/DOCPRDSJOB` and `DOCPRDDATA` tables that hold metadata relevant to the production method.



The async framework then manages the output of these documents and passes the document references, printer details and metadata to the relevant service. These details can then be used by the relevant service to output the documents via the chosen production method.

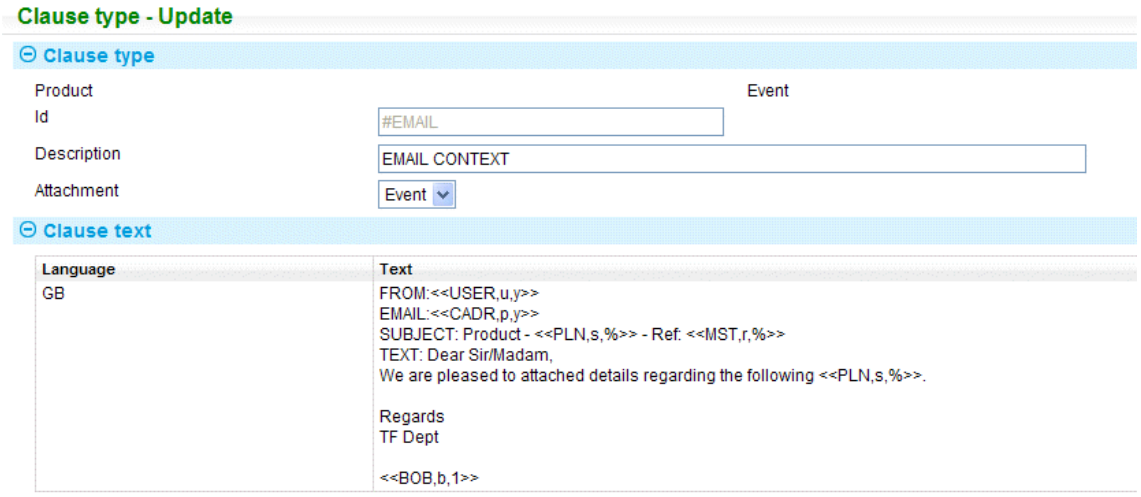
A standard print service and an optional email service are supplied; however the fax service needs to be implemented according to your bank's specific requirements.

Before you Begin

The following steps need to be performed as a preparation for developing an outgoing fax plug-in for Trade Innovation:

- Use Trade Innovation's system tailoring facilities to configure the document mapping rules that produce the required release items, see the *System Tailoring User Guide – Trade Innovation*.
- Use Trade Innovation's system tailoring facilities to set up the relevant document context metadata field (using special clauses #FAX, #EMAIL and #MAIL). This includes any required event fields such as user identifier, FAX number, and master document reference number. Trade Innovation will perform substitutions based on these special clauses and assign values to metadata at run-time. The exact format of the metadata field is driven by what the implemented plug-in needs it to be.

For example, the illustration below shows the special clause #EMAIL being set up to include event fields.



These are resolved at run-time as follows:

```
FROM:user@misys.com
EMAIL:customer@abc.co.uk
SUBJECT: Product - Import Letter of Credit - Ref: ILC00001583
TEXT: Dear Sir/Madam,
We are pleased to attached details regarding the following Import Letter of Credit.

Regards
TF Dept

MISYS BANK LTD
```

Create a clause type of #FAX, holding the information relevant to interfacing to the chosen FAX software.

Developing the FAX Plug-in

The class created for the plug-in needs to implement the interface:

```
com.misys.tiplus2.print.process.DocumentControlModule
```

This interface has a single method called *process* that accepts a *DocumentProductionJobDetail* object that includes the metadata and a list of documents represented as *DocumentProductionDocumentDetail* objects.

A `DocumentProductionDocumentDetail` object allows you access to the document from the DMS where it is stored. This means you will have access to a binary version of the document as well its mime-type.

The rest of the code needs to transfer these documents to the configured FX software to manage the faxing of these documents.

If a problem occurs, It is possible to signify if an error occurred processing a single document, or the set of documents by invoking the `setStatusFailed()` method on the individual document object, or the `setStatusFailedForAll()` method on the job object.

This allows the user to re-submit based on the details of the failure from the **Jobs** menu.

Note: This project will require the `com.misys.tiplus2.print.jar` file in its classpath.

Configuring the Plug-in

Once the plug-in is created it and packaged in a jar file, it can be included with the Trade Innovation software by using the */local* module in the following way. For more information see the *SDK – Common Facilities Guide – Trade Innovation*.

Place the jar file containing the plug-in in the */local/lib* folder.

Create a new spring configuration file (for example *example.fax.client.spring.xml*) in the */local/conf/path/conf/printing* *folder*. The file should include the following bean definition:

```
...
<!-- Control module for: FAX -->
<bean id="fax.control.module" class="<classname of plugin>" scope="prototype"/>
...
```

The `build-tiplus2.xml` file in the */local* module will copy the jar file and the spring configuration file. The file needs to be updated to reference this configuration file, and set the new plug-in as the FAX document production service using the *<springbridge>* ant task by adding the following:

```
<springbridge file="${zone.folder}/conf/path/conf/printing.bridge.spring.xml">
    <import resource="classpath:conf/printing/example.fax.client.spring.xml"/>
    <alias alias="F.control.module" name="fax.control.module"/>
</springbridge>
```

After completion of the above steps, whenever the Trade Innovation application is assembled and deployed the FAX plug-in will be used where appropriate.

Controlling Document Windows

When Trade Innovation generates a hyperlink that allows a user to view a generated or attached document for an event, javascript functions are invoked to keep track of the windows opened.

A configuration option is available to force all document windows opened to be closed once the respective master summary page is exited from. To activate this feature, add the `override.document.window.recording` property to the appropriate `tiplus2.configuration.properties` file.

It is also possible to override the javascript functions that open the window that will retrieve the document, and the method that will close the windows that have been opened.

All standard functionality is implemented in the javascript file js_window_mgr.js. Calls are made to two functions:

- `win_controller_open_window` – this is passed a reference to the main window, the target URL, the target window name and the title to use for the window.
- `win_controller_close_windows` – this is passed the array of windows that have been opened

These functions are defined in the `js_window_mgr_controller.js` file, and they simply call the default implementations in the `js_window_mgr.js` file.

If these methods need to be overridden, then a localised version of this file can be created, and overwrite the supplied one during application assembly. This is achieved by including the new version in the `/local` module. For more details refer to the *SDK Common Facilities Guide - Trade Innovation*.

The `js_window_mgr_controller.js` is provided simply as a linkage file, and so can be overwritten.

Using an External DMS

Trade Innovation provides a facility for interacting with an external DMS. This can be in the form of:

- Java implementation
- Web Service implementation

This chapter will describe what would be required for a Java implementation. For details on the web service implementation see the *SDK Systems Interfacing Guide - Trade Innovation*.

How Trade Innovation Uses a DMS

Trade Innovation stores the following type of artefacts in a DMS:

- Attachments – uploaded in an event
- Documents – generated as part of an event
- Reports – ad-hoc or during end of day
- Templates – uploaded explicitly

As well as the artefact itself, Trade Innovation associates meta-data with it.

Note: When an attachment or document is first uploaded to a DMS, it has minimal meta-data, so any mandatory meta-data required by the DMS being interfaced to is the responsibility of the interface to provide defaults for.

Attachments and Documents may be configured through the branch hierarchy, while Reports and Templates will use the embedded DMS, unless the system options *ExternalReportingDMSID* and *ExternalTemplatesDMSID* are specified respectively.

Note: Templates are specific to Trade Innovation, and it is likely that they will always use the embedded DMS.

Trade Innovation uses the concept of attaching to a document and detaching from it. So when adding an attachment to an event, there are two operations performed - *upload* followed by *attach*.

The purpose of *attach* is to signify to the DMS that there is a transaction in Trade Innovation with a reference to that document. Once that transaction is archived, or in the case of a document that is re-generated due to data changing in an event, then the old linkages are *detach'*ed.

This provides information to the external DMS that something is interested in keeping this item in the DMS, and so can prevent the DMS from archiving the item independently.

An implementation of this interface may choose to equate *detach* as being 'delete', though it may be more appropriate to allow detached items live in the DMS for at least a short period of time. Especially considering the following scenario:

- If an item is uploaded as an attachment to an event, and subsequently during review, it is discovered that the attachment is actually for the wrong event, then it may be detached from this event and attached to the correct event. If the *search* facility is being used, it is looking for detached items – those that have not yet been associated with an event. If *detach* is treated as delete, then the search mechanism is not required, as there will be no transient detached items.

Allowing detached items to exist in the DMS may require an end of day user exit step to be created to perform some cleanup – or there may be facilities in the DMS software itself to purge those items on a daily basis.

Once an event progresses, meta-data associated with attachments or documents will be updated according to the data as it changes.

Generated documents are uploaded in a similar way the first time – they are uploaded and then attached to. When they are regenerated, the default processing is to upload the new document, attach to it and detach from the old one. It is possible for the regenerated document to replace the previously generated document within the DMS, retaining the same document id – however since this behaviour may not be supported by external DMS systems, the capability is optional.

If the DMS you are building an interface for does not allow meta data to change, then it should be written based on gateway document messages. It is possible to design a gateway message document that when a transaction is authorised and released, a message will be produced that contains the attachments and generated documents as base64 encodings, together with whatever event data is appropriate for the meta data required by the external DMS. This means that documents and attachments continue to exist and be referenced in Trade Innovation using the embedded DMS, and are simply copied at the point when the document content and meta-data cannot change.

The expectation is that the DMS facility being used provides the means to view a document in response to a URL being used, and the security aspects of doing this are covered by the sharing of the single-signon configuration of both products.

The Java Interface

The interface to a DMS is represented by:

`com.misys.tiplus2.facilities.content.repository.ContentRepository2`

An item in that repository is represented by the interface:

`com.misys.tiplus2.facilities.content.repository.ContentRepositoryItem`

ContentRepositoryItem

A `ContentRepositoryItem` represents an item in the DMS. It provides an option of deferring the actual document from being loaded until directly being accessed.

The methods on `ContentRepositoryItem` are as follows:

`String getDocumentID()`

Returns the unique identifier of the item in the repository

`InputStream getData()`

Returns the content of the item as a `BinaryInputStream`

`String getMimeType()`

Returns the mime type of the item as a string

`Map<String, String> getMetaData()`

Returns the meta data associated with the item

Each of these methods has an appropriate set method.

The methods `setUnique()/isUnique()` are no longer used and will eventually be deprecated.

Meta-Data

The following items of meta-data are used for attachments and documents:

Metadata name	Description
PRODUCT	Product code and long name e.g. ELC-Export Letter of Credit
EVENT	Event code and long name e.g. ADE-Advise
CUSTOMER	The transaction customer mnemonic. (Principal party)
SBB	The transaction customer source banking business mnemonic
BRANCH	The transaction behalf of branch
TEAM	The responsible team for the current event
OUR_REFERENCE	The master reference
THEIR_REFERENCE	Their event reference
EVENT_REFERENCE	The event reference number e.g. ADV001
DOC_TYPE	Document type
DESCRIPTION	Document description
DOCUMENT_CATEGORY	The document category • Attachment • Generated • Report • Template
DOC_FACE_REF	Document face reference
BANK_DOCUMENT	Is an internal bank document • Y • N
ZONE	The zone id
FILE_NAME	The file name
FINAL_FORM	Identifies whether this is the final form of the document • N

For reports and templates, the only meta-data used is:

- FILE_NAME
- FINAL_FORM

The metadata values are available as a java enum called `ContentRepositoryMetaData`.

ContentRepository2

The methods on `ContentRepository2` are as follows:

```
ContentRepositoryItem createContentRepositoryItem()
```

Returns an initial instance of the `ContentRepositoryItem` used by this implementation of `ContentRepository2`

```
String upload(ContentRepositoryItem item)
```

Uploads the item to the DMS, and returns the allocated identifier in the DMS

```
Boolean replace(ContentRepositoryItem item)
```

Replaces the document stored, keeping the same document id (optional functionality – isReplaceable() must return true for this to be used)

```
Boolean attach(ContentRepositoryItem item)
```

Registers interest in the document – indicates there is a reference to that document within Trade Innovation – return true if successful

```
Boolean detach(ContentRepositoryItem item, DetachedContext context)
```

Indicates item is no longer referenced in Trade Innovation – return true if successful

The `DetachedContext` provides a means of understanding why the document is no longer referenced. Possible values are `EXPIRED` or `REMOVED`.

If it is `EXPIRED`, then this is equivalent to Trade Innovation removing a completed transactions details from the database after its retention period is passed. This also applies to reports that are removed from the appropriate DMS after their retention period.

If it is `REMOVED` it represents a version of a document that is in the process of being replaced.

If it is `UNLINK` it represents a document that may be currently attached to the wrong transaction

```
ContentRepositoryItem retrieve(String documentId)
```

Returns the item referenced – the meta-data should also be populated from the DMS

```
Boolean update(ContentRepositoryItem item)
```

Indicates that it is time to update the meta-data associated with this item – return true if successful

```
List<ContentRepositoryItem> search(Map<String, String> metadata,
                                   DocumentStatus status)
```

Performs a search on the DMS based on the meta data values passed, and returns the items that match. Values passed in the meta data may contain '%', which indicates a wildcard search on that item. The status field indicated what item states are also required in the search. This is an enumeration that can take the values `ATTACHED`, `DETACHED`, and `BOTH`. If no results are found, return an empty list.

```
String view(String documentID)
```

Creates a URL that when clicked will open a browser window or tab and show the content of the item referenced by the `documentID`. If the item referenced does not exist, then return null

```
Boolean isAttached(String documentID)
```

Return true if the referenced document is attached to a transaction, otherwise return false.

`Boolean isReplaceable()`

Return true if the `replace()` method has been implemented – otherwise return false.

Each method has an associated `isXXX()` method that returns a Boolean. The purpose of these methods is to check to see if the operation is supported by the interface. If Trade Innovation calls one of these methods and it returns false – then an error is shown. This is to prevent use of the interface implementation.

If the operations are not applicable to the implementation being written, then return true for the `isXXX()` and provide a default successful implementation to the `XXX()` method.

If you want to prevent Trade Innovation from calling an `XXX()` method at all, then return false for the `isXXX()` method.

The method `isBrowseable()` is used as a means to control whether the interface implementation will support a search with multiple results feature. If not, then the search button on the attachments page in Trade Innovation is disabled.

These methods declare a throws clause of `ContentRepositoryException`. Where more detail can be supplied, then throw an exception, and the message string is used to report to the user that an error has occurred.

The following table details when the methods of the `ContentRepository2` are called.

API	Mailing Details	Document Generation
upload	Uploading any document	Document is generated
replace	N/A	When a document is re-generated (optional – must indicate capability to be used)
attach	On Ok On Pend	During document generation on input step
detach	On Ok On Pend	During document generation on input step
retrieve	Event field access	Event field access
update	On Ok On Update	N/A
search	N/A	N/A
view	Viewing the document	Viewing the document
isAttached	On Ok On Pend	N/A

Registering the Implementation

An External System that implements the DMS document and/or DMS attachment services represents an external DMS. The External System name is stored together with a document id in the database. This identifier is referred to as a DMSID.

Each DMSID required a configuration definition that references the correct implementation for the External System.

Create a spring definition file for your implementation, and include it in the `/local` module in the `conf/path/conf/facilities-content-manager` folder. The jar file containing your implementation should also be included in the `/lib` folder.

In the `build-tiplus2.xml` file, make sure the jar file and spring file is copied to the assembly area, and use the `<springbridge>` task to register your implementation:

```
<springbridge
file="${zone.folder}/conf/path/conf/facilities.content.manager.bridge.spring.xml">
    <import resource="classpath:conf/facilities-content-
manager/<your.DMS.spring.xml>"/>
    <alias alias="<DMSID>.CR.repository" name="your.DMS.implementation"/>
</springbridge>
```

where `<DMSID>` is the name of the External System used.

If this implementation is to be used for attachments uploaded from Trade Innovation or documents generate by Trade Innovation, then they can be configured using the Branch/Service mapping function in the *System Tailoring* application.

Existing Implementations

Pre-2.6

If a `ContentRepository` component has already been implemented using a pre-2.6 version of the interface, then a bridging module is available to translate calls from the new operations to the old operations.

Create a new spring configuration file called something like `example.CR.spring.xml` in the `/local` module in the `conf/path/conf/facilities-content-manager` folder, then define the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans- 2.0.xsd">
    <bean id="cr.bridge-229"

class="com.misys.tiplus2.enigma.external.interfaces.contentmanager.ContentRepositor
yBridge">
        <constructor-arg index="0" ref="implemented.contentrepository.bean" />
    </bean>
    <bean id="implemented.contentrepository.bean"
class="examples.ContentRepositoryImpl"
          scope="prototype" />
</beans>
```

Update the `build-tiplus2.xml` file in `/local` module to copy this configuration file as well as the corresponding jar file.

To update the existing configuration to use the content repository bridge as the default DMS (instead of the embedded DMS) use the `<springbridge>` ant task to define the alias as follows:

```
<springbridge
file="${tiplus2.folder}/conf/path/conf/facilities.content.manager.bridge.spring.xml
">
    <import resource="classpath:conf/facilities-content-
manager/example.CR.spring.xml"/>
```

```
    <alias alias="ti.CR.repository" name="cr.bridge-229"/>
</springbridge>
```

This will mean your implementation will continue to be used as the default DMS.

Post 2.6

Trade Innovation version 2.6 introduced a standardised external DMS support. This did not include the `replace()` method described above, and the interface was called `ContentRepository`. To support the new feature, `ContentRepository2` extends `ContentRepository`, adding the `replace()` functionality.

If an implementation of `ContentRepository` has been created already to provide external DMS interaction, there is no need to change it to implement `ContentRepository2` – the system will detect that the previous interface is being used, and will provide the default processing behaviour associated with generated documents.

RoutingContentRepository

Included with the software is a sample implementation class called RoutingContentRepository.java is included inside the `sdk/resources/samples/dms` folder

The sample class can be used as a guide for how the FINAL_FORM metadata value to route uploaded documents to one of two DMS implementations. The actual implementations would be injected using spring as `transientRepository` and `permanentRepository`.

Archive Delete

Over time, transactions will become eligible for deletion. This is after a configured retention period has passed once the transaction has been booked off.

Whatever archive mechanism is employed to provide offline historical access to transactions, it may be useful to provide a check that can be made to make sure that a transaction about to be deleted exists in the archived data.

Once Trade Innovation has identified a transaction that is eligible for deletion, it will call out to a configured function to verify if the master can be deleted.

Implementation

To implement a check, write a java class that implements the interface:

```
com.misys.tiplus2.enigma.external.interfaces.data.MasterDetails
```

The only method to implement is:

```
boolean checkMasterDelete(String reference)
```

Perform the necessary logic to check if the master may be deleted. If it can, return `true`, otherwise return `false`.

Create a spring configuration file to reference your class:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans- 2.0.xsd">
    <bean id="my-master-delete" class="example.MasterDelete" scope=”prototype”/>
</beans>
```

Note: The example configuration configures the bean as a prototype, a singleton could be used. However it is possible that multiple branch-level actions during end of day may call this, so make sure it is thread-safe.

Place this file in an `/apps.ti` subfolder of the `/conf` folder. Use the `<springbridge>` ant task in your local module ant script as follows:

```
<springbridge file="${zone.folder}/conf/path/conf/apps.ti.bridge.spring.xml">
    <import resource="classpath:conf/apps.ti/example.master.delete.spring.xml"/>
    <alias alias="check-master-delete" name="my-master-delete"/>
</springbridge>
```

Your module will now be called as part of the archive delete end of day action.

Appendix A Converting an Existing Customisation Project

There is currently no automatic conversion process that will take a customisation project from Trade Innovation 2.2.x and make it available to work at Trade Innovation 2.3.x. However, there has been little change to the runtime mechanism, and the conversion process, though manual, should not take too long.

Customisation Definition

The customisation editor must be used to re-define the customised fields. Make sure all of the extra fields and static data definitions are entered as they were defined before. Note that the UI controls are defined with the fields themselves rather than when defining a layout.

As part of this process it is worth reviewing how the groups of fields within an extension are used. If not required, then for each product/event combination used, create a field group, and for each group define a layout. Note that where previously, interface definitions could be shared – this is now represented by a single layout defined against a field group. The re-use comes when the association is defined.

The hand written java files previously had to be created for the product and event combination through a naming convention. This usually ended up with an intermediary tier in the hierarchy where a common subclass was written with the required extra business logic, and then several simple product/event based classes created to subclass this. With the new editor, these intermediary classes may be referenced directly several times. The build process no longer detects classes via a naming convention – they are explicitly referenced in the association definitions.

For each product/event combination required, create an association and attach a relevant layout, specifying any extension handlers or extension panes.

If you did not originally require a layout for an extension, then still define a layout, but with nothing on it – the fields in the field group are still available to the customised extension code.

Hand Written Code

In the handwritten java, note that the following class names have changed:

`CustomerCustomisedPane`	`CustomerPane`
`EventCustomisedPane`	`EventPane`
`MasterCusotmisedPane`	`MasterPane`
`PostingCustomisedPane`	`PostingPane`
`TICustomerDetailsCustomisedPane`	`TICustomerPane`

The following class names have changed, and the new classes reside in the `com.misys.tiplus2.cusomtisation.extensions` package:

`CustomerCustomisation`	`CustomerExtension`
`EventCustomisation`	`EventExtension`
`MasterCustomisation`	`MasterExtension`
`PostingCustomisation`	`PostingExtension`
`TICusotmerDetailsCustomisation`	`TICustomerExtension`

Any methods used previously that began with an uppercase letter, now begins with a lowercase letter. For instance `GetEventFieldasText()` is now `getEventFieldAsText()`.

The `IValidationDetials` interface is replaced by the `ValidationDetails` class.

All getter and setter method names use the logical field names rather than the column names.

The following methods have been renamed:

`postInitialiseBankAdditionalCode`	`onPostInitialise`
`validatePane`	`onValidate`
`onSavingBankAdditionalCode`	`onSaving`
`AutoEventBankAdditionalCode`	`onAutoEventProcessing`

Once these changes have been made, the customisation project should be able to be re-generated and redeployed.



The future of digital finance can start here.
Consider our planet before printing.

To find out more: visit finastra.com

© 2026 Finastra. All rights reserved

Corporate Headquarters
4 Kingdom Street
Paddington
London W2 6BD
United Kingdom

T: +44 20 3320 5000